ZamZam

SHAZAM FOR MALAYSIAN BIRDS

Building a TinyML Mobile App with Flutter and TensorFlow Lite

- On-Device Audio Recording
- Mel Spectrograms in Dart
- TensorFlow Lite Inference
- Cross-Platform Flutter UI

FLUTTER — One Codebase

ANDROID — Play Store Ready

iOS — App Store Ready

RECORD → ANALYSE → IDENTIFY → SHARE

A HANDS-ON GUIDE TO ECO-AWARE SMART APPS.

Contents

Chapter 1

Introduction to Mobile Bird ID Apps

Learning Objectives

- Appreciate bird watching as one of the world's largest citizen science activities, and understand the friction that limits its scientific yield.
- Understand how Shazam identifies a song from a short audio clip, and why the same trick does not translate directly to bird sounds.
- Recognise the specific limitations that make a general-purpose music ID app the wrong tool for a birder in a Malaysian rainforest.
- Survey the current landscape of bird ID apps (Merlin, BirdNET, eBird) and identify the gap a Malaysian-focused app fills.
- Understand the pitch of **ZamZam**: a lightweight, offline-friendly, Malaysian-focused bird ID app.
- Compare running a bird ID model on a phone versus on a microcontroller.
- See the shape of the technology stack this book will build — Flutter, TensorFlow Lite, on-device audio.

1.1 Motivation: From Birding to a Bird Shazam

Bird Watching as Citizen Science

Bird watching—or "birding"—is one of the world's largest citizen science activities. Millions of enthusiasts record bird sightings, generating biodiversity data on a scale unattainable by professional researchers alone. Since 2002, the Cornell Lab of Ornithology's eBird platform has accumulated more than 1.6 billion observations from over 900,000 contributors, making it the world's largest biodiversity-focused citizen science project. In Malaysia, organisations such as the Malaysian Nature Society conduct regular bird counts and migration surveys that provide invaluable long-term monitoring data. With more than 850 recorded bird species [1, 2], Malaysia also offers abundant opportunities for bird discovery.

These observations hold substantial scientific value, having been used to quantify population declines, document climate-driven range shifts, and detect invasive

species. In well-surveyed regions, a dedicated birder can contribute more observations in a single year than many funded field studies produce over several years.

The Notebook Workflow

Birders have traditionally used a simple field kit: binoculars, a field guide, and a notebook for dates, locations, species, counts, and behaviors. After returning home, notes are transferred to personal life lists and uploaded to databases like eBird.

This workflow is reliable but has limitations:

- **Identification bottleneck.** Birds calling from the forest canopy are often heard but not seen, leaving observations marked as "sp." (species unknown). Accurate audio identification requires extensive experience or a reviewable recording.
- **Transcription overhead.** Handwritten notes must be manually digitised, species names standardised, and locations reconstructed. This extra effort delays submission, and many observations are never uploaded.
- **Limited verification.** Once written, an identification is difficult to verify. Rare-species records that could be confirmed via an audio recording must instead rely solely on the observer's written notes and expertise.

Motivation for a Mobile App

Smartphones eliminate these limitations by capturing audio, logging GPS, performing on-device identification, and streamlining uploads. Existing apps address parts of this workflow but none fit Malaysian acoustic conditions—dense, overlapping tropical vocalizations.

Mobile-first identification at point of observation reduces post-hoc annotation and expert reliance. This matches modern practice: birders now use phones for navigation, photography, and checklists. Automated detection reduces friction and encourages consistent recording, especially for casual observers.

Mobile apps also provide immediate feedback with real-time suggestions. Critically, on-device inference works in remote forests without network connectivity.

Bridging these practical needs—field usability, ecological specificity, and accessibility—motivates the development of a dedicated Malaysian bird sound recognition app.

The idea of identifying an unknown sound with a smartphone is hardly new. For millions of users, the benchmark is Shazam, an application capable of recognising a song after listening for only a few seconds. If a phone can identify music almost

instantly, it is natural to wonder whether the same approach could identify birds as well. That seemingly simple question provides the motivation for the rest of this chapter.

The App That Inspired This Work

Figure 1.1: The Shazam application icon. Image: Wikimedia Commons, file `Shazam_icon.svg`, in the public domain (below threshold of originality). "Shazam" is a registered trademark of Apple Inc. / Shazam Entertainment Ltd. and is used here for identification only.

Most have used **Shazam** (Fig. 1.1). Hear a song, tap the button, hold the phone up, and Shazam returns the title and artist—it feels magical. The mechanics are simpler: listen to 10 seconds, build a compact *acoustic fingerprint* from Mel spectrogram peaks, and look it up in a cloud database of pre-fingerprinted songs [3]. Owned by Apple since 2018, Shazam runs on all major platforms and handles over a billion identifications monthly.

Shazam is a useful benchmark for the experience we want ZamZam (both the app and this book) to deliver: *one tap, a few seconds of listening, and a confident answer*. However, the similarity largely ends with the user interface. Underneath, bird identification poses a very different computational problem. To understand why, it is helpful to first examine how Shazam actually works before exploring why the same approach breaks down for birdsong.

1.2 How Shazam Works, and Why It Fails on Birds

The Music-ID Pipeline in One Page

1. **Record** ∼10 s of audio at 44.1 kHz.
2. **Spectrogram.** Compute the Short-Time Fourier Transform, producing a time-frequency image.

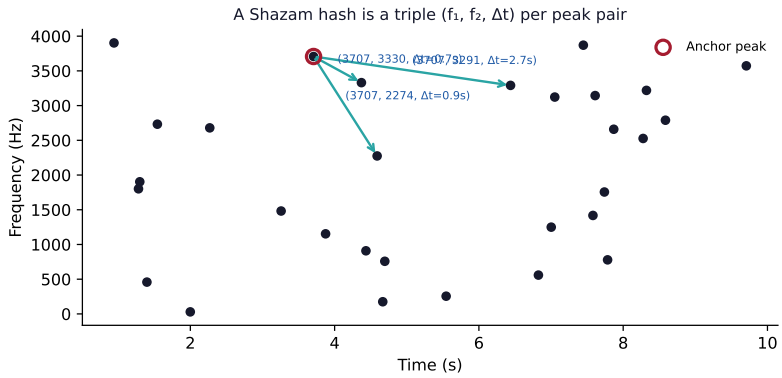

Figure 1.2: Shazam's fingerprint: an anchor peak is paired with a small number of peaks in its *target zone*. Each pair becomes a hash triple $(f_1, f_2, \Delta t)$. Thousands of such hashes per song enable robust, noise-tolerant matching against the catalogue.

3. **Peak picking.** Keep only the local maxima — these are the loudest, most distinctive time-frequency points.
4. **Hashing.** Pair up nearby peaks and turn each pair into a compact hash triple $(f_1, f_2, \Delta t)$. A 10-second clip yields a few thousand hashes.
5. **Database lookup.** Send the hashes to a cloud service that has pre-fingerprinted every song in the catalogue the same way. Whichever song has the most matching hashes at consistent time offsets wins.
6. **Return** title, artist, album.

Elegance lies in the fingerprint (Fig. 1.2). Each peak is an *anchor*. The algorithm looks ahead in time within a frequency range—the *target zone*—and pairs the anchor with nearby peaks. Each pair becomes a triple: two frequencies and the time offset. Since the triple encodes only relative structure, the same song produces the same hashes whether recorded from a Bluetooth speaker or tinny radio.

The catalogue lookup is done in a cloud database. This makes Shazam fast when you have signal — and useless when you do not.

Why the Same Trick Fails on Birdsong

Applying Shazam directly to birdsong fails for several interconnected reasons:

Songs are the same take. Birdsong is not. Shazam works because every playback of "Bohemian Rhapsody" is the *same recording*: the same peaks, in the same order, at the same relative times. A Zebra Dove calling from a

mango tree in Kuala Lumpur is a different *utterance* from the same species calling in Ipoh. Same species, different peaks. Fingerprint hashes will not match.

Individual and regional variation. Many species have dialects. A Straw-headed Bulbul in Peninsular Malaysia sings a subtly different phrase from one in Sumatra. Music has no dialects.

Overlapping vocalisations. The dawn chorus is a soup of five to ten species calling at once. Shazam's algorithm assumes one song at a time and gives up when peaks come from multiple sources.

Environmental noise. Rain, wind, cicadas, and traffic all inject broadband noise that the music fingerprint tolerates but the sparse-species fingerprint does not, because bird calls are already quiet.

No offline mode. Shazam needs a network to hit its catalogue. Birders are often in forests with no signal. A useful bird app must work offline.

No local catalogue. Even if we replaced the music database with a bird database, Shazam's cloud service does not know Malaysian species. Existing bird apps like *Merlin* and *BirdNET* are catching up, but coverage of Southeast Asian avifauna is still patchy compared with North America and Europe.

1.3 The Bird-ID App Landscape Today

If acoustic fingerprinting is unsuitable for birds, what do modern bird-identification systems use instead? Rather than matching recordings against a database of fingerprints, today's bird ID apps rely primarily on machine learning models trained to recognise species despite differences between individual birds, recording conditions, and background noise. Although they share this common foundation, they differ considerably in their goals, scope, and target users.

Three apps dominate the space, and each solves a slightly different problem.

Merlin Bird ID (Cornell Lab of Ornithology, 2014). [4] The most popular bird ID app in the English-speaking world. Offers sound ID, photo ID, and a step-by-step interview mode. Sound ID uses a CNN model trained on Cornell's Macaulay Library. Coverage is strongest in North America and Europe. Malaysian species coverage exists but is patchy for Bornean endemics.

eBird (Cornell Lab, 2002). [5] A citizen-science checklist app, not primarily an identifier. Users manually log sightings; the app aggregates them into range maps and bar charts. eBird is where the *observations* go; Merlin is where the *identifications* happen.

BirdNET (Chemnitz University / K. Lisa Yang Center, 2018). [6] The reference model in bioacoustic research. Recognises around 6,000 species globally. The BirdNET Analyzer runs on desktops and Raspberry Pi; a mobile app (BirdNET Sound ID) is available, powered by the same model. Excellent coverage but heavy: the full model is ∼70 MB and inference is slow on entry-level phones.

A pattern emerges. Merlin gives a polished, well-designed experience but is trained on a global catalogue with a North-American emphasis. BirdNET has the best model but is heavy and produces a research-flavoured UI. eBird complements both but does not identify. None of them is optimised for a Redmi-class Android phone running on a spotty rural connection, and none surfaces local Malay common names alongside the English and scientific ones.

That is the gap ZamZam fills. It aims to identify the top ∼50 species of Peninsular Malaysia and Borneo, chosen for how commonly a first-time birder is likely to encounter them. Rather than promise to identify any bird on Earth, we promise to identify the ones you are likely to actually hear here, quickly, without a signal. Table 1.1 lists the ten most common birds found in KL.

Table 1.1: Top Malaysian urban birds [7]. Kuala Lumpur citizen science data shows these species dominate urban bird surveys, making them natural candidates for a mobile sound classifier.

Common Name	Scientific Name	Abundance in KL
Eurasian Tree Sparrow	*Passer montanus*	Most abundant
Oriental Magpie-Robin	*Copsychus saularis*	Very common
Common Myna	*Acridotheres tristis*	Very common
Javan Myna	*Acridotheres javanicus*	Very common
Rock Pigeon	*Columba livia*	Common
Spotted Dove	*Streptopelia chinensis*	Common
Yellow-vented Bulbul	*Pycnonotus goiavier*	Common
House Crow	*Corvus splendens*	Abundant in some areas
Asian Glossy Starling	*Aplonis panayensis*	Regular

1.4 What ZamZam Will Do Differently

We keep Shazam's user-facing promise — *tap, listen, answer* — and change almost everything under the hood. Figure 1.3 shows the two pipelines side by side.

Five design decisions set ZamZam apart from a simple Shazam port to birds:

- **Deep-learning classifier, not fingerprint matching.** We train a compact

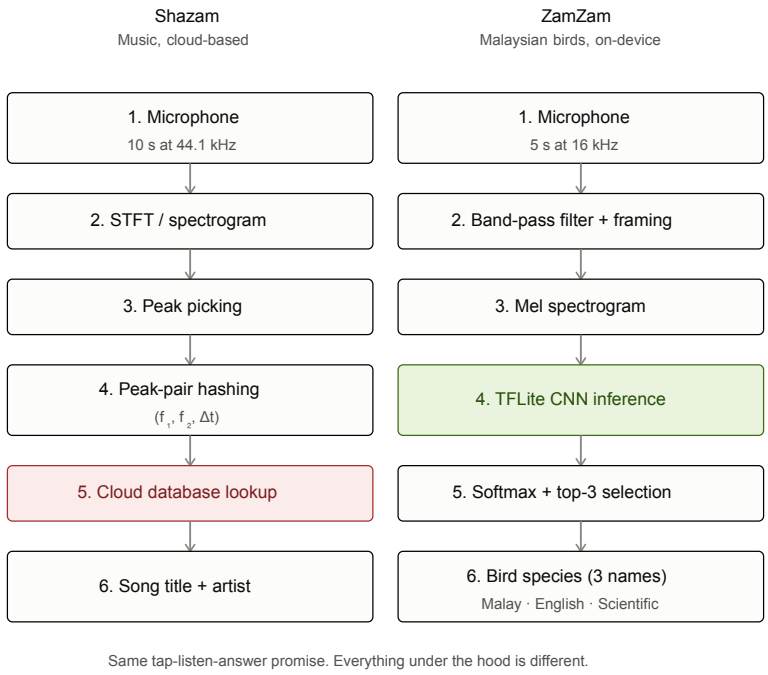

Figure 1.3: The Shazam pipeline (left) versus the ZamZam pipeline (right). The user gesture is the same; the internals are almost entirely different.

CNN on Mel spectrograms of Malaysian bird calls [8]. The model generalises across individual variation the way a human birder does: it learns the *pattern* of the call, not one specific recording.

- **On-device inference.** The model runs on the phone via TensorFlow Lite. No network, no cloud database, no data leaves the device. Works in a forest with zero bars of signal.
- **Focused species list.** Rather than covering every bird on Earth, we target the $\sim$100 most common species of Peninsular Malaysia and Borneo. A smaller output space means higher accuracy and a smaller model. Later releases add species as the training data grows.
- **English UI, three-name species results.** The Flutter UI is in English — almost every Malaysian phone user reads English comfortably, and one language keeps the app simple. But results show three names per bird: the Malay common name (e.g. *Murai Kampung*), the English common name (*Oriental Magpie-Robin*), and the scientific name (*Copsychus saularis*). Users learn the local vocabulary while using the app.
- **Lightweight.** The app is small ($<$ 20 MB installed), works on mid-range

Android phones the majority of Malaysian users actually own, and does not drain the battery.

We are not trying to out-Shazam Shazam. Instead, ZamZam focuses on a problem that existing music-identification technology was never designed to solve: fast, offline recognition of common Malaysian bird species on everyday smartphones. Although this book concentrates on mobile applications for citizen scientists, the same underlying machine learning techniques are also widely used in professional biodiversity monitoring, where the engineering constraints are quite different.

1.5 Professional Monitoring Versus Citizen Science

Bird-sound recognition serves two distinct communities: professional ornithologists conducting long-term biodiversity monitoring, and citizen scientists capturing casual observations. Although both applications rely on the same underlying machine learning techniques, they impose very different workflows and engineering requirements.

Professional ornithologists typically deploy autonomous recording units (ARUs) in forests to collect acoustic data over extended periods. These battery-powered embedded devices record audio continuously or on a schedule, storing raw data on removable SD cards for post-deployment offline analysis. This workflow prioritises data completeness and battery longevity over immediate feedback. Citizen scientists, by contrast, rely on smartphones for real-time identification. Instead of collecting weeks of unparsed audio, mobile applications perform instant on-device inference to provide immediate species suggestions, allowing observations to be verified and submitted on the spot to platforms like eBird.

These differing priorities translate directly into the engineering trade-offs summarised in Table 1.2. While ARUs emphasise extreme energy efficiency, ruggedness, and months-long unattended operation, mobile applications benefit from abundant memory, faster processors, and the global reach of consumer smartphone ecosystems.

Despite these differences in deployment, the underlying processing pipeline remains remarkably similar. Both smartphones and autonomous recording units acquire audio, transform it into spectrograms, perform neural network inference, and generate predictions. The primary distinction lies not in the machine learning itself, but in the surrounding software ecosystem and hardware constraints. Since this book focuses on smartphone deployment, the remainder of the chapter briefly introduces the technology stack that will be used throughout the implementation chapters.

Table 1.2: The bird-ID problem on an ARU versus on a smartphone

Concern	Autonomous Recording Unit (ARU)	Smartphone
Primary function	Continuous audio logging to SD card	Real-time on-device identification
Inference location	Offline/Post-hoc (Desktop, Server, Cloud)	On-edge (Local device processor)
Model constraints	None during capture (Large server models)	Restrained by mobile hardware (5–50 MB)
Power profile	Micro-power (mW range, months of logging)	Active compute (100s of mW during active ID)
Acoustic input	Weatherproof external capsule	Built-in multi-mic array
Data latency	Weeks to months (Until physical retrieval)	Instantaneous feedback to user
Deployment scale	Hundreds of specialised stations	Billions of consumer devices
Update workflow	Physical hardware/firmware maintenance	App store over-the-air updates
Target user	Ecological researchers and land managers	Citizen scientists and casual birders

1.6 The Stack This Work Uses

- **Flutter** for the UI, so one Dart codebase ships to both iOS and Android.
- **TensorFlow Lite** (via the `tflite_flutter` plugin) for on-device inference.
- **Native audio APIs** accessed through a Flutter package (e.g. `record` or `mic_stream`) for microphone capture at 16 kHz mono.
- **Dart or on-device TF ops** for computing Mel spectrograms from raw audio.
- **Localized species metadata** in a small JSON catalogue bundled with the app.

The next chapters revisit ML foundations—Deep Learning Basics (Ch 2), ML at the Edge (Ch 3), and Audio DSP (Ch 4)—with mobile focus. Chapter 5 covers requirements and architecture. Chapter 6 addresses UI design early, so you prototype before writing Dart. Chapters 7 and 8 build the dataset and train the model; from Chapter 8 onward, you write the Flutter app.

1.7 Key Takeaways

- Shazam works for music because a song is the exact same recording every time; birdsong varies across individuals, seasons, and background noise, so acoustic fingerprinting does not transfer.
- Existing bird apps cover different parts of the problem: Merlin polishes the UX, BirdNET provides global coverage, eBird supports the checklist workflow — none is optimised for Malaysian users on mid-range Android phones with unreliable rural signal.
- ZamZam's niche is a focused Malaysian species list, on-device inference (no signal required), and three-name results (Malay, English, scientific).
- Lightweight footprint and offline operation are the two constraints that shape every architectural decision in the rest of the book.

1.8 Exercises

1.1 Install Shazam and try to identify (a) a song you know, (b) a whistle you make yourself, and (c) a bird call from YouTube. Note what it does with each.

1.2 Read Wang's original Shazam paper [3]. In two sentences, why is the peak-pair hash robust to background noise?

1.3 Install *Merlin Bird ID* and *BirdNET Sound ID*. Try three Malaysian bird calls with each. Report which app got each right, and how confident it was.

1.4 List five Malaysian bird species you would prioritise for a first version of ZamZam. Justify each choice on frequency of occurrence, distinctiveness of call, or cultural significance.

1.5 Sketch what the fingerprint-hash diagram in Fig. 1.2 would look like if the anchor peak's target zone were doubled in size. What would change in the number of hashes generated per song? What would change in match precision?

1.9 Reflection

- Where would a Shazam-style fingerprint approach still be useful in bioacoustics — for example, matching a specific known individual bird's territorial song against a library of that bird's earlier recordings?

- If a Malaysian user in a rural area has an entry-level Android phone with 2 GB of RAM and a spotty data connection, which of ZamZam's design decisions matters most to them?
- What would break first if we tried to grow ZamZam from 50 species to 5,000 species?
- Merlin, BirdNET, and eBird are all products of Northern-hemisphere institutions. Are there structural reasons Southeast-Asian species coverage lags, and what would it take to close that gap?

Bibliography

[1] Malaysia Biodiversity Information System (MyBIS). *Birds of Malaysia*. Accessed: 2026-03-10. 2026. URL: https://www.mybis.gov.my/.

[2] Denis Lepage. *Checklist of the birds of Malaysia*. Accessed: 2026-03-10. 2026. URL: https://avibase.bsc-eoc.org/.

[3] Avery Wang. "An Industrial-Strength Audio Search Algorithm." In: *Proceedings of the 4th International Conference on Music Information Retrieval (ISMIR)*. 2003, pp. 7–13.

[4] Cornell Lab of Ornithology. *Merlin Bird ID*. Mobile app for bird identification by sound and photo. 2014. URL: https://merlin.allaboutbirds.org/.

[5] Cornell Lab of Ornithology. *eBird Mobile App*. Citizen-science database and mobile app for bird observations. 2002. URL: https://ebird.org/.

[6] Stefan Kahl et al. "BirdNET: A deep learning solution for avian diversity monitoring." In: *Ecological Informatics* 61 (2021), p. 101236. DOI: 10.1016/j.ecoinf.2021.101236.

[7] Chong Leong Puan et al. "Influence of landscape matrix on urban bird abundance: evidence from Malaysian citizen science data." In: *Journal of Asia-Pacific Biodiversity* 12.3 (2019), pp. 369–375. DOI: 10.1016/j.japb.2019.01.008.

[8] Dan Stowell. "Computational bioacoustics with deep learning: a review and roadmap." In: *PeerJ* 10 (2022), e13152.

Chapter 2

Deep Learning Basics

Learning Objectives

By the end of this module, you should be able to:

- Explain what Machine Learning (ML) and Deep Learning (DL) are.
- Describe the structure of a simple neural network.
- Train a small neural network in Python using TensorFlow/Keras.
- Understand how model size and accuracy trade-offs matter for microcontrollers.

2.1 What is Machine Learning?

Machine Learning (ML) teaches computers to recognize patterns and make decisions by learning from data and experience, rather than following rigid, hand-coded rules for every scenario.

Traditional Programming vs. Machine Learning

In traditional programming:

- Humans write explicit rules: "If this condition is true, then do that action."
- The program follows these rules step by step.

In machine learning:

- We provide the computer with *examples* (data) rather than rules.
- The computer learns the rules itself by adjusting internal parameters.

Analogy: Imagine teaching someone to recognize birds.

- Traditional: you write a rule like "If it has feathers and a beak, then it is a bird."
- Machine learning: you show them thousands of bird and non-bird images, and they learn the common patterns themselves.

From Machine Learning to Deep Learning

- **Machine Learning:** Uses algorithms like decision trees or regression, but often requires human input to select important data features.
- **Deep Learning:** A subset of ML using **multi-layer neural networks** that automatically extract features from raw data (e.g., images, audio).

TinyML is the fusion of machine learning and embedded systems, enabling on-device intelligence for resource-constrained devices. This is achieved through:

- Model compression (reducing size from GBs to kBs),
- Optimization (running on devices with milliwatt power and real-time response).

2.2 Training vs. Inference

Machine learning systems have two distinct phases that typically run on different machines.

Training. Fitting the model's weights to a dataset. This is compute-heavy — hours to days on a GPU workstation or a cloud VM — but it only runs *once* (or a few times as the dataset grows). Latency is not important; throughput and wall-clock time to a good checkpoint are. In this book, all training is done in a Google Colab notebook with GPU acceleration (Chapters 7 and 8 walk through the full pipeline).

Inference. Running the trained model on new data to produce a prediction. This runs *constantly* for every new user recording. It is interactive, latency-bound, and often energy-bound. Where inference runs — cloud server, phone, microcontroller — is a design choice with big downstream consequences, covered in the next chapter.

Nobody trains a model on a phone or microcontroller—they lack RAM, compute power, and energy budgets. But they can run a compact pre-trained model efficiently, which makes on-device ML (and this book) possible. The pattern of training-on-cloud and inference-on-edge is now standard in modern ML products and is the approach this book follows throughout.

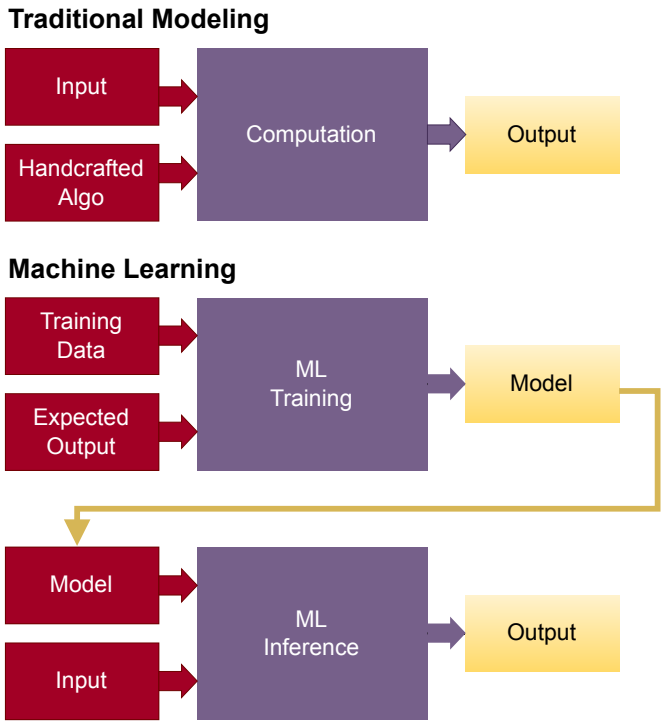

Figure 2.1: Traditional vs ML

2.3 Neural Networks: The Intuition

Neural networks are inspired by biological neurons but are far simpler. They combine weighted inputs, add a bias term, and pass the result through a nonlinear function called an *activation*.

From Linear Algebra to Intelligence

You can think of a neural network as a sequence of operations:

1. Take several inputs (like sensor readings or pixels).
2. Multiply each input by a **weight** that determines its importance.
3. Add the results together and include a **bias** term.
4. Pass the result through an **activation function** to decide the output.

Mathematically, for inputs $x_1, x_2, \ldots, x_n$ with weights $w_1, w_2, \ldots, w_n$, and bias b,

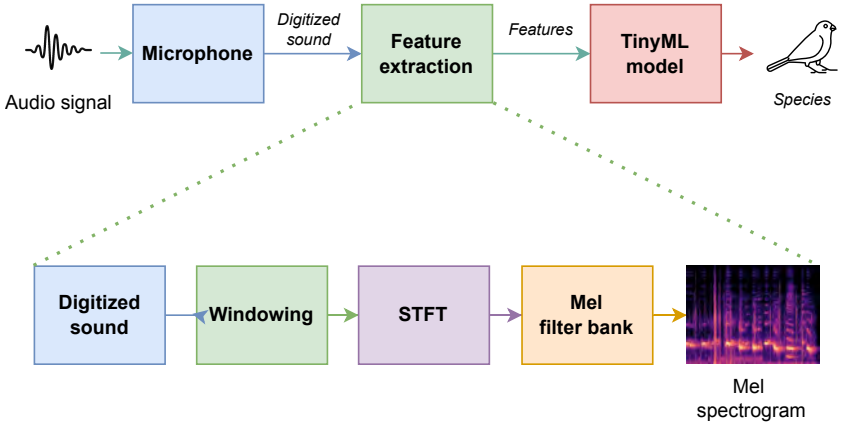

Figure 2.2: Bird sound recognition pipeline: training on GPU once; inference on-device constantly.

the neuron output is:

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

where $f(\cdot)$ is the activation function.

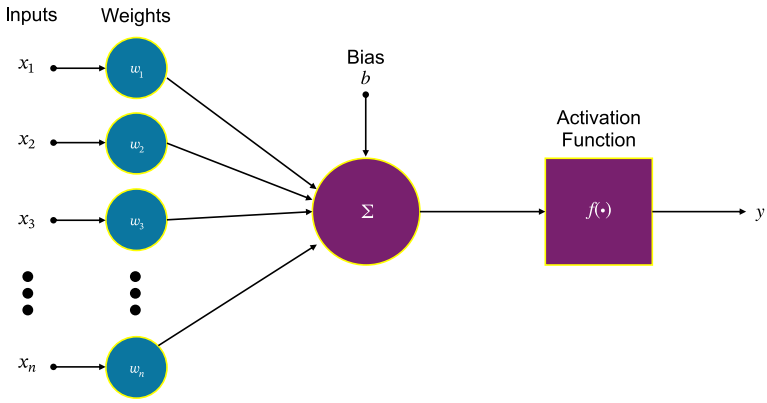

Figure 2.3: Artifical neural network structure

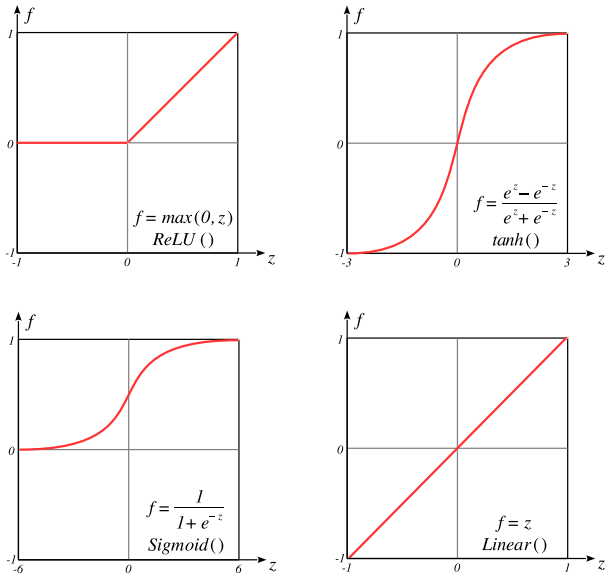

Figure 2.4: Four widely used activation functions

Why Activation Functions Matter

Without nonlinear activations, a network is just a stack of linear operations — equivalent to a single linear function. Activations introduce nonlinearity, allowing neural networks to approximate complex patterns. Common examples include:

- **Sigmoid:** Smooth step between 0 and 1.
- **ReLU (Rectified Linear Unit):** Passes positive values, zeros out negative ones.
- **tanh:** Like sigmoid but centered at zero.

From One Neuron to a Network

By stacking neurons into layers and layers into networks, we build systems capable of learning complex input-to-output mappings. Training adjusts the weights w_i and biases b to perform tasks like recognizing speech commands or detecting gestures.

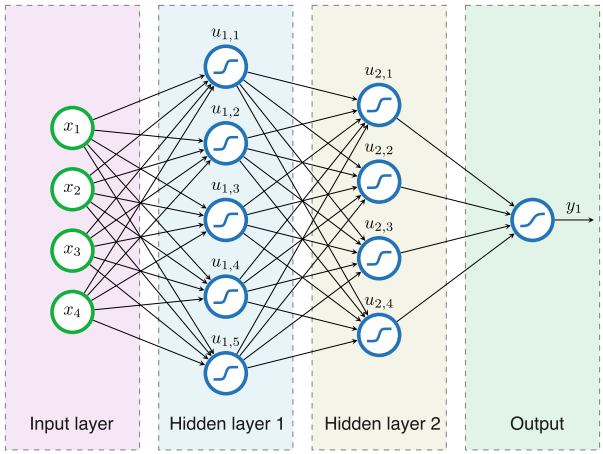

Figure 2.5: A deep learning network: multiple hidden layers build up from simple features to abstract concepts

From Neural Networks to Deep Learning

A basic neural network with only one hidden layer is called *shallow*; such networks solve simple tasks but struggle with complexity.

Deep learning uses multiple hidden layers between input and output, where *deep* refers to the layer count. Each layer learns increasingly abstract features: early layers capture simple patterns (like edges), middle layers combine them into shapes, and deeper layers recognize high-level concepts (objects, spoken words). As data flows through this hierarchy, the network transforms raw input into increasingly useful representations—this is the key to deep learning's power for complex tasks like vision, speech, and language.

Convolutional Neural Networks (CNNs)

Deep learning broadly refers to neural networks with many hidden layers that learn complex patterns. A convolutional neural network (CNN) is a specialized deep learning model for spatially structured data like images or spectrograms. Instead of fully connecting every neuron, CNNs use small filters that slide across the input—this makes them efficient, resistant to overfitting, and excellent at detecting local patterns like edges, textures, or repeating sound features. In brief: **deep learning is the family; CNN is a specialized member for structured data**.

2.4 Running Code in Google Colab

Before training our first neural network, we need a ready-made environment. Instead of installing Python, TensorFlow, and dependencies locally, we will use **Google Colab**, a free cloud-based Jupyter notebook service.

Why Colab?

- Free cloud platform for Python notebooks.
- Pre-installed with TensorFlow, NumPy, and many scientific libraries.
- Provides free GPU access for faster training.
- Requires only a Google account and a web browser.
- Easy to share notebooks with others (like Google Docs).

Getting Started

1. Visit: `https://colab.research.google.com`
2. Sign in with your Google account.
3. Create a new notebook: `File` → `New Notebook`.

You now have a working Python environment.

Checking TensorFlow

In a Colab cell, type:

```
import tensorflow as tf
print(tf.__version__)
```

It should display a version like `2.x`.

> **Tip**
>
> Colab notebooks save automatically in Google Drive. You can also download them as `.ipynb` or export to `.py` scripts.

Enabling GPU Acceleration (Optional but Recommended)

1. Click `Runtime` → `Change runtime type`.
2. Under `Hardware accelerator`, select `GPU`.

3. Click `Save`.

You now have free GPU access, which significantly speeds up training.

2.5 Your First Neural Network in Code

Let us build our intuition by training a simple neural network on **MNIST**, a dataset of 70,000 grayscale images of handwritten digits (0–9), each 28×28 pixels.

While MNIST itself won't run on a microcontroller, it is deep learning's "Hello World." The principles here transfer directly to sensor data and audio features later.

Step 1: Import Libraries and Load Data

We use `TensorFlow/Keras`, which gives us convenient access to the MNIST dataset. Open a new notebook in Google Colab and paste this cell:

```
import tensorflow as tf
from tensorflow import keras
import numpy as np

(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
```

At this point:

- `x_train` is an array of 60,000 images (training data).
- `y_train` are the corresponding labels (digits 0–9).
- `x_test`, `y_test` form the test set (10,000 images).

Step 2: Preprocess the Data

Neural networks work better when inputs are normalized.

```
x_train = x_train.reshape(-1, 28*28).astype("float32") / 255.0
x_test = x_test.reshape(-1, 28*28).astype("float32") / 255.0
```

Here:

- Each 28×28 image is flattened into a 784-element vector.
- Pixel values are scaled to the range [0, 1].

Step 3: Define the Model

We start with a simple network: a single hidden layer.

```
model = keras.Sequential([
    keras.layers.Dense(128, activation="relu", input_shape=(784,)),
    keras.layers.Dense(10, activation="softmax")
])
```

Explanation:

- `Dense(128)`: A fully connected layer with 128 neurons and ReLU activation.
- `Dense(10)`: Output layer with 10 neurons (for digits 0–9), using softmax to produce probabilities.

Step 4: Compile the Model

We specify the optimizer, loss function, and metrics.

```
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
```

- **Optimizer (Adam):** Adjusts weights efficiently.
- **Loss (Cross-Entropy):** Measures how well predictions match labels.
- **Accuracy:** Human-friendly metric for evaluation.

Step 5: Train the Model

Now we let the model learn from the data.

```
model.fit(x_train, y_train, epochs=5, batch_size=32)
```

Each *epoch* means the model sees all 60,000 images once. The *batch size* means it processes 32 images at a time before updating weights.

Step 6: Evaluate on Test Data

Finally, we test the trained model on unseen data.

```
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test accuracy:", test_acc)
```

You should see around **97–98% accuracy** after just a few epochs, which is quite impressive for such a small network.

The next chapter asks: *where does the network run?* Bird-sound classification can target platforms from kilobyte-memory microcontrollers to GPU-accelerated boards. This choice cascades throughout the design—affecting architecture, power budget, enclosure, and cost—and deserves focused treatment.

2.6 Key Takeaways

- Machine learning replaces hand-written rules with patterns learned from labelled examples. Deep learning is the subset that uses multi-layer neural networks to extract features automatically.
- TinyML fuses ML with embedded systems: model compression turns gigabyte models into kilobyte ones so they run on microcontrollers with milliwatt power budgets.
- Cloud AI and edge AI are complementary — the cloud trains and aggregates, the edge runs the compressed model locally with lower latency, better privacy, and battery-scale power.
- A neural network is a stack of weighted sums followed by non-linear activations (ReLU, sigmoid, tanh). Without the non-linearity, the whole stack collapses to a single linear function.
- Training runs on a full framework (TensorFlow/Keras); deployment often does not. Plan the export path from the start.
- Bigger models are not always better — accuracy, size, and latency trade off, and the right choice depends on the target device.

2.7 Exercises

Refer to the Google Colab script:

1. Change the number of neurons in the hidden layer.
2. Change the activation function from ReLU to sigmoid.
3. Train for 1, 5, and 10 epochs. Compare results.

2.8 Reflection

- Why do smaller models train faster but may have lower accuracy?

- Why does accuracy sometimes stop improving after many epochs?
- When are smaller models better than bigger models?

References

[1] Martín Abadi et al. “TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems.” In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. USENIX Association, 2016, pp. 265–283.

Chapter 3

ML at the Edge

Learning Objectives

- Compare cloud inference and on-device ("edge") inference across latency, privacy, cost, and connectivity.
- Understand why smartphones are the fastest path to real users, and what the mobile ML pipeline looks like.
- Place any embedded platform on the five-tier **NEST** scale (Tier 0 through Tier 4).
- Compare microcontroller boards by processor, memory, power, and ML library support.
- Compare single-board computers (Raspberry Pi family, NVIDIA Jetson Nano) and pick the right one for a tropical field deployment.
- Reason about the trade-off cascade from board choice through battery, cooling, enclosure, and total cost.

Chapter 2 covered ML fundamentals. This chapter addresses: *where does the model run?* A classifier trivial on a laptop may be impossible on an ATmega328, excessive on a Jetson Nano, and impractical in a tropical field. Choosing the inference target determines everything else—battery, enclosure, cost, model architecture, and update strategy. Chapter 4 then covers the signal processing needed for on-device feature extraction.

3.1 Choosing an Inference Target

Chapter 2 established the training/inference split: training happens once on a GPU workstation, inference runs constantly on the user's device. This chapter picks the *inference* target for ZamZam. Three options dominate:

Cloud inference — The device (phone, sensor, browser) sends raw data to a remote server, which runs a large model and returns the result. Google's Speech-to-Text, ChatGPT, and Shazam all work this way.

Mobile inference — The model runs on the user's smartphone or tablet. No round-trip. Face ID on iPhones, Google's on-device keyboard prediction,

Table 3.1: Inference target comparison. Mobile and embedded together form the "edge" but differ enough on almost every axis to warrant separate treatment.

Concern	Cloud	Mobile	Embedded
Model size	Unlimited (GB)	5–50 MB	kB to a few MB
Compute	GPUs, TPUs, autoscaling	Mobile CPU/GPU/NPU	MCU or SBC CPU (rarely GPU)
Latency	100–2000 ms round-trip	10–500 ms local	10 ms–1 s local
Connectivity	Required (breaks in the forest)	Optional	None needed
Privacy	Data leaves the device	Data stays local	Data stays local
Battery	Radio + upload burns power	Modest per inference	Very low (mW-scale)
Distribution	API endpoint	App store	Custom hardware, flashing
Update workflow	Deploy new model instantly	App-store release	Reflash firmware or SD card
Reach	Anyone with a browser	Billions of phones	Hundreds of custom devices
Time to first user	Days (deploy an API)	Hours (`flutter run`)	Weeks (hardware iteration)
Ideal for	Very large models, analytics	Consumer apps, private + offline	Long-lived unattended sensors

and Merlin's Sound ID all work this way.

Embedded inference — The model runs on a purpose-built device: a microcontroller, single-board computer, or GPU-accelerated edge box. AudioMoth loggers, BirdNET-Pi installations, and industrial-vision cameras all work this way.

Mobile and embedded together form the "edge" side of the classical cloud-vs-edge divide, but the differences between them are large enough to deserve their own column. Table 3.1 makes the full three-way trade-off concrete.

Mobile inference wins for a birder in Taman Negara with unreliable signal, privacy-conscious recordings, and need for instant feedback. That is why ZamZam runs on the phone. The rest of this chapter surveys mobile and embedded platforms, starting with ZamZam's target: the phone.

3.2 ML on Mobile Devices

Smartphones and tablets are ubiquitous, powerful, and always with the user. A phone usually comes with a microphone, a GPU, gigabytes of RAM, a battery, a

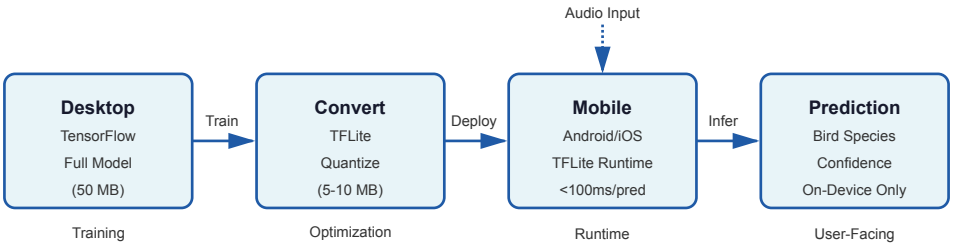

Figure 3.1: End-to-end ML pipeline from training on desktop to inference on a mobile device. Audio is captured locally, processed through a lightweight neural network, and predictions are displayed instantly without server communication.

screen, and a working app-distribution channel. For a consumer-facing app like ZamZam, mobile is the fastest path from a trained model to real users.

Why Mobile?

- **Always available.** Birdwatchers carry phones everywhere, enabling opportunistic recordings.
- **Instant feedback.** Predictions arrive without network latency or cloud round-trips.
- **Privacy.** Audio never leaves the device, addressing concerns about recording wildlife in sensitive areas.
- **Offline operation.** Works in forests and rural areas where cellular data is unreliable.
- **Reach.** Two billion Android + iOS devices — global distribution through app stores.
- **Rich UI.** Touchscreens, GPUs for animation, and haptic feedback — everything an MCU lacks.

The Mobile ML Pipeline

Mobile frameworks optimize for size, latency, and power—unlike desktop deep learning. The standard workflow is:

1. **Train on desktop.** Use full TensorFlow or PyTorch to build and train a model (no size constraints).
2. **Convert for mobile.** Compress the model to TensorFlow Lite (`.tflite`) or Core ML (`.mlmodel`) format.

3. **Deploy on device.** Load the compressed model in a mobile app (Flutter, Swift, Kotlin).

TensorFlow Lite models are typically 90–99% smaller than their desktop counterparts. A bird classifier trained on 50 MB of full-precision weights can be compressed to 5–10 MB and still maintain 95% accuracy.

Constraints and Trade-offs

Mobile devices impose strict resource limits:

- **RAM.** 2–8 GB on modern phones, but the app itself is usually given only 100–500 MB.
- **Storage.** Model files must be compact enough to bundle in the app (∼50 MB app size budgets are common).
- **Latency.** Real-time UX means inference should complete in 100–500 ms per prediction.
- **Battery.** Sustained inference (continuous listening) drains power; intermittent inference is preferable.
- **Quantisation.** Converting 32-bit floats to 8-bit integers reduces size and improves speed, sometimes at a small accuracy cost.

Mobile works for consumer apps but not the entire edge story. When a researcher needs a device that stays in the rainforest for six months unattended on a single battery, a phone is unsuitable. This is where microcontrollers and single-board computers become necessary.

3.3 ML on Embedded Devices

Unlike mobile platforms, which are carried by users and leave the site with the observer, embedded devices are designed to remain in the field for continuous monitoring. They can be deployed in remote or difficult-to-access locations and continue collecting acoustic data at night and during adverse weather conditions.

Embedded machine learning platforms span a wide spectrum, ranging from low-cost STM32 microcontrollers to NVIDIA Jetson Nano systems—a difference of roughly three orders of magnitude in memory and two orders of magnitude in power consumption. Comparing such diverse hardware fairly requires a common language.

3.4 The NEST Taxonomy

This book uses the **Neural Embedded Systems Taxonomy (NEST)** introduced by [1], which stratifies edge platforms into five tiers based on memory capacity, computational capability, and power envelope. Table 3.2 summarises the tiers; each is discussed in its own subsection below.

Table 3.2: The NEST taxonomy of embedded ML platforms

Tier	Description	Typical hardware	Power	Model class
0	Non-ML signal processing	AVR, PIC	<10 mW	Rule-based / classical DSP only
1	Low-power ML MCU	ESP32, M5Stack, RP2040	20–240 mW	TFLite Micro, <1 MB
2	High-performance MCU	Portenta H7, STM32H7	100–500 mW	TFLite Micro, 1–10 MB
3	Single-board computer	Raspberry Pi Zero 2 W/3/4/5	2–12 W	Full TF/PyTorch, 10–500 MB
4	GPU/TPU-accelerated edge	Jetson Nano, Coral	5–15 W	CUDA-accelerated CNNs

NEST maps microarchitecture directly to feasible model classes, so a system designer can answer "will my model fit?" before writing any embedded code.

Tier 0 includes 8-bit MCUs like the ATmega328 (32 kB flash, 2 kB RAM) that *cannot run neural networks*—they execute hand-tuned filters, threshold detectors, or classical correlators. They work well as low-power *triggers* that wake Tier 1 boards when interesting audio arrives, but are not ML platforms. Tiers 1–4 support machine learning at the edge.

Tier 1: Low-Power ML Microcontrollers

ESP32 and M5Stack Core2. The ESP32 microcontroller and the M5Stack Core2 (an ESP32 packaged with a touchscreen, battery, and enclosure) represent NEST Tier 1. Dual-core Xtensa LX6 at 240 MHz, 520 KB SRAM plus 8 MB PSRAM, Wi-Fi and Bluetooth built in. They run TensorFlow Lite Micro with the ESP-NN kernel library and target battery-operated IoT with sub-1 MB models — lightweight keyword spotting or MNIST-scale classification [2].

Their sweet spot for bird monitoring: a low-cost solar-powered sensor node in a fixed location, doing binary bird-activity detection and uploading events over Wi-Fi when in range.

Beyond bird monitoring, Tier 1 hardware powers a broad family of **TinyML** applications — tasks small enough to run on a milliwatt-scale microcontroller:

- **Keyword spotting.** “Hey Google,” “Alexa,” or a custom wake-word detector on a sub-dollar MCU.

- **Gesture recognition.** Accelerometer-based flick/shake/tap detection for wearables and remote controls.

- **Environmental monitoring.** Machinery-fault detection from vibration, bird presence from audio, or leak detection from a pressure sensor.

- **Healthcare devices.** Low-power arrhythmia detection from a wearable ECG, running for months on a coin cell.

(a) ESP32

(b) M5Stack Core2

(c) STM32 Nucleo H755ZI

(d) Arduino Portenta H7

Figure 3.2: MCU development boards

Table 3.3: Microcontroller boards for on-device machine learning

Specification	ESP32 / M5Stack Core2	Arduino Portenta H7	STM32 Nucleo H755ZI
NEST Tier	1	2	2
Processor(s)	Dual-core ESP32 Xtensa 32-bit LX6	Dual-core STM32H747: Cortex-M7 + Cortex-M4	Dual-core STM32H755XI: Cortex-M7 + Cortex-M4
Clock Speeds	Up to 240 MHz (both cores)	M7: 480 MHz, M4: 240 MHz	M7: 480 MHz, M4: 240 MHz
Flash Memory	16 MB external Flash	2 MB internal + 8 MB QSPI	2 MB internal dual-bank Flash
RAM	520 KB + 8 MB PSRAM	1 MB SRAM + 8 MB SDRAM	1 MB SRAM (192 KB TCM + 864 KB user + 4 KB backup)
Power Consumption	Low-medium: 20–240 mA active, $<$1 mA deep sleep	Medium: 280 µA/MHz active, 6 µA standby	Medium: 280 µA/MHz active, 6 µA standby
Key ML Libraries	• TensorFlow Lite Micro • ESP-NN • ESP-WHO (vision) • Arduino ML libraries	• TensorFlow Lite Micro • CMSIS-NN • OpenMV libraries • STM32 AI (X-CUBE-AI) • Arduino libraries	• TensorFlow Lite Micro • CMSIS-NN • CMSIS-DSP • STM32 AI (X-CUBE-AI) • STM32CubeIDE ML
Dev Ecosystems	Arduino IDE, PlatformIO, ESP-IDF, MicroPython	Arduino IDE, PlatformIO, STM32CubeIDE, Mbed OS	STM32CubeIDE, Mbed OS, PlatformIO, Keil µVision
Connectivity	Wi-Fi 802.11b/g/n, Bluetooth 4.2/BLE	Wi-Fi 802.11b/g/n, Bluetooth 5.1/BLE, Ethernet (via shield)	Ethernet, USB, CAN, multiple serial interfaces
Best Use Cases	IoT sensors, simple edge AI, prototyping, battery-powered devices	Industrial IoT, computer vision, advanced edge ML, real-time control	High-performance embedded ML, industrial applications, DSP processing

Note: All specifications are typical values and may vary based on configuration and workload.

Definition: What is TinyML?

TinyML is a subfield of machine learning focused on running deep learning models on low-power, resource-constrained hardware like microcontrollers. By performing inference directly at the edge, it enables three key advantages:

- **Ultra-low Power.** Operates under **1 mW**, running for months on a battery.
- **Zero Latency.** Processes data locally for immediate, real-time responses.
- **Offline Autonomy.** Functions in remote areas without internet connectivity.

Case Study: Google Speech Commands

The canonical TinyML example is **keyword spotting**. Google released the *Speech Commands* dataset — thousands of short audio clips of words like "yes," "no," "up," "down," and "stop." A small CNN trained on this dataset can run continuously on a microphone stream at 100 mW or less, waking a larger cloud model only when a wake word is heard. Three benefits fall out:

- **Privacy.** Audio only leaves the device *after* the wake word.
- **Efficiency.** Constant listening costs milliwatts, not watts.
- **Scalability.** The model fits on a sub-dollar MCU, so it can ship in millions of devices.

The bird-sound detector running on a microcontroller is architecturally similar: a compact CNN listening continuously on an embedded device, escalating only interesting audio.

Tier 2: High-Performance Microcontrollers

Arduino Portenta H7 and STM32 Nucleo H755ZI. The Portenta H7 and Nucleo H755ZI are dual-core Cortex-M7/M4 STM32H7 boards — the M7 at 480 MHz for compute, the M4 at 240 MHz for real-time I/O. Both offer 1 MB SRAM (Portenta adds 8 MB external SDRAM) and strong floating-point plus DSP support. They can run 1–10 MB models, making them the practical choice for on-device Mel-spectrogram computation and species-level bird identification without leaving MCU-land.

Their sweet spot: a battery-powered device that must classify tens of species locally, with weeks of unattended operation per charge.

Table 3.3 details the four MCU-class boards.

Tier 3: Single-Board Computers

Tier 3 marks the transition from bare-metal microcontrollers to Linux-based single-board computers (SBCs). Unlike MCU platforms, Raspberry Pi boards run a full operating system, allowing developers to use standard machine learning frameworks such as TensorFlow, PyTorch, ONNX Runtime, and BirdNET Analyzer with little or no modification. The additional software flexibility comes at the cost of higher power consumption, larger memory footprints, and more demanding thermal management.

Raspberry Pi Zero 2 W. The Raspberry Pi Zero 2 W is the smallest member of the Raspberry Pi family. Despite its compact size, it is a complete Linux computer based on a quad-core Cortex-A53 processor with 512 MB RAM. Its mod-

est power consumption makes it suitable for lightweight edge inference, sensor gateways, and compact embedded applications where physical size is critical. However, the limited memory restricts the size of deployable neural networks and reduces inference throughput compared with larger Raspberry Pi models.

Raspberry Pi 3 (the field-monitoring sweet spot). The Raspberry Pi 3 combines a quad-core Cortex-A53 processor running at 1.2 GHz with 1 GB of RAM and integrated Wi-Fi. Typical power consumption is only 2–3 W, allowing the board to operate without active cooling under most workloads. This passive thermal design simplifies waterproof enclosure construction, making the Pi 3 particularly attractive for autonomous recording units deployed in tropical rainforests, mangroves, and other harsh outdoor environments. Although newer Raspberry Pi models offer significantly higher performance, the Pi 3 often provides the best balance between computational capability, energy efficiency, and deployment simplicity.

Raspberry Pi 4 and Raspberry Pi 5. The Raspberry Pi 4 and Raspberry Pi 5 substantially increase processing capability through newer Cortex-A72 and Cortex-A76 processors, larger memory options, faster storage interfaces, and Gigabit Ethernet. These improvements enable larger neural networks, shorter inference times, and more sophisticated on-device processing pipelines. The additional performance comes with increased power consumption (typically 3–12 W) and greater thermal output. In particular, the Raspberry Pi 5 generally requires active cooling during sustained workloads, increasing system complexity for unattended field deployments.

Choosing the right Raspberry Pi. Selecting a Raspberry Pi affects much more than inference speed. Higher computational performance increases energy consumption, which in turn determines battery capacity, enclosure design, cooling

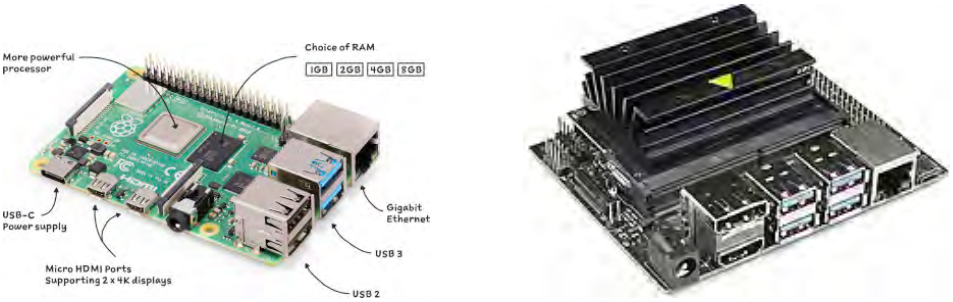

Figure 3.3: SBC embedded Linux platforms. Left: Raspberry Pi (Tier 3). Right: NVIDIA Jetson Nano (Tier 4).

Table 3.4: Single-board computers

Specification	Raspberry Pi	NVIDIA Jetson Nano
NEST Tier	3 (SBC)	4 (Accelerated SBC)
CPU	Quad-core ARM Cortex-A72/A76 @ 1.5–2.4 GHz	Quad-core Cortex-A57 @ 1.43 GHz
GPU / Accelerator	VideoCore VI/VII (graphics only)	128-core Maxwell CUDA GPU (472 GFLOPS)
RAM	1 / 2 / 4 / 8 / 16 GB (LPDDR4/4X)	4 GB LPDDR4-1600 (shared CPU/GPU)
Storage	MicroSD; optional USB-SSD or NVMe	MicroSD; optional M.2 NVMe
Power Consumption	3–12 W typical (Pi 4 lower, Pi 5 higher)	5–10 W typical (5W/10W modes)
Key ML Libraries	• TensorFlow / TFLite • PyTorch • OpenCV • ONNX Runtime	• TensorRT (CUDA-accelerated) • TensorFlow / PyTorch • DeepStream SDK • JetPack + CUDA/cuDNN
Connectivity	Wi-Fi 5, Bluetooth 5.0, Gigabit Ethernet, USB 3.0	Gigabit Ethernet, USB 3.0 (Wi-Fi via USB dongle or M.2)
Best Use Cases	Full-pipeline bird-ID at a fixed monitoring site; BirdNET Analyzer host; long-term ARU deployments	Real-time video + audio inference; large CNNs; multi-model deployments; GPU-accelerated workloads

Note: All specifications are typical values and may vary based on configuration and workload.

requirements, and ultimately deployment cost.

A Raspberry Pi Zero 2 W can support lightweight Linux applications while operating from relatively small battery packs. A Raspberry Pi 3 remains practical for long-term autonomous monitoring because its modest power requirements allow passive cooling and manageable battery sizes. Raspberry Pi 4 and Pi 5 systems demand larger batteries or solar panels for extended operation, and the Pi 5 additionally requires thermal management. Consequently, the Raspberry Pi 3 continues to occupy a particularly attractive niche for long-duration field monitoring, while the Pi 4 and Pi 5 are better suited to installations with reliable power or applications where maximum inference performance is required.

Tier 4: GPU/TPU-Accelerated Edge

NVIDIA Jetson Nano. The Jetson Nano adds a 128-core Maxwell CUDA GPU (472 GFLOPS) to a quad-core ARM Cortex-A57 CPU with 4 GB of shared LPDDR4. It runs TensorRT, the full TensorFlow and PyTorch stacks, and NVIDIA's DeepStream SDK — meaning it can host models that would otherwise require a workstation, and can do so with hardware acceleration. The trade-off

is power: typical draw is 5–10 W (5 W and 10 W modes are user-selectable), which effectively rules out battery-only deployments.

Its sweet spot: a permanent field station with mains or solar-plus-large-battery power, running real-time video-plus-audio inference over multiple species simultaneously, or serving as a base station that aggregates data from a network of Tier 1 sensors.

Similar Tier 4 devices include Google's Coral Dev Board (Edge TPU) and Intel's Movidius NCS — each with its own accelerator and software stack, but all in the same 5–15 W envelope.

Table 3.4 covers the SBC and GPU-accelerated options (Tiers 3 and 4).

3.5 Cost of Deployment

The board itself is usually the cheapest part of a field deployment. Batteries, cooling, enclosures, and solar all add up. Table 3.5 gives typical Malaysian retail prices for the boards themselves; Table 3.6 lists the surrounding infrastructure.

Table 3.5: Approximate board prices (Malaysia)

Tier	Board	Price (RM)
Microcontrollers (Tier 1–2)		
1	Raspberry Pi Pico	25–40
1	Raspberry Pi Nano	35–50
2	STM32 Nucleo H755ZI-Q	140
1	M5Stack Core2	200
2	Arduino Portenta H7	500
Single-Board Computers (Tier 3)		
3	Raspberry Pi 3 (used, 1 GB)	120–150
3	Raspberry Pi 4 (4 GB)	430
3	Raspberry Pi 5 (8 GB)	600
GPU-Accelerated Edge (Tier 4)		
4	NVIDIA Jetson Nano (4 GB dev kit)	950

Table 3.6: Supporting infrastructure prices (Malaysia)

Component	Price (RM)
Small battery (2000 mAh)	20–40
Large battery (10 000 mAh)	100–200
Solar panel (10 W rated)	300–500
Waterproof enclosure (IP67)	50–150

Adding these up gives the true cost picture. Entry-level Tier 1–2 boards (Pico, Nano, MCUs) cost RM 30–200 each but run on small batteries, so a complete sensor node stays under RM 300. Mid-range boards (Pi 3) cost RM 120–150 and still run passively cooled, keeping system complexity low; a 3000 mAh battery adds another RM 50. Higher-power boards (Pi 4/5) cost RM 430–600 and demand active cooling and larger batteries (RM 100–200), or a solar panel (RM 300–500 installed). The Jetson Nano bridges the gap at RM 950 but requires mains power or a large solar array, suitable only for well-resourced field stations.

3.6 Key Takeaways

- Train on a workstation, run inference at the edge — training and deployment have different hardware needs and should not be conflated.
- Three inference targets: cloud, mobile, embedded. ZamZam picks *mobile*.
- NEST tiers 1–4 order the embedded options from milliwatts (Cortex-M0/M4 MCUs) to Jetson-class watts (GPU/TPU edge accelerators).
- Board choice cascades into battery capacity, cooling, enclosure, and system cost. Pick the target first; everything else follows.
- For continuous-listening field sensors, sub-100 mW Tier 1–2 boards on small batteries dominate on cost and autonomy; Tier 3–4 boards are only worth it when accuracy or model size demands it.

3.7 Exercises

1. Given a 2 MB Mel-spectrogram CNN with 500 KB of activation memory, list all NEST tiers on which it will run. Which tier is the cheapest?
2. Estimate the battery life of a Pi 3 running continuous inference at 3 W, powered by a 10 000 mAh, 3.7 V lithium cell.

3. Design a Tier 1 deployment for a Straw-headed Bulbul monitoring station in Taman Negara. Specify board, battery, enclosure, and expected months of unattended operation.
4. Compare the total system cost (board + battery + enclosure + solar if needed) of a Pi 3 versus a Pi 5 for the same task. When does the Pi 5's speed advantage justify the extra cost?
5. You are asked to add a face-recognition feature to a mobile app that already includes ZamZam. Which of the cloud / mobile / embedded trade-offs in Table 3.1 would change your answer?

3.8 Reflection

- Why does the NEST paper stratify by *memory* rather than by clock speed?
- In what scenarios would you deploy on Tier 4 despite the power cost, rather than on Tier 3?
- Would you use a Raspberry Pi Pico for the ZamZam mobile app? Why or why not?
- For a bird-monitoring project with 100 sensor nodes in Sabah, would you prefer a mesh of Tier 1 boards feeding into one Tier 4 base station, or 100 independent Tier 3 boards? Justify.

References

[1] Muhammad Mun'im Ahmad Zabidi. "Edge AI for Avian Acoustic Monitoring: A Systematic Review and Deployment Framework." In: *IEEE Access* 4 (2026). In preparation.

[2] Sumit Kumar and Shubham. *Recognizing MNIST-based handwritten digits on M5Stack Core2*. https://www.hackster.io/mech/recognizing-mnist-based-handwritten-digits-on-m5stack-core2-7cbc4a. Accessed: July 6, 2026. 2022.

[3] Amin Biglari and Wei Tang. "A Review of Embedded Machine Learning Based on Hardware, Application, and Sensing Scheme." In: *Sensors* 23.4 (2023). ISSN: 1424-8220. DOI: 10.3390/s23042131.

[4] Arduino Pro Team. *Arduino Portenta H7 Documentation*. https://docs.arduino.cc/hardware/portenta-h7. Accessed: 2025-09-09. 2025.

[5] Raspberry Pi Foundation. *Raspberry Pi Documentation*. https://www.raspberrypi.com/documentation/. Accessed: 2025-09-09. 2025.

[6] Espressif Systems. *ESP32 Technical Reference Manual, Version 5.5*. Technical Reference Manual detailing hardware architecture, registers, and usage. 2025. URL: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf.

[7] Pete Warden and Daniel Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Chapter 3 introduces feature extraction for audio. O'Reilly Media, 2019. ISBN: 9781492052043.

Chapter 4

DSP for Audio

Learning Objectives

- Explain how audio signals are represented in the time and frequency domains.
- Apply basic DSP techniques such as framing, windowing, and spectrogram generation.
- Extract audio features (e.g., Mel spectrograms, MFCCs) for input to machine learning models.

4.1 Audio Signals

Understanding how digital audio signals are represented is essential before working with audio on embedded devices.

What is a Digital Audio Signal?

Audio is a continuous pressure wave in air. To process it digitally, we convert this wave into discrete numbers via **sampling** and **quantization**.

Sampling Rate and Bit Depth

- **Sampling rate:** how many samples per second are taken from the continuous waveform. For example, a rate of 16 kHz means 16,000 measurements per second. The sampling rate also sets the highest frequency the recording can represent — we will revisit this idea in the next section when we introduce the FFT.
- **Bit depth:** how many bits are used to represent each sample. A 16-bit depth allows values from -32768 to $+32767$.

For bird sounds, 16 kHz or 22.05 kHz sampling rates are typical. BirdNET uses 48 kHz.

Python Example: Visualizing Audio

The following code loads an example WAV file, displays the waveform, and plays back the audio:

Listing 4.1: Downloading a bird sound then view the waveform

```
import librosa
import librosa.display
import matplotlib.pyplot as plt
import numpy as np
import requests

# Download bird sound from Xeno-Canto
url = "https://xeno-canto.org/175830/download"
filename = "xc175830.mp3"

r = requests.get(url)
with open(filename, "wb") as f:
  f.write(r.content)

# Load audio with librosa
y, sr = librosa.load(filename, sr=16000)

# Plot waveform (time domain)
plt.figure(figsize=(10, 3))
librosa.display.waveshow(y, sr=sr)
plt.title("Waveform in Time Domain (Bird Sound)")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.show()
```

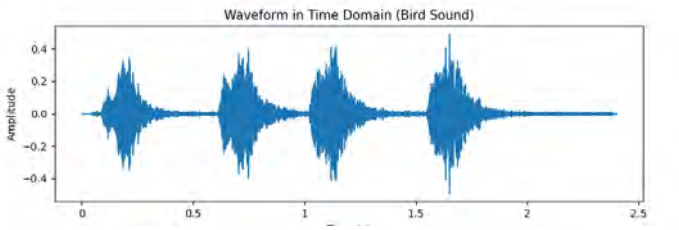

Figure 4.1: Waveform of a bird sound in the time domain. The horizontal axis is time (in samples), and the vertical axis is amplitude. This representation shows how the signal varies over time but does not reveal frequency content.

Note: You only need Colab to run all the Python code in this module. Available at: https://colab.research.google.com/drive/1Cddpe3AFoExEMVNFXD5wuyd7IF1_bcJb?usp=sharing

So far we have viewed audio only as amplitude over time. However, bird calls are more easily distinguished by their frequency content than by waveform alone.

4.2 Frequency-Domain Analysis

Frequency-domain analysis reveals which frequencies are present at each moment—the key to distinguishing a Zebra Dove's low coo from a Straw-headed Bulbul's ringing whistle. This section builds toward the **spectrogram**, the time-frequency image that machine-learning pipelines in this book consume.

DFT and FFT

The **Discrete Fourier Transform (DFT)** converts a time-domain signal into frequency components via the formula

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0, 1, \ldots, N-1$$

The **Fast Fourier Transform (FFT)** is the efficient algorithm that computes the DFT.

Since recordings are sampled, the FFT can only represent frequencies up to the **Nyquist frequency**, $f_s/2$. At 16 kHz, the highest representable frequency is 8 kHz—adequate for most Malaysian birds but insufficient for insects or bat echolocation. The **Nyquist theorem** states: to represent frequency f, you must sample at least $2f$ times per second.

Windowing and STFT

Real audio is non-stationary—its frequency content changes over time. To capture this variation, the signal is split into overlapping windows and an FFT is computed for each window. This is the **Short-Time Fourier Transform (STFT)**.

Spectrograms

A **spectrogram** stacks STFT frames into a 2D map:

- Horizontal axis: time
- Vertical axis: frequency
- Color or intensity: energy of frequency components

Spectrograms transform sound into image-like representations, allowing CNNs to detect structures—harmonics, chirps, bird calls—that are hidden in raw waveforms. This makes them essential inputs for audio classification.

Spectrograms are visualized using a **colormap** that maps energy to color. Common perceptually uniform choices include `gray_r`, `magma`, and `viridis`. Many

other colormaps are available in `matplotlib.pyplot.cm` to emphasize different features.

Python Example: Generating a Bird Sound Spectrogram

Listing 4.2: Generating a spectrogram with Librosa

```
import librosa
import librosa.display
import matplotlib.pyplot as plt
import requests
import numpy as np

# Download and save the bird sound (Xeno-Canto)
url = "https://xeno-canto.org/175830/download"
filename = "xc175830.mp3"
r = requests.get(url)
with open(filename, "wb") as f:
  f.write(r.content)

# Load the audio
y, sr = librosa.load(filename, sr=16000)

# Compute STFT
D = librosa.stft(y, n_fft=1024, hop_length=512)
S_db = librosa.amplitude_to_db(abs(D), ref=np.max)

# Plot spectrogram
plt.figure(figsize=(10, 4))
librosa.display.specshow(S_db, sr=sr, hop_length=512,
x_axis='time', y_axis='hz', cmap='viridis')
plt.colorbar(format="%+2.0f dB")
plt.title("Spectrogram of Bird Sound")
plt.tight_layout()
plt.show()
```

This standard spectrogram is the **STFT spectrogram**, distinguishing it from specialized variants.

Once we can analyze frequency content, we can selectively retain useful frequencies and suppress unwanted ones. This is the role of digital filtering.

4.3 Digital Filtering

Filtering modifies frequency content. Removing frequencies below 300 Hz suppresses wind noise and DC offset; attenuating above 5 kHz reduces hiss. Since most Malaysian bird vocalizations lie between 300 Hz and 5 kHz, a well-designed band-pass filter is one of the simplest ways to improve signal-to-noise ratio before feature extraction. Filtering is implemented mathematically via **convolution**.

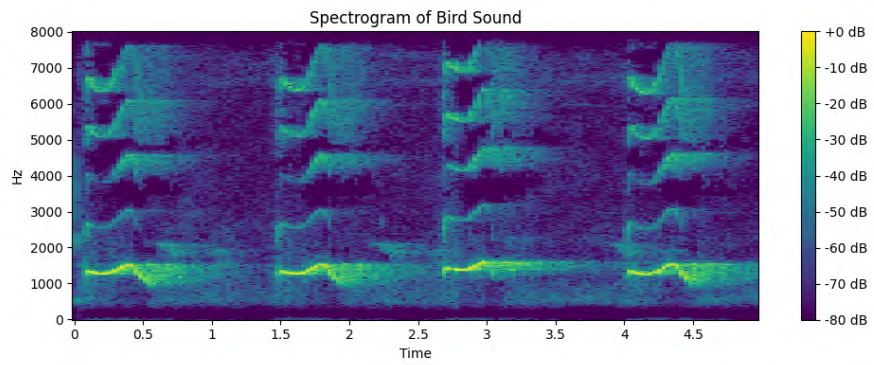

Figure 4.2: Spectrogram (STFT) of a bird sound. The horizontal axis is time, the vertical axis is frequency (Hz), and the colour represents energy (dB). This representation reveals which frequencies are present at each moment, making harmonic structure and frequency sweeps visible.

Discrete-Time Signals and Systems

A digital audio signal is a sequence of numbers:

$$x[n] = x(nT_s), \quad n = 0, 1, 2, \ldots$$

where $T_s = 1/f_s$ is the sampling interval. Digital systems process these sequences using addition, multiplication, and convolution.

Convolution and Filtering

Convolution describes how an input signal $x[n]$ interacts with a system's impulse response $h[n]$:

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k]\, h[n-k]$$

- This is the foundation of digital filtering. - Filters can be used to emphasize or suppress certain frequency components.

FIR and IIR Filters

Two common types of digital filters:

- **Finite Impulse Response (FIR)**: output depends only on current and past input samples.

$$y[n] = \sum_{k=0}^{M} b_k\, x[n-k]$$

Advantages: always stable, linear phase possible.

- **Infinite Impulse Response (IIR)**: output depends on current and past inputs *and* past outputs.

$$y[n] = \sum_{k=0}^{M} b_k \, x[n-k] - \sum_{k=1}^{N} a_k \, y[n-k]$$

Advantages: can achieve sharp frequency responses with fewer coefficients, but may be unstable.

Python Example: Removing Noise and DC Offset

In Python, we can implement a simple FIR band-pass filter to keep only the 300–5000 Hz band where Malaysian birds vocalise:

Listing 4.3: High-pass FIR filter in Python using SciPy

```
import requests
import librosa
import numpy as np
import matplotlib.pyplot as plt
import scipy.signal as signal

# --- Step 1: Download audio ---
url = "https://xeno-canto.org/175830/download"
filename = "xc175830.mp3"

try:
  with open(filename, "rb") as f:
    print(f"{filename} already exists, skipping download.")
except FileNotFoundError:
  print(f"Downloading {filename}...")
r = requests.get(url)
with open(filename, "wb") as f:
  f.write(r.content)
print("Download complete.")

# --- Step 2: Load audio ---
y, sr = librosa.load(filename, sr=16000)
print(f"Audio loaded: {y.shape[0]} samples at {sr} Hz")

# --- Step 3: Apply FIR band-pass filter (300 Hz - 5000 Hz) ---
nyquist = sr / 2
low_cut = 300 / nyquist
high_cut = 1500 / nyquist
numtaps = 101  # number of FIR coefficients
fir_coeff = signal.firwin(numtaps, [low_cut, high_cut], pass_zero=False)
y_filtered = signal.lfilter(fir_coeff, [1.0], y)

# --- Step 4: Plot original and filtered waveform ---
plt.figure(figsize=(12, 4))
plt.subplot(2, 1, 1)
plt.plot(y)
plt.title("Original Bird Sound Waveform")
plt.ylabel("Amplitude")
plt.subplot(2, 1, 2)
plt.plot(y_filtered)
```

```
plt.title("Filtered Bird Sound (300-1500 Hz)")
plt.xlabel("Sample")
plt.ylabel("Amplitude")
plt.tight_layout()
plt.show()
```

Explanation:

- **Download and load:** Same as before.
- **Filter design:** `firwin` creates a band-pass FIR filter with 101 taps.
- **Filtering:** `lfilter` applies the FIR filter to the audio.
- **Visualization:** Shows both original and filtered waveforms for comparison.

  Note: FIR filters are simple, always stable, and suitable for preprocessing audio before feature extraction. IIR filters are more efficient for sharper responses but require careful design to ensure stability.

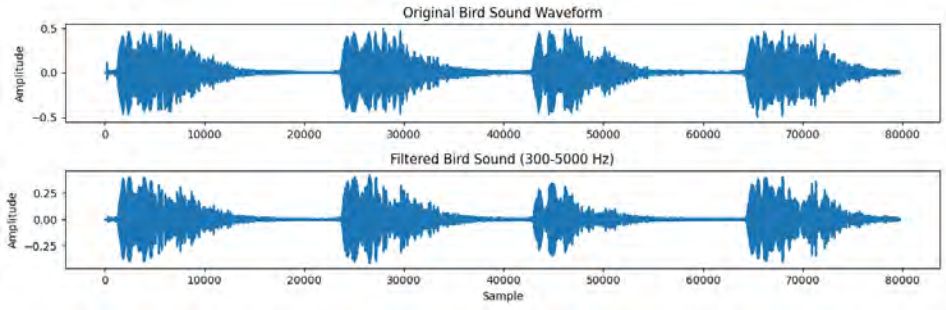

Figure 4.3: Original bird sound waveform (top) and the same waveform after applying a band-pass FIR filter (300–1500 Hz, bottom). Filtering removes low-frequency noise and DC offset, emphasizing the frequency band where most bird vocalizations lie. It can be hard to visually distinguish the effects of a band-pass filter on a raw time-domain waveform plot. Frequency content is better visualized using spectrograms.

Although the STFT provides detailed frequency information, many machine learning systems transform it to match human hearing better—the Mel scale.

4.4 Mel-Scale Frequency Analysis

While spectrograms show raw frequency content, human hearing is not linear: we are more sensitive to lower frequencies and less so to higher ones. The **Mel scale** models this perception and underpins the Mel spectrogram, a time-frequency representation widely used in audio deep learning, including CNNs for bird sound classification.

Mel Filter Bank

A Mel spectrogram applies a set of triangular filters spaced according to the Mel scale to the magnitude spectrogram:

- Each filter sums energy from a band of FFT bins
- Lower-frequency bands are narrower, higher-frequency bands are wider
- This emphasizes perceptually important frequencies

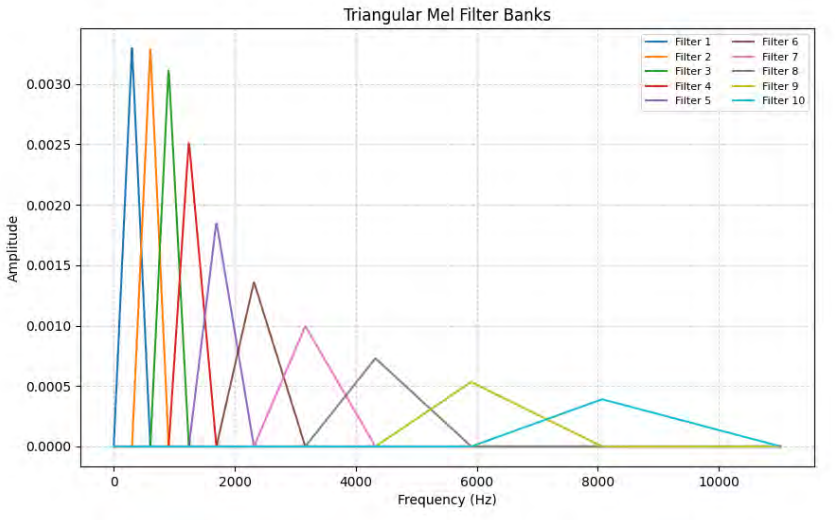

Figure 4.4: Mel filter bank applied to a linear-frequency spectrogram. Each triangular filter sums energy over a band of frequencies, with narrower spacing at low frequencies (more perceptually important) and wider spacing at high frequencies. This approximates how the human ear perceives pitch.

The standard formula to convert a linear frequency f in Hz to the Mel scale m is:

$$\text{Mel frequency: } m = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right)$$

And the inverse, converting from Mel m back to Hz f, is:

$$\text{Inverse: } f = 700 \cdot \left(10^{m/2595} - 1\right)$$

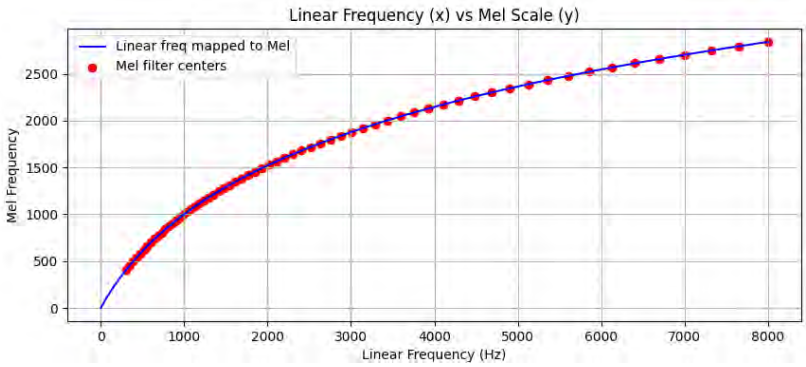

Figure 4.5: Comparison of linear-frequency (top) and Mel-scale (bottom) spectrograms of the same bird sound. The Mel scale stretches low frequencies and compresses high frequencies, emphasizing perceptually important details in the range where most bird vocalizations occur.

Why Mel Spectrograms Matter in Deep Learning

- Provides a compact, perceptually meaningful representation
- Reduces dimensionality compared to raw FFT spectrograms
- Retains important patterns for tasks such as speech or bird sound classification

Python Example: Computing a Mel Spectrogram

Listing 4.4: Generating a Mel spectrogram with Librosa

```
import librosa
import librosa.display
import matplotlib.pyplot as plt
import numpy as np

# Load the audio
y, sr = librosa.load("xc175830.mp3", sr=16000)
```

```
# Compute Mel spectrogram (Librosa >=0.10 requires keyword arguments)
S = librosa.feature.melspectrogram(y=y, sr=sr, n_fft=1024, hop_length=512,
n_mels=64, fmin=300, fmax=8000)
S_db = librosa.power_to_db(S, ref=np.max)

# Plot Mel spectrogram
plt.figure(figsize=(10, 4))
librosa.display.specshow(S_db, sr=sr, hop_length=512,
x_axis='time', y_axis='mel', fmax=8000, cmap='viridis')
plt.colorbar(format='%+2.0f dB')
plt.title("Mel Spectrogram of Bird Sound")
plt.tight_layout()
plt.show()
```

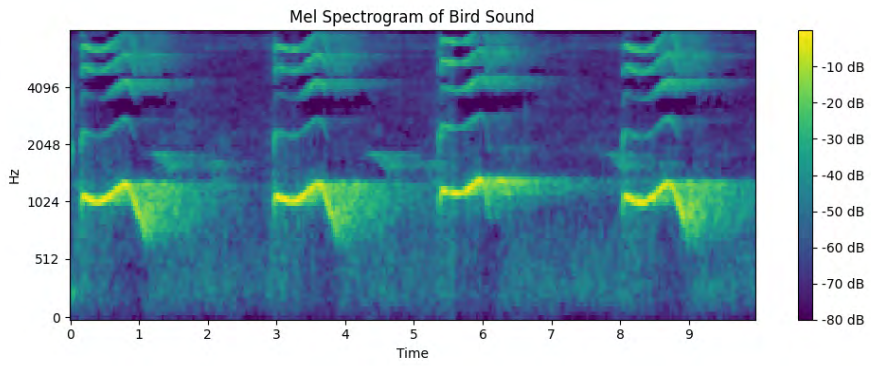

Figure 4.6: Mel spectrogram of a bird sound. This representation combines the time-frequency structure of a spectrogram with the perceptual weighting of the Mel scale. The narrower spacing at low frequencies reveals fine detail in the range where most bird vocalizations are concentrated, making it well-suited for CNN-based bird classification.

CNNs can consume Mel spectrograms directly, or you can compress them further into compact descriptors called MFCCs (Mel-Frequency Cepstral Coefficients).

4.5 Feature Extraction with MFCCs

MFCCs (Mel-Frequency Cepstral Coefficients) are another common audio representation. They compress spectral information into a smaller feature set that captures sound timbre:

Computation:

1. Compute the STFT of the audio.
2. Apply the Mel filter bank to the magnitude spectrum.
3. Take the logarithm of Mel energies.

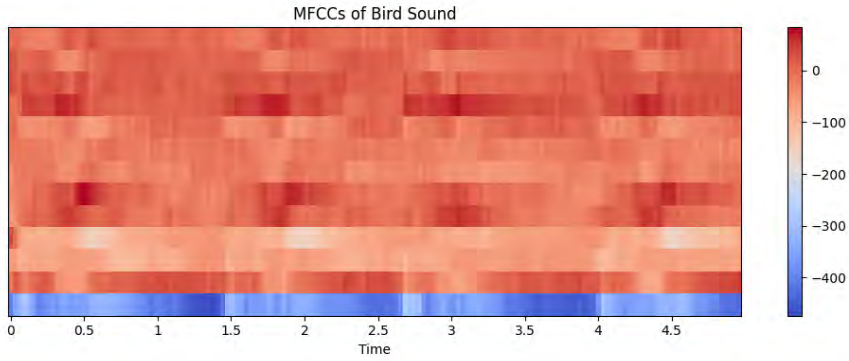

Figure 4.7: Mel-Frequency Cepstral Coefficients (MFCCs) extracted from a bird sound. MFCCs compress the Mel spectrogram into 12–13 coefficients per frame via Discrete Cosine Transform. While computationally efficient and popular in speech recognition, they can lose fine spectral detail compared to Mel spectrograms, particularly for bird sounds.

4. Apply the Discrete Cosine Transform (DCT) to get 12–13 coefficients per frame.

Why useful: MFCCs reduce dimensionality, emphasize perceptually relevant features, and are noise-robust.

Python Example: Extracting MFCCs from a Bird Call

Listing 4.5: Compute MFCCs from a bird sound

```
import librosa
import librosa.display
import matplotlib.pyplot as plt

y, sr = librosa.load("xc175830.mp3", sr=16000)

# Compute MFCCs
mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13, n_fft=1024, hop_length=512)

# Plot MFCCs
plt.figure(figsize=(10,4))
librosa.display.specshow(mfccs, x_axis='time', sr=sr, hop_length=512, cmap='
    coolwarm')
plt.colorbar()
plt.title("MFCCs of Bird Sound")
plt.tight_layout()
plt.show()
```

4.6 Comparing Feature Representations

The three representations in this chapter—STFT spectrogram, Mel spectrogram, and MFCC—form a hierarchy of increasingly compressed views of the same audio (Fig. 4.8):

- **STFT Spectrogram:** Simplest to compute; efficient but less precise.
- **Mel Spectrogram:** Best accuracy at moderate cost; most common in bioacoustics.
- **MFCC:** Widely used in speech; more complex to compute; generally less accurate than Mel spectrograms for bird sounds.

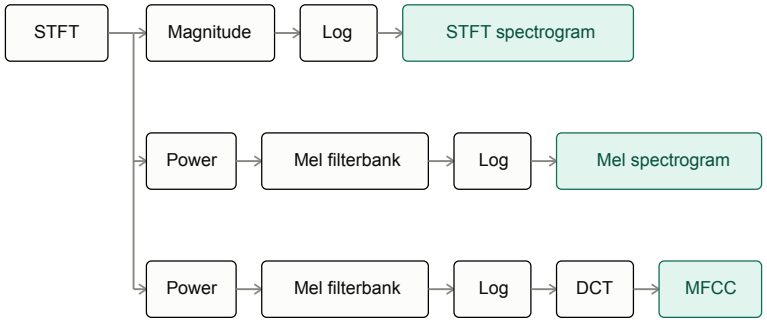

Figure 4.8: The audio feature extraction pipeline: from raw waveform through STFT, Mel spectrogram, and MFCC. Each representation trades off temporal and spectral resolution differently. Mel spectrograms are most commonly used for bird sound classification due to their balance of perceptual relevance and computational efficiency.

4.7 Key Takeaways

- Digital audio is sampled and quantized. The Nyquist theorem sets the limit: to represent frequency f, sample at least $2f$ times per second. 16 kHz suffices for most Malaysian birds; BirdNET uses 48 kHz.
- Time-domain waveforms hide the frequency content that distinguishes species. The frequency domain is where bird calls become distinguishable.
- The FFT efficiently computes the DFT. The STFT applies FFTs to overlapping windows, tracking how frequency content evolves. A spectrogram is the image formed by stacking STFT frames.
- Filtering (FIR/IIR) selects or suppresses frequency bands. A 300 Hz–5 kHz band-pass is the simplest way to improve signal-to-noise before feature extraction.

- The Mel scale models human perception—more resolution at low frequencies, less at high. Mel spectrograms and MFCCs are standard inputs for audio ML.
- Colormap choice affects readability. Perceptually uniform maps (`viridis`, `magma`) are better than legacy rainbow palettes.

4.8 Exercises

1. Generate a spectrogram of a bird sound with different FFT sizes (e.g., 256, 512, 1024). Compare the time–frequency resolution.
2. Compare the resolution of Mel filters in the low-frequency range versus the high-frequency range. Why does the Mel scale emphasize low frequencies more?
3. Extract MFCCs from the same bird sound using 13 coefficients and then 20 coefficients. Compare the feature sets.
4. Try different colormaps (`gray_r`, `magma`, `viridis`) for spectrogram visualization. Which one makes the patterns easiest to see?

4.9 Reflection

- Why does increasing FFT size improve frequency resolution but worsen time resolution?
- Why are Mel spectrograms better aligned with human hearing than linear spectrograms?
- Why are MFCCs popular in speech recognition, but sometimes less effective in bird sound analysis?
- How might the choice of features (spectrogram vs. Mel vs. MFCC) affect neural network performance?

References

[1] Alan V. Oppenheim and Ronald W. Schafer. *Discrete-Time Signal Processing*. 3rd. Comprehensive textbook on digital signal processing and discrete-time systems. Prentice Hall, 2010. ISBN: 9780137081899.

[2] McFee, Brian and others. *Librosa Documentation*. https://librosa.org/doc/latest/index.html. Accessed: 2025-09-09. 2025.

[3] Brian McFee et al. “librosa: Audio and Music Signal Analysis in Python.” In: *Proceedings of the 14th Python in Science Conference*. Ed. by Kathryn Huff and James Bergstra. 2015, pp. 18–25.

[4] S.S. Stevens, J. Volkmann, and E.B. Newman. “A Scale for the Measurement of the Psychological Magnitude of Pitch.” In: *Journal of the Acoustical Society of America* 8.3 (1940). The paper that introduced the Mel scale, pp. 185–190. DOI: 10.1121/1.1915700.

[5] Pete Warden and Daniel Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Chapter 3 introduces feature extraction for audio. O’Reilly Media, 2019. ISBN: 9781492052043.

Chapter 5

Requirements and Architecture

MVP (Minimum Viable Product) is the smallest feature set that delivers user value. For ZamZam, this means a working bird classifier for Android and iOS without extra features.

Learning Objectives

- Write a one-page MVP scope for ZamZam that a small team can ship in ten weeks.
- Draw the audio pipeline from microphone to on-screen prediction.
- Decide the questions that must be answered before writing any Flutter code: clip vs streaming, offline scope, target species list, top-K predictions, target device tier.
- Set up the development environment: Flutter SDK, Android Studio / Xcode, an emulator or physical device.

This chapter maps to **Week 1** of the intern work plan (`10WEEK_PLAN.md`).

5.1 Scope: What Ships in the MVP

Before touching a text editor, agree on what the first shippable version does. The MVP is deliberately narrow:

1. Record audio on the phone.
2. Run inference entirely **on-device**.
3. Display the top 3 predicted Malaysian bird species with confidence.
4. Show each species with three names: Malay common name, English common name, and scientific name.
5. Store detections locally with a timestamp.
6. Ship the same codebase to both Android and iOS.

Everything else—GPS tagging, maps, sharing, streaming, expanded species—is v2 or later. Say no now to ship a working app in ten weeks.

5.2 The Questions to Answer First

Resist jumping straight into Flutter. A few early decisions shape everything that follows. Changing direction mid-project requires rewriting major components. Fortunately, sensible defaults exist for a first version. Unless you have good reason to differ, these recommendations will keep the project manageable and on schedule.

5.2.1 Clip Recording or Live Listening?

Should the app listen continuously, or should users press a button to record a short clip?

Continuous listening sounds polished but is surprisingly difficult—continuous capture, overlapping windows, battery efficiency, and responsive UI. For v1, keep it simple: record a 5-second clip, run inference, display results. Add live listening in v2.

5.2.2 Offline or Cloud Processing?

Should the recording be sent to a server, or should everything happen on the phone?

Cloud processing enables larger models but requires internet—impractical for forest birding. On-device inference works anywhere, provides instant results, and avoids server maintenance. TensorFlow Lite is designed for mobile, making it the default.

5.2.3 How Many Results Should You Show?

Bird sounds are rarely unambiguous—similar species and background noise complicate identification. Display the top three predictions with confidence scores rather than a single answer. This lets users see alternatives if the model is uncertain.

5.2.4 How Many Bird Species?

Don't try to recognise every bird in Malaysia.

Start with around 50 common species. This is enough to make the app useful while keeping the dataset and training effort manageable.

Once the app is working, adding more species is much easier than trying to build everything at once.

5.2.5 Which Phones Should You Test On?

Don't develop only on a flagship phone.

If your app runs smoothly on a mid-range Android device, it'll usually run even better on more powerful phones. This also gives you a realistic idea of inference speed and battery usage.

5.2.6 Language

Keep the interface in English for Version 1.

You can still display both English and Malay bird names. That gives the app a local flavour without doubling the amount of interface text you have to maintain.

5.2.7 Recommended Choices

If you're unsure, use these settings for your MVP.

- 5-second clip recording
- Offline inference using TensorFlow Lite
- Top three predictions with confidence scores
- Around 100 common Malaysian bird species
- Test primarily on a mid-range Android phone
- English interface with English and Malay bird names

5.3 The Audio Pipeline

The end-to-end data flow is small enough to fit on one page:

Every subsequent chapter takes one row of this pipeline and shows how to build it in Flutter.

5.4 System Architecture

As the project grows, resist dumping every file in one folder. Clean structure improves readability, testability, and extensibility.

The application is divided into five independent modules, each with a single responsibility. Whenever you write a new class, ask yourself which module it belongs to. If a file seems to belong to two modules, it is probably doing too much.

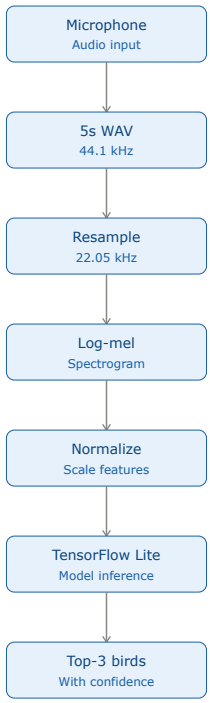

Figure 5.1: Full audio pipeline.

Figure 5.2 shows the high-level organisation of the application.

5.4.1 The `audio` Module

The `lib/audio/` directory contains everything related to recording sound from the microphone.

Typical classes include:

- microphone permission handling
- audio recording
- WAV file encoding
- temporary file management

This module should know nothing about neural networks or bird species. Its only job is to produce clean audio samples.

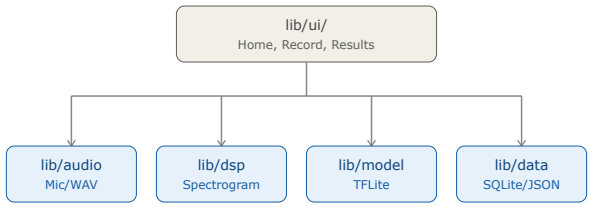

Figure 5.2: ZamZam's five-module Flutter application structure inside `lib/`. Each module has one responsibility and depends only on lower layers. This is the *app*'s architecture; Flutter's own engine architecture is discussed in Ch 10.

5.4.2 The `dsp` Module

The `lib/dsp/` directory converts raw audio into the format expected by the neural network.

Typical processing steps include:

- band-pass filtering
- resampling
- log-Mel spectrogram generation
- normalization
- tensor preparation

This module performs digital signal processing only. It should never display dialogs, access the database, or load TensorFlow Lite models.

5.4.3 The `model` Module

The `lib/model/` directory performs bird recognition.

Its responsibilities include:

- loading the TensorFlow Lite model
- creating the interpreter
- running inference
- sorting prediction scores
- returning the top predictions

The model module should not know where the audio came from or how the results will be displayed. It simply accepts an input tensor and returns predictions.

5.4.4 The `data` Module

The `lib/data/` directory stores information that must persist after the application closes.

This includes:

- SQLite detection history
- species metadata
- application settings
- JSON files containing English, Malay, and scientific names

Keeping all persistent storage in one place makes it much easier to replace SQLite or modify the data format later.

5.4.5 The `ui` Module

Everything the user sees belongs in `lib/ui/`.

Typical screens include:

- Home
- Record
- Results
- Detection History
- Settings

The UI should contain as little processing as possible. It should collect user input, call the appropriate modules, and display the results. Heavy computation belongs elsewhere.

5.4.6 Module Dependencies

The modules are intentionally kept independent.

The `audio`, `dsp`, `model`, and `data` modules should not depend on one another. This makes each module easier to test and allows individual components to be replaced without affecting the rest of the application.

Only the UI layer coordinates the entire application. When the user presses the Record button, the UI asks the audio module to record a clip, passes the recording

to the DSP module, sends the generated tensor to the model, and finally saves the result through the data module.

Keeping the dependencies flowing in one direction avoids circular references and helps prevent the codebase from becoming tightly coupled.

Figure 5.2 illustrates these relationships.

5.5 Development Environment

Installing the toolchain (Flutter SDK, Android Studio, Xcode, emulators, and confirming with `flutter doctor`) is covered end-to-end in Chapter 8. This chapter focuses on the *what* and *how* of the app; Chapter 8 sets up the *where you build it*. In Week 1 the team should install the toolchain in parallel with the architecture work here — there is nothing in the install steps that depends on the decisions made in this chapter, and nothing in this chapter that depends on the toolchain being running.

Once installation is done, the project skeleton the rest of the book builds on comes from either `git clone` of the shared repo followed by `flutter pub get`, or (for sandbox experiments) `flutter create`. Both are covered in Chapter 8.

5.6 Deliverables for Week 4

- One-page architecture diagram (pipeline + module layout).
- Written MVP scope with each Week-1 decision recorded.
- Empty Flutter project pushed to Git, building on both platforms (see Ch 10 for the install steps).

5.7 Key Takeaways

- Week 1 is about *deciding*, not coding. Skipping the requirements phase forces expensive rewrites later.
- An MVP scope is a written contract with yourself: what ships in the first release, and what explicitly does not.
- The pipeline diagram (microphone → DSP → model → UI) is the map for the next nine weeks; every later chapter fills in one box.
- Module boundaries are architectural, not just organisational — keeping audio capture, DSP, model inference, and UI as separate modules lets you test each in isolation and swap implementations without cross-module rework.

- By the end of Week 1 you should have: a one-page MVP scope, a pipeline diagram, a folder skeleton, and an empty Flutter project building on both Android and iOS.

5.8 Exercises

5.1 Draw the audio pipeline for a v2 of ZamZam that also identifies frog calls. What would change? What stays the same?

5.2 Argue in one paragraph for *streaming* inference instead of clip-based, then argue against it. Which side wins for MVP?

5.3 List three MVP features you would cut if the timeline shortened from 10 to 6 weeks.

5.9 Reflection

- Which Week-1 decision do you think will bite the team hardest if it turns out to be wrong?
- If your project were commercial rather than educational, which decisions would change?

Chapter 6

User Interfaces for Mobile Apps

Learning Objectives

- Understand what a user interface (UI) is and why it matters for a mobile app.
- Recognise the touchscreen UI elements available on a smartphone.
- Apply core UI design principles to a mobile app.
- Follow a lightweight design process — empathise, define, ideate, prototype, test — when planning your own app.
- Choose the right level of fidelity (sketch, wireframe, mockup, prototype) for each design stage.
- Run a small usability test with real users.

6.1 Why the UI Matters

A mobile app is only as useful as the interface people use to reach it. Whether the app is a birding tool, a note-taker, or a health tracker, users do not care about the code, the model, or the backend — they want to open the app, accomplish their task, and get out.

A well-designed UI turns your engineering work into a product. A poor UI can bury even excellent functionality behind confusing menus and unclear feedback. According to [1], the user interface typically accounts for about half of the design time, implementation time, maintenance time, and code size of a software product. That share is even higher on mobile, because the small screen leaves no room for clutter.

Two related terms are worth distinguishing:

- **User interface (UI)** covers what users see and touch — layout, colours, typography, buttons, icons, and how the screen responds.
- **User experience (UX)** covers the whole journey — from the moment a user first encounters a need, opens your app, completes the task, to how they share the outcome with others.

Great mobile apps get both right.

> *Note on tooling.* This chapter's activities — sketching, wireframing, running a five-user usability test — are all offline. You do not need Flutter installed, or even a laptop, to complete Ch 6. Flutter shows up in Ch 10; sketch first, code second.

6.2 Touchscreen UI Elements

Modern smartphones use **touchscreen user interfaces**. Unlike a desktop GUI driven by a mouse and keyboard, a phone UI is driven by fingers, gestures, sensors, and short bursts of attention. When you design your app, you are working with these building blocks:

Touch input — The screen detects taps and long-presses. A single tap should trigger the most common action.

Gestures — Swipe, pinch, and rotate replace many buttons. Swipe left/right to scroll through items, pinch-to-zoom to enlarge content.

Multi-touch — Recognising more than one finger enables pinch-to-zoom, two-finger rotation, or multi-finger selection.

Haptic feedback — A short vibration confirms that an action has occurred. This is invaluable when the user is not looking at the screen.

Visual cues — Buttons, icons, colour changes, and small animations tell the user what is happening: highlighted state for active elements, checkmarks for success, amber or red for warnings.

Sensors — Microphone, GPS, accelerometer, gyroscope, and camera each offer new interaction possibilities beyond touch.

Common on-screen elements you will assemble from your mobile framework (SwiftUI, Jetpack Compose, Flutter, React Native) are the familiar buttons, text fields, sliders, toggles, lists, and dialog boxes. The mobile platform's design system (Material Design on Android, Human Interface Guidelines on iOS) already provides polished versions of each. Use them — do not reinvent them.

6.3 Design Principles for Mobile Apps

The classical principles of UI design apply on mobile, but each takes on a specific flavour when the user is holding a phone in one hand, often outdoors, with limited attention:

- **Ease of use and learnability** — A first-time user should be able to complete the primary task within seconds of opening the app. No sign-up walls, no tutorials that block the main flow.
- **Consistency** — Use the platform conventions. The back button, the share icon, and the settings gear should look and behave as they do in every other app on the user's phone.
- **Feedback and response time** — The user must know the app is working. Live progress indicators are far more reassuring than a static "loading..." label. Return results within one or two seconds where possible.
- **Error prevention and recovery** — Warn the user before destructive actions, and offer a clear path to recover from mistakes. Prefer undo over confirmation dialogs.
- **Efficiency** — Reduce the number of taps needed to complete the core task. The best apps open directly to the main action screen.
- **Flexibility** — Offer light and dark modes, configurable settings, and optional overlays for power users.

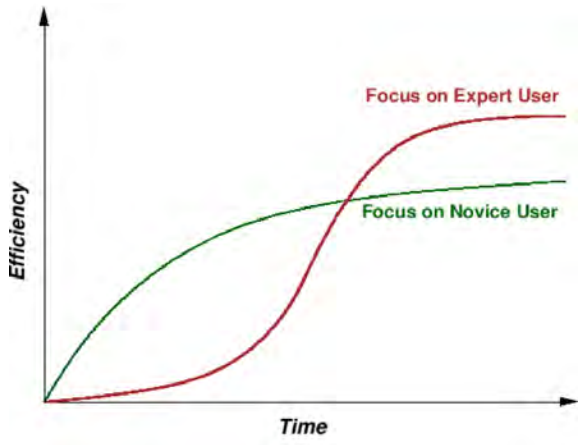

Figure 6.1: Novice users prefer software that is easy to learn even if less efficient in the long run; expert users tolerate a harder learning curve for higher steady-state productivity. Design decisions should be explicit about which curve you are optimising for.

- **Aesthetic and minimalist design** — Show what is necessary; hide the rest. The main screen should feature the primary action prominently and little else.

Some **user-centred** considerations that shape mobile apps in particular:

- **Limited short-term memory** — Users can typically hold about seven items at once. Do not present long lists with equal weight; rank by relevance and show the top few results.
- **People make mistakes** — Provide an undo option rather than a modal confirmation dialog on every action.
- **People are different** — Support accessibility: larger fonts, high-contrast mode, VoiceOver / TalkBack labels for every button.
- **Interaction preferences vary** — Where possible, let the user choose between list, grid, or map views of the same data.

6.4 A Lightweight Design Process

Even a solo developer benefits from following a light version of the classical **design thinking** process [2]. Five steps, shown in Fig. 6.2:

Empathise — Talk to real users. Watch them use existing apps in a similar domain. Note what frustrates them: too many taps to complete an action, tiny buttons, poor performance in low light, or unclear feedback.

Define — Write a one-sentence problem statement that captures who the user is, what they need, and under what constraints.

Ideate — Sketch many alternatives. Should the main action be one big button

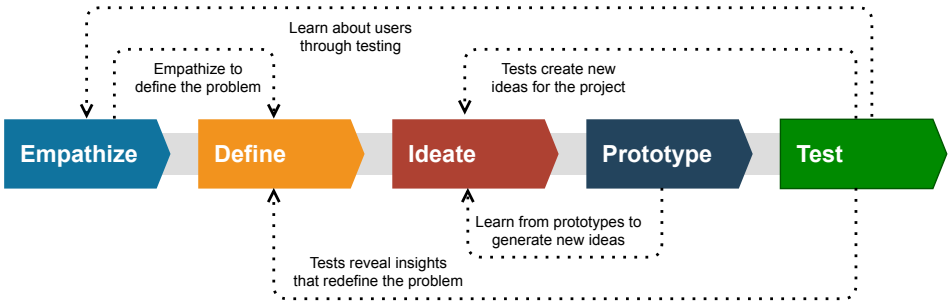

Figure 6.2: The five stages of the design thinking process. The dotted arrows show that the process is iterative — tests reveal new insights that feed back into earlier stages.

or a smaller icon? Should the result screen show a summary or full detail? Do not judge ideas at this stage — generate many.

Prototype — Build a low-fidelity version first (paper sketches, then digital wireframes). Only later invest in a working prototype that runs on a phone.

Test — Give the prototype to a small number of users and watch them use it. Iterate.

Design decisions become sharper when you have a specific person in mind. A useful reference persona describes a real user in enough detail to make trade-offs concrete: their age, their device, the context in which they use the app, and what they hope to achieve. For example, a birding-app persona might read: "Aiman, 24, a biology undergraduate. Owns an Android phone. Wants to confirm bird IDs on weekends without relying on a data connection."

Fidelity: Sketch, Wireframe, Mockup, Prototype

Designers use four levels of fidelity, each answering a different question:

- **Sketch** — pencil on paper. Answers: "Roughly what does this screen do?"
- **Wireframe** — greyscale digital layout. Answers: "Where does each element go, and how do screens link together?"
- **Mockup** — coloured, styled, static screen. Answers: "What does the finished app look like?"
- **Prototype** — interactive, clickable version. Answers: "What does it feel like to use?"

Fig. 6.3 shows the four fidelities applied to the same mobile screen, from a rough pencil sketch to a fully styled interactive prototype.

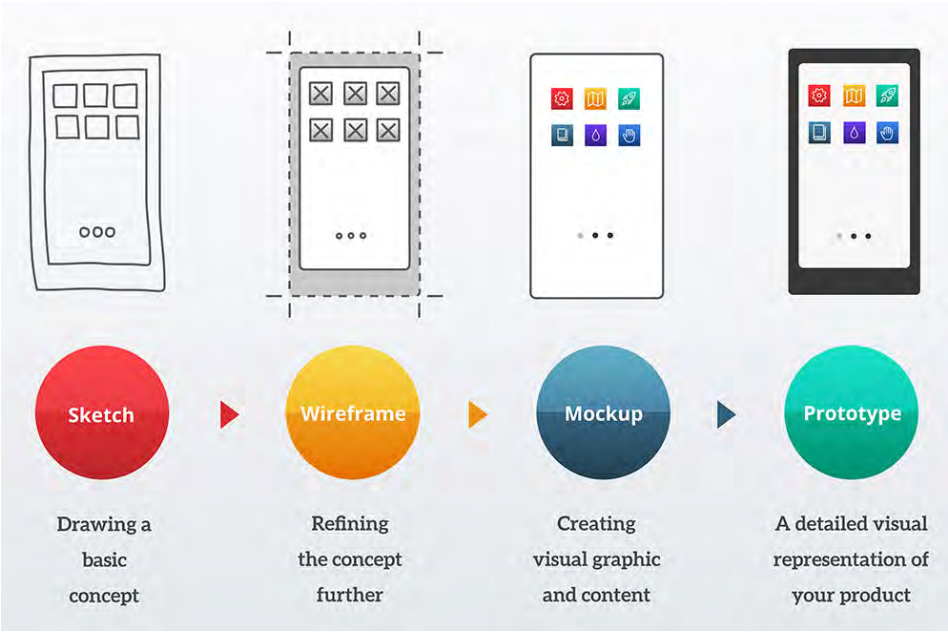

Figure 6.3: From sketch to prototype: the same mobile screen at increasing levels of fidelity. Each level answers a different question and takes progressively more time to produce.

Tools such as Figma, Sketch, and Adobe XD let you move between these fidelities without switching apps. A common shortcut is to sketch four or five key screens on paper first, then jump straight to a clickable Figma prototype. Skip mockups if you are short on time — the platform design system already handles most of the visual styling.

Table 6.1 compares the four fidelity levels.

Table 6.1: Fidelity levels for mobile app design

Sketch	Wireframe	Mockup	Prototype
Low-fidelity		Medium	High
Paper	Digital	Digital	Digital
Basic idea	Layout & flow	Visual style	Interactive
Minutes	Hours	Hours to days	Days

6.5 Usability Testing With Five Users

[3] showed that most usability problems can be uncovered with just five test users — adding more users yields diminishing returns for the effort spent (Fig. 6.4). For a small student team, three rounds of five users each is far more valuable than a single test with fifteen.

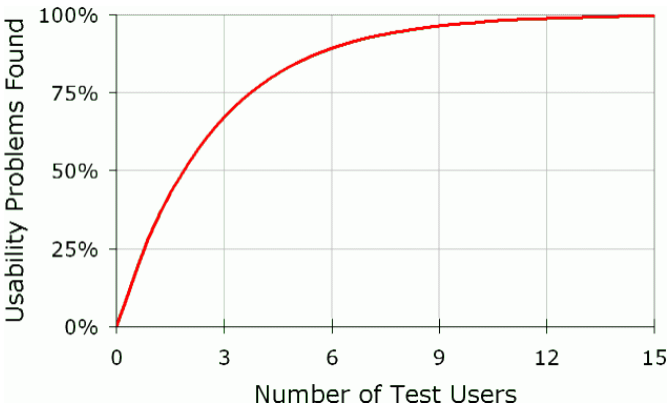

Figure 6.4: Usability problems found as a function of the number of test users [3]. About 80% of problems surface with the first five users; the curve flattens sharply after that.

A simple protocol for testing your app:

1. Recruit five people who match your target persona.
2. Prepare three tasks that cover the primary flow, a secondary flow, and a settings change.
3. Ask each participant to **think aloud** while attempting the tasks. Record their voice (with permission).
4. Take notes on where they hesitate, tap the wrong button, or misinterpret the result.
5. After the session, ask three questions: *What did you like? What frustrated you? What would you change first?*

Evaluate the results against five usability components liste in Table 6.2.

Table 6.2: Usability heuristics for mobile field applications

Heuristic	Evaluation Metric
Learnability	How quickly a first-time user completes the primary task.
Efficiency	How fast an expert user gets to a result after months of use.
Memorability	Can a user who has not opened the app in a month still use it easily?
Error handling	Does the app help the user recover when something goes wrong?
Acceptability	Would the user recommend the app to a friend?

6.6 Trends and a Case Study

Trends That Matter for Mobile Apps

Several ongoing trends shape modern mobile app design:

- **Voice interfaces** — Integrating with Siri or Google Assistant lets users trigger actions hands-free.
- **On-device AI** — Running models on the phone (no cloud round-trip) preserves privacy, cuts latency, and works without a network connection.
- **Responsive and adaptive design** — Your app should look right on a small budget phone, a large flagship phone, and a tablet.
- **Dark mode** — Dark mode saves battery on OLED screens and reduces glare in low-light environments.
- **Microinteractions** — Small animations on state changes, subtle sounds on completion, or a haptic tap on confirmation all make the app feel alive without adding clutter.
- **Augmented reality** — Emerging apps overlay information on the camera view, opening new interaction possibilities.

Case Study: Shazam's UI Rearchitecture

The music-identification app Shazam has undergone multiple UI rearchitectures that offer useful lessons for any single-action mobile app. Originally, Shazam's interface was centred entirely on a single "tap to identify" affordance — a large blue button that dominates the screen. This design philosophy reflected a core insight: the user's attention is split between looking at their device and engaging with the world. The UI must therefore:

- **Minimise cognitive load.** No menus, no choices, no distractions. One big button. Tap it.

- **Optimise for glance and go.** The result should appear within seconds and be readable at a glance.
- **Provide rich, non-textual feedback.** Animations (the circular pulse around the button during listening), haptics (the phone vibrating when a match is found), and subtle sound cues (a musical note on success) all communicate status without requiring the user to read text.
- **Support the main flow first, secondary flows second.** The primary path is trigger → wait → see result. Settings, history, sharing, and login come later (or are deferred to secondary screens).

Over time, Shazam added features (search history, sharing to Spotify, song lyrics) without cluttering the main screen. They achieved this through **progressive disclosure**: the main screen shows only what is needed to perform the primary action; secondary features are a swipe or button-press away. A history screen appears when the user swipes up. Sharing options appear after a result. Settings hide behind a gear icon in the corner. This hierarchy respects the user's moment — they want a result right now, but might want to share or save later.

The general principle: **optimise for the 80% use case, and make the 20% easy to find without impeding the 80%**.

Design documentation from this case study are adapted from [4].

6.7 Key Takeaways

- The UI is where your engineering work meets its audience — a poor UI can bury excellent functionality.
- Respect the platform: Material Design on Android, Human Interface Guidelines on iOS. Do not reinvent the back button, share icon, or settings gear.
- Apply core principles: learnability, consistency, feedback, error recovery, efficiency, flexibility, and minimalism.
- Follow the five-stage design-thinking process: empathise, define, ideate, prototype, test. Iterate.
- Match fidelity to stage: sketch on paper for ideation, wireframe for layout, mockup for style, prototype for interaction — do not jump to high fidelity too early.
- Test with five users at a time. About 80% of usability problems surface with the first five participants; more users offer diminishing returns.
- Optimise for the 80% use case; make the remaining 20% (settings, history, sharing) easy to find without impeding the primary flow.

6.8 Exercises

6.1 Sketch the main screen of your app on paper. Include only the elements a first-time user needs.

6.2 Interview two potential users of your app. Write a one-paragraph persona for each based on what you learn.

6.3 Draft a one-sentence problem statement for your app that follows the SMART criteria (specific, measurable, achievable, relevant, time-bound).

6.4 List five gestures you could support in your app and describe what each would do.

6.5 Design an "uncertain result" or error screen. How does the app communicate the situation to the user without discouraging them?

6.6 Recruit five people and run the three-task usability test described in this module. Report the top three issues you observed.

6.7 Compare two existing apps in the domain of your project. Which takes fewer taps to reach the primary result? Which gives clearer feedback?

6.8 Propose a strategy for handling offline use. What does the app do when the network is unavailable?

6.9 Reflection

- Which of the design principles in this module is easiest for you to apply, and which is hardest?
- Have you ever abandoned an app because of its UI? What went wrong, and how would you avoid that mistake in your own app?
- If you had to cut half of your planned features to keep the app simple, which half would you cut, and why?

Bibliography

[1] Brad A Myers and Mary Beth Rosson. "Survey on user interface programming." In: (1992), pp. 195–202.

[2] Tim Brown. *Change by Design: How Design Thinking Transforms Organizations and Inspires Innovation*. New York: HarperBusiness, 2009.

[3] Jakob Nielsen. *Usability Engineering*. San Diego, CA: Academic Press, 1993. ISBN: 978-0125184069.

[4] Marie Lejeail. *Shazam UX/UI*. https://marie-lejeail.com/shazam-rearchitecture. A case study showing the redesign of Shazam's structure, from ideation to final visuals. ; accessed 2026-07-04.

Chapter 7

Bird Sound Datasets and Baseline Training

Learning Objectives

- Tell sound repositories apart from curated ML-ready datasets, and pick the right kind of corpus for a species-identification app.
- Read the ML-ready dataset table well enough to reject detection corpora when you need classification.
- Download MyGardenBird from Zenodo and load it into a Colab notebook ready for training.
- Fix the preprocessing conventions all downstream models expect, so training, export, and on-device inference stay in sync.
- Train a compact CNN on the 12-species baseline and read the accuracy and loss curves.
- Evaluate the trained model with a confusion matrix and per-species F1 to see where it actually fails.

Every ZamZam model in this book starts with the same data and the same twelve-species classifier. This chapter walks the full baseline: what MyGardenBird is and why it fits, how to load it into Colab, which preprocessing constants must be pinned before you write model code, and how to train and evaluate a compact CNN end-to-end. By the end you will have a saved `.keras` file, a matching `labels.txt`, and a confusion matrix that tells you where the model is confident and where it is guessing. Chapter 8 picks up from those artifacts to build the mobile-deployment bundle; Chapter 9 shows how to grow the species list once the baseline is stable.

7.1 Overview of Bioacoustic Datasets

Bioacoustic datasets sit at the foundation of every model that classifies animal sounds. They support ecological work on species distribution, abundance, and conservation [1], and they arrive in two flavours: **coarse-grained annotations**

that mark only presence or absence, and **fine-grained annotations** that identify the species. Classification apps need the fine-grained kind.

The persistent difficulty is that models transfer poorly across ecosystems. A CNN trained on European songbirds underperforms in a Malaysian rainforest because the vocalisations, the ambient noise, and the species mix are all different [2, 3]. If ZamZam is going to work in Kuala Lumpur, its training data has to come from Malaysia.

Sound Repositories vs. Curated Datasets

Two distinct kinds of resource meet this need. **Repositories** such as Macaulay Library and Xeno-Canto are large, community-driven archives of raw recordings across many species and habitats. They are where you go to source new material [4, 5]. **Curated ML-ready datasets** such as BirdCLEF and MyGardenBird are pre-segmented, labelled, and split into train/val/test folders. They are what you feed a training loop.

Table 7.1 lists top bioacoustics repositories used by researchers. Xeno-Canto in particular has strong Malaysian coverage. Figure 7.1 shows the geographic distribution of recordings for Malaysian species: dense in Peninsular Malaysia

Table 7.1: Major bioacoustic repositories and their suitability for machine learning. Species counts are approximate and continually increasing.

Repository	Primary focus	ML Suitability	Species
Macaulay Library	Largest curated wildlife media archive maintained by the Cornell Lab of Ornithology; widely used for research and model training	■■■■■	9,000+
Xeno-Canto	Community-contributed bird vocalisations with extensive global coverage; the primary source for many bird-sound datasets	■■■■■	12,900+ birds
Animal Sound Archive	Museum-curated collection spanning birds, mammals, amphibians, insects and other taxa	■■■□□	2,380+
iNaturalist	Citizen-science observations containing photographs and audio recordings linked to biodiversity records	■■■□□	5,500+ animals
FishSounds	Specialised repository of fish vocalisations and underwater acoustic recordings	■■■□□	1,185 fish
AmphibiaWeb	Global reference collection of frog and amphibian calls	■■■■□	8,000+ amphibians
Borror Laboratory of Bioacoustics	Historic academic archive of bird and animal vocalisations used in ecological and behavioural research	■■■■□	Thousands
British Library Sounds	National archive of wildlife and environmental recordings from around the world	■■■□□	Thousands
MyBIS / ASEAN biodiversity portals	Regional biodiversity databases providing species metadata and occurrence records for Malaysia and Southeast Asia (limited audio)	■■□□□	Regional

Table 7.2: ML-ready bioacoustic datasets. Task: D = detection, C = classification. Annot.: C = clip-level, S = segment, W = weak. Lic.: O = open, R = restricted. Adapted from [6].

Dataset	Taxa	Task	Region	Clips	Sp.	Clip (s)	Annot.	Lic.
Bird — Detection (binary presence)								
BirdVox-DCASE-20k	Bird	D	N. America	20,000	—	10	W	O
Freefield1010	Bird	D	Europe	7,690	—	10	W	O
Warblr	Bird	D	Europe	8,000	—	10	W	O
TinyChirp	Bird	D	UK/Global	8,000	1	3	C	O
SEABAD	Bird	D	SE Asia	50,000	1,677	3	C	O
Bird — Classification (species-level)								
BirdCLEF 2024	Bird	C	Global	182,000	182	5–10	W	R
Picidae	Bird	C	Europe	1,669	7	var.	S	O
W. Mediterranean	Bird	C	Europe	5,795	20	var.	S	O
E. North America	Bird	C	N. America	12,000+	14	var.	S	O
Hainan Rainforest	Bird	C	China (Hainan)	12,187	46	5	C	O
Birdsdata	Bird	C	China	14,311	20	2	C	O
Gothenburg Urban PAM	Bird	D/C	Sweden	239,597	61	3	W	O
MyGardenBird	Bird	C	Malaysia	7,200	12	3	C	O
Non-bird bioacoustics (cross-taxa)								
AnuraSet	Amphibian	C	Brazil	93,000	42	3	C	O
FrogID	Amphibian	C	Australia	∼10,000	20+	var.	C	O
Bat Detective	Bat	C	Global	∼5,000	8–12	short	S	O
InsectSet459	Insect	C	Global	26,298	459	var.	S	O
HumBugDB	Insect	C	Global	Continuous	36	0.3	S	O
Watkins Marine Mammal	Marine mammal	C	Global	15,554	var.	var.	S	O
BioSoundSCape	Multi-taxa	D/C	S. Africa	825,832	not fixed	60	W	O
BEANS	Multi-taxa	D/C	Global	multi	multi	var.	C/S	O

and increasingly filled in across Sabah and Sarawak. That density is what makes MyGardenBird — and any future Malaysian-focused corpus — feasible to build.

Figure 7.1: Xeno-Canto bird recordings in Malaysia. Each dot represents one or more recordings; coverage is densest in Peninsular Malaysia and around urban centres.

ML-Ready Bioacoustic Datasets

Curated datasets are the shortcut. Table 7.2 catalogues the ones worth knowing along six dimensions: task type, geographic origin, scale, annotation granularity, clip duration, and licensing [6].

How to Read the Table

Check the **Task** column first. **D** (detection) means binary presence/absence: does the target sound appear anywhere in the clip? SEABAD, Freefield1010, and Warblr train lightweight gatekeeper models for this. **C** (classification) means species-level ID: which of these N species is calling? MyGardenBird, BirdCLEF, and InsectSet459 are classification corpora. **D/C** means the dataset supports both tasks.

The other columns set expectations for the model you can train. **Sp.** determines the softmax width: twelve species means a 12-way output; 182 needs a much larger model. **Clip (s)** tells you whether the preprocessing conventions match yours; three-second clips at 16 kHz fit the Tier 1/2 SRAM budget, but ten-second clips do not. **Annot.** indicates label granularity: **C** (clip-level) is ready to train on directly; **S** (segment) provides onset and offset times inside longer recordings; **W** (weak) means the label covers the whole clip even if the target only appears briefly, which tends to inflate reported accuracy.

ZamZam needs a **C** task, clip-level annotation, and a species list matching Malaysian garden birds. MyGardenBird is the fit: 7,200 clips, 12 species, three seconds at 16 kHz, pre-split, open licence. SEABAD's detection role belongs to an autonomous edge sensor listening on battery power, not a user-triggered mobile app.

Geographic Bias

Roughly 87% of labelled bioacoustic recordings originate in North America and Europe, even though tropical forests hold about 65% of global bird species diversity [6]. A classifier trained on temperate data will underperform systematically in Malaysian soundscapes, where the insect chorus is denser and the species mix is entirely different. Locally curated data like MyGardenBird is not a preference; it is a requirement.

7.2 MyGardenBird: The ZamZam Training Dataset

MyGardenBird [7, 8] is a machine-learning-ready dataset of twelve common Malaysian garden bird species, curated for Tier 1/2 embedded deployment and the ZamZam mobile classifier. Its design choices reflect the target hardware:

- **7,200 clips** — exactly 600 per species, derived from 1,381 unique source recordings.
- **3 seconds, 16 kHz 16-bit PCM mono WAV** — matches the model input tensor and the on-device audio buffer budget. A supplementary 44.1 kHz

version with 6,950 clips is also provided for applications that need higher fidelity.

- **12 Malaysian garden species**: Asian Koel, Collared Kingfisher, Common Iora, Common Tailorbird, Coppersmith Barbet, Large-tailed Nightjar, Olive-backed Sunbird, Pied Fantail, Spotted Dove, White-breasted Waterhen, White-throated Kingfisher, and Yellow-vented Bulbul.
- **Source:** Xeno-canto only, restricted to ASEAN/Indo-Malayan longitudes (60°–140° E) so recordings match the acoustic environment users will encounter.
- **Pre-split:** 80/10/10 train/val/test with mixed-integer programming (MIP) at the source-recording level. All clips from a given source recording stay in the same split, which eliminates the recording-specific leakage that random clip-level splitting would introduce.
- **Metadata:** `recordings.csv` (per-source provenance including quality grade, licence, coordinates, country), `clips.csv` (per-clip SNR, RMS, peak amplitude, clipping flag), and `splits_mip_80_10_10.csv` (train/val/test assignment). Full lineage from each clip back to its Xeno-canto source ID.
- **Validation:** BirdNET v2.4 zero-shot check reports 97.94% same-source label consistency at 16 kHz across all 7,200 clips, confirming the labels are free from systematic mislabelling.
- **Licence:** CC BY-NC-SA 4.0.

Baseline experiments with three CNN architectures on log-Mel features report 92–96% test accuracy, so published numbers are directly comparable to whatever you produce with the code below. The dataset is also the reference benchmark for MynaNet [9].

7.3 Using MyGardenBird in Colab

The dataset lives on Zenodo. Downloading, unzipping, verifying, and wrapping it in a `tf.data.Dataset` takes under five minutes on a Colab GPU runtime.

Step 1: Download from Zenodo

The Zenodo record at DOI `10.5281/zenodo.20306877` bundles all data and metadata into a single archive. Grab it and extract:

Listing 7.1: Download MyGardenBird from Zenodo

```
import os, zipfile, requests
from tqdm import tqdm
```

```
ZENODO_URL = "https://zenodo.org/records/20306877/files/mygardenbird.zip"
ZIP_PATH   = "/content/mygardenbird.zip"
DATA_ROOT  = "/content/mygardenbird"

if not os.path.exists(DATA_ROOT):
    print("Downloading MyGardenBird ...")
    with requests.get(ZENODO_URL, stream=True) as r:
        r.raise_for_status()
        total = int(r.headers.get("content-length", 0))
        with open(ZIP_PATH, "wb") as f, \
                tqdm(total=total, unit="B", unit_scale=True) as bar:
            for chunk in r.iter_content(chunk_size=8192):
                f.write(chunk)
                bar.update(len(chunk))
    print("Extracting ...")
    with zipfile.ZipFile(ZIP_PATH, "r") as z:
        z.extractall(DATA_ROOT)
    print(f"Done: {DATA_ROOT}")
else:
    print("Already downloaded.")
```

Step 2: Understand the Layout

The 16 kHz release lives under `mygardenbird16khz/`, with one folder per species and clip filenames encoding the Xeno-canto source ID plus the clip's onset time in milliseconds (`xc{source_id}_{onset_ms}.wav`). The train/val/test assignment is not encoded in the folder structure — it lives in a separate CSV so the same clips can be re-split under a different criterion later.

Listing 7.2: MyGardenBird repository layout

```
mygardenbird/
 mygardenbird16khz/
    Asian Koel/
       xc1002657_2860.wav
       ... (600 clips)
    Collared Kingfisher/
       ... (600 clips)
    ... (12 species folders, 7,200 clips total)
 mygardenbird44khz/          # 6,950 clips at native 44.1 kHz
 metadata16khz/
    recordings.csv          # per-source provenance
    clips.csv               # per-clip SNR, RMS, etc.
    qc_report.csv
    splits_mip_80_10_10.csv # file_id -> {train, val, test}
 metadata44khz/              # same schema, 44.1 kHz clips
 mygardenbirdplus/           # Common Myna + Zebra Dove addendum
 recordings.csv
 mygardenbird_code/          # curation pipeline (Stage1..Stage9)
```

Confirm the folder counts before proceeding:

Listing 7.3: Verify dataset structure and clip counts

```
import pathlib

DATA_16K = pathlib.Path(DATA_ROOT) / "mygardenbird16khz"
```

```
totals = 0
for sp_dir in sorted(DATA_16K.iterdir()):
    if sp_dir.is_dir():
        n = len(list(sp_dir.glob("*.wav")))
        totals += n
        print(f"  {sp_dir.name:<35} {n:>4}")
print(f"Total: {totals} clips (expect 7,200)")
```

Twelve species, 600 clips per species, 7,200 total. Any deviation means the download was truncated; re-run Step 1.

Step 3: Build the `tf.data` Pipeline

Rather than walk the filesystem for train/val/test, load `splits_mip_80_10_10.csv` and use it to route each clip to its assigned split. This keeps the pipeline honest against the source-level split the paper's MIP solver produced.

Listing 7.4: tf.data pipeline for MyGardenBird

```
import tensorflow as tf
import pandas as pd
import pathlib

DATA_16K   = pathlib.Path(DATA_ROOT) / "mygardenbird16khz"
META_16K   = pathlib.Path(DATA_ROOT) / "metadata16khz"
SPLITS_CSV = META_16K / "splits_mip_80_10_10.csv"
CLIPS_CSV  = META_16K / "clips.csv"

SR       = 16000
N_FFT    = 1024
HOP      = 512
N_MELS   = 128
BATCH    = 32
AUTOTUNE = tf.data.AUTOTUNE

# Species list from the folder names; sorted for a stable index
SPECIES     = sorted([p.name for p in DATA_16K.iterdir() if p.is_dir()])
NUM_CLASSES = len(SPECIES)    # 12
sp_to_idx   = {s: i for i, s in enumerate(SPECIES)}

# Join the split CSV with clip metadata to recover species per file_id
splits = pd.read_csv(SPLITS_CSV)               # columns: file_id, split
clips  = pd.read_csv(CLIPS_CSV)                # columns: file_id, ...
# recordings.csv links source_id to species; join through clips
recs   = pd.read_csv(pathlib.Path(DATA_ROOT) / "recordings.csv")
clips  = clips.merge(recs[["source_id", "species_common"]],
                     on="source_id", how="left")
all_df = splits.merge(clips[["file_id", "species_common"]],
                      on="file_id", how="left")

def load_wav(path):
    raw = tf.io.read_file(path)
    wav, _ = tf.audio.decode_wav(raw, desired_channels=1,
                                 desired_samples=SR * 3)
    return tf.squeeze(wav, axis=-1)          # (48000,)

def wav_to_logmel(wav):
    stfts   = tf.signal.stft(wav, frame_length=N_FFT,
                             frame_step=HOP)
```

```
    power   = tf.abs(stfts) ** 2
    filters = tf.signal.linear_to_mel_weight_matrix(
                  N_MELS, N_FFT // 2 + 1, SR, 0.0, 8000.0)
    mel     = tf.tensordot(power, filters, 1)
    logmel  = tf.math.log(mel + 1e-6)
    return logmel[..., tf.newaxis]           # (time, 128, 1)

def make_dataset(split):
    rows = all_df[all_df["split"] == split]
    paths  = [str(DATA_16K / row.species_common / f"{row.file_id}.wav")
              for row in rows.itertuples()]
    labels = [sp_to_idx[row.species_common] for row in rows.itertuples()]
    ds = tf.data.Dataset.from_tensor_slices((paths, labels))
    ds = ds.map(lambda p, l: (wav_to_logmel(load_wav(p)), l),
                num_parallel_calls=AUTOTUNE)
    if split == "train":
        ds = ds.shuffle(buffer_size=len(paths))
    return ds.batch(BATCH).prefetch(AUTOTUNE)

train_ds = make_dataset("train")
val_ds   = make_dataset("val")
test_ds  = make_dataset("test")

for x, y in train_ds.take(1):
    print(f"Batch shape: {x.shape}")   # (32, ~94, 128, 1)
```

Expected sizes: 5,760 training clips (480 per species), 720 val (60 per species), 720 test (60 per species).

Step 4: Save a Labels File for Deployment

The Zenodo archive does not ship a `labels.txt` because index order is a downstream choice. Write it out now in the same sorted order the pipeline uses, so the mobile side reads species names in the same order the model outputs them.

Listing 7.5: Save species labels in classifier output order

```
with open("labels_12sp.txt", "w") as f:
    f.write("\n".join(SPECIES))
for i, sp in enumerate(SPECIES):
    print(f"{i:>2}: {sp}")
```

With `train_ds`, `val_ds`, and `test_ds` in memory and `labels_12sp.txt` on disk, the pipeline is ready. The next section trains a classifier on top of it.

7.4 Preprocessing Conventions

Before writing model code, pin the preprocessing constants. Every model in this book expects the same input format, and mismatches here are the single most common cause of accuracy collapse once the model reaches the phone. Training (this chapter), TFLite export (Chapter 8), and Dart-side inference (Chapter 12) must all agree on:

- **Sample rate:** 16 kHz mono WAV.
- **Clip duration:** exactly 3 seconds (48,000 samples). Pad short clips with silence on the right; centre-crop long clips.
- **Feature:** log-Mel spectrogram with $N_{\text{FFT}} = 1024$, hop 512, 128 Mel bins, 0–8 kHz.
- **Normalisation:** $\log(S + 10^{-6})$ per clip. No global mean/std subtraction for the compact CNN.
- **Tensor shape:** `(time, 128, 1)` per clip — about 94 frames for a 3-second clip at the above settings.

Change any of these in one place and the model will silently misbehave in another. Treat them as invariants.

7.5 Training the 12-Species Classifier

With the pipeline in place and the conventions locked down, training is straightforward. This section builds a compact CNN, trains it to convergence, and evaluates it with a confusion matrix so you can see per-species performance rather than just an aggregate number.

The Compact CNN Architecture

Start with the smallest architecture that can demonstrate the full training loop: four convolutional blocks followed by global average pooling and a softmax head. Total parameters land near 180 K — comfortably inside the 1 MB INT8 budget that Chapter 8 will enforce.

Listing 7.6: Compact CNN for 12-species classification

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Reuse pipeline from earlier sections
# NUM_CLASSES = 12, input shape = (~94, 128, 1)

def build_compact_cnn(num_classes, input_shape=(94, 128, 1)):
    inputs = keras.Input(shape=input_shape)
    x = layers.Conv2D(32, 3, activation="relu",
                      padding="same")(inputs)
    x = layers.MaxPooling2D(2)(x)
    x = layers.Conv2D(64, 3, activation="relu",
                      padding="same")(x)
    x = layers.MaxPooling2D(2)(x)
    x = layers.Conv2D(128, 3, activation="relu",
                      padding="same")(x)
    x = layers.MaxPooling2D(2)(x)
    x = layers.Conv2D(128, 3, activation="relu",
```

```
                        padding="same")(x)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.3)(x)
    outputs = layers.Dense(num_classes,
                           activation="softmax")(x)
    return keras.Model(inputs, outputs,
                       name="zamzam_compact_cnn")

model = build_compact_cnn(NUM_CLASSES)
model.summary()
# Parameters: ~180K --- well within a 1 MB INT8 budget
```

Compiling and Training

Adam at 10^{-3} with sparse categorical cross-entropy is a safe starting point. Three callbacks handle the practical details: early stopping to avoid wasted epochs once validation accuracy plateaus, learning-rate reduction when the validation loss stalls, and a model checkpoint that keeps the best weights on disk.

Listing 7.7: Training the 12-species model

```
model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

callbacks = [
    keras.callbacks.EarlyStopping(
        monitor="val_accuracy", patience=8,
        restore_best_weights=True),
    keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss", factor=0.5,
        patience=4, min_lr=1e-6),
    keras.callbacks.ModelCheckpoint(
        "best_12sp.keras", save_best_only=True,
        monitor="val_accuracy"),
]

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=50,
    callbacks=callbacks,
)
```

On a Colab T4 GPU, 50 epochs on 5,760 clips takes roughly 8 minutes. The model usually converges around epoch 25–35 and reaches 88–92% validation accuracy on the balanced 12-species dataset.

Reading the Training Curves

Before trusting the number in `history`, plot the curves. A model that converges cleanly looks nothing like a model that has overfit or oscillated its way to a similar-

looking final accuracy.

Listing 7.8: Plot training and validation accuracy/loss

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, metric, title in zip(
        axes,
        [("accuracy", "val_accuracy"),
         ("loss",     "val_loss")],
        ["Accuracy", "Loss"]):
    for key, label in zip(metric,
                          ["Train", "Val"]):
        ax.plot(history.history[key], label=label)
    ax.set_title(title)
    ax.legend()
plt.tight_layout()
plt.show()
```

Four patterns tell you what happened:

- **Train and val curves converge together.** The model is learning and generalising. Ship it.
- **Val accuracy plateaus while train keeps rising.** Overfitting. Increase dropout, add augmentation (Chapter 8 shows how), or shrink the model.
- **Both curves oscillate.** Learning rate too high. Reduce the initial LR or let ReduceLROnPlateau fire earlier.
- **Val loss rises before epoch 20.** The dataset is too small for this architecture. Try a smaller CNN, or reach for augmentation before adding parameters.

Evaluation: Confusion Matrix and Per-Species F1

Aggregate accuracy hides the interesting failures. Two species that look similar on the spectrogram — Spotted Dove and Peaceful Dove, for instance — will confuse the model, but the confusion averages out across the softmax and never shows up in the top-1 number. A confusion matrix and a per-species classification report make those failures visible.

Listing 7.9: Confusion matrix and classification report

```
import numpy as np
from sklearn.metrics import (confusion_matrix,
    classification_report, ConfusionMatrixDisplay)

# Collect all predictions on the test set
y_true, y_pred = [], []
for x_batch, y_batch in test_ds:
    probs  = model.predict(x_batch, verbose=0)
    y_pred.extend(np.argmax(probs, axis=1))
    y_true.extend(y_batch.numpy())
```

```
y_true = np.array(y_true)
y_pred = np.array(y_pred)

# Confusion matrix
cm = confusion_matrix(y_true, y_pred)
fig, ax = plt.subplots(figsize=(10, 8))
ConfusionMatrixDisplay(cm,
    display_labels=SPECIES).plot(ax=ax,
    colorbar=False, xticks_rotation=45)
plt.tight_layout()
plt.show()

# Per-species F1
print(classification_report(y_true, y_pred,
      target_names=SPECIES))
```

On the 12-species baseline you will typically see three specific patterns:

- Collared Kingfisher and White-throated Kingfisher are the most-confused pair — both produce sharp, similar-frequency calls and occupy overlapping habitats.
- Large-tailed Nightjar has high precision but lower recall: its distinctive *tock-tock-tock* is unambiguous when detected, but it appears in fewer clips because it is a dusk/night caller.
- Common Iora and Pied Fantail overlap in frequency range and get swapped in noisier clips.

None of these are model bugs. They are honest reflections of how similar the acoustic signatures are, and they are what motivate the architecture choices in Chapter 8 and the data-expansion strategies in Chapter 9.

Finally, save everything you will hand to the deployment pipeline:

Listing 7.10: Save the 12-species model

```
model.save("zamzam_12sp.keras")

with open("labels_12sp.txt", "w") as f:
    f.write("\n".join(SPECIES))

print(f"Test accuracy: "
      f"{np.mean(y_true == y_pred):.1%}")
```

7.6 Key Takeaways

- Repositories supply raw recordings; ML-ready datasets supply pre-segmented, labelled, split data. Match the tool to the job.
- Check the Task column first: D is binary detection, C is species classification. ZamZam is a classification app; SEABAD's gatekeeper role belongs to

autonomous edge sensors.

- 87% of public bioacoustic clips come from North America and Europe. Using MyGardenBird is not a preference for a Malaysian deployment; it is a requirement.
- MyGardenBird gives you 7,200 clips, twelve species, three seconds at 16 kHz, balanced 600 per species, pre-split, CC BY-NC-SA 4.0.
- Pin the preprocessing conventions before writing model code. Sample rate, FFT size, hop, and Mel bins must stay identical across training, export, and on-device inference.
- Train the 12-species baseline to convergence and read the curves before trusting the accuracy number.
- Evaluate with a confusion matrix and per-species F1. Aggregate accuracy hides the interesting failures.
- Save `.keras` and `labels.txt` together; both feed the deployment pipeline in Chapter 8.

7.7 Exercises

7.1 Download MyGardenBird and run the verification script. Report the clip counts per species per split — do they match the expected 480/60/60?

7.2 Modify `wav_to_logmel` to use $N_{\text{FFT}} = 512$ and 64 Mel bins. Visualise one spectrogram at both settings side by side. What frequency detail is lost?

7.3 What would break if you used clip-level random splitting instead of source-aware splitting? Describe a scenario where the same recording appears in both train and test.

7.4 Train the compact CNN to convergence. Report test top-1 accuracy, macro F1, and the species pair with the highest confusion.

7.5 Look up one non-bird dataset in Table 7.2. What would need to change in the preprocessing pipeline to use it alongside MyGardenBird?

7.8 Reflection

- Why does balanced data (600 clips per species) matter more for a 12-class mobile classifier than for a 200-class server-side model?
- If you were adding a 13th species to the dataset, what two things would need to change before you could retrain?

- Your confusion matrix shows two species confusing each other in 40% of cases. Is that a data problem, a model problem, or an acoustic-similarity problem? How would you tell?

Bibliography

[1] Dan Stowell. "Computational bioacoustics with deep learning: a review and roadmap." In: *PeerJ* 10 (2022), e13152.

[2] Kristina Von Rintelen, Evy Arida, and Christoph Häuser. "A review of biodiversity-related issues and challenges in megadiverse Indonesia and other Southeast Asian countries." In: *Research Ideas and Outcomes* 3 (2017), e20860.

[3] K. Otter et al. "Continent-wide Shifts in Song Dialects of White-Throated Sparrows." In: *Current Biology* 30 (2020), 3231–3235.e3. DOI: 10.1016/j.cub.2020.05.084.

[4] Macaulay. *Macaulay Library Cornell Library of Ornithology*. http://macaulaylibrary.org. Accessed: 2024-11-25. 2024.

[5] Willem-Pier Vellinga and Robert Planqué. "The Xeno-Canto collection and its relation to sound recognition and classification." In: *CLEF (Working Notes)*. 2015.

[6] Muhammad Mun'im Ahmad Zabidi. "Edge AI for Avian Acoustic Monitoring: A Systematic Review and Deployment Framework." In: *IEEE Access* 4 (2026). In preparation.

[7] Muhammad Mun'im Ahmad Zabidi, Mohd Yamani Idna Idris, and Norisma Idris. "MyGardenBird: A Machine-Learning-Ready Bird Sound Dataset for Twelve Common Malaysian Birds." In: (2026). arXiv: 2606.06975 [cs.SD]. URL: https://arxiv.org/abs/2606.06975.

[8] Munim Zabidi et al. *MyGardenBird: A Machine-Learning-Ready Bird Sound Dataset for Twelve Common Malaysian Birds*. 7,200 manually reviewed 3-second WAV clips at 16 kHz, CC BY-NC-SA 4.0. Zenodo, 2025. DOI: 10.5281/zenodo.20306877. URL: https://zenodo.org/records/20306877.

[9] Muhammad Mun'im Ahmad Zabidi. *MynaNet: A Specialised Audio-Focused Lightweight CNN for Bird-Sound Classification*. MIT-licensed reference implementation; 267 KB INT8 model achieves 94.9% accuracy on the MyGardenBird 12-species dataset and outperforms MobileNetV3-Small on audio tasks. 2025. URL: https://github.com/mun3im/mynanet.

Chapter 8

Deploying the Model to Mobile

Learning Objectives

- Walk the five-step mobile-deployment pipeline: architecture, TFLite export, quantisation, verification, packaging.
- Pick an architecture appropriate for on-device inference — a custom compact CNN, MynaNet, or MobileNetV3-Small.
- Convert a trained Keras model to TensorFlow Lite (TFLite) with a single script.
- Apply post-training quantisation and reason about the accuracy/size/latency trade-off.
- Verify the exported `.tflite` matches the Keras original before shipping it to Flutter.
- Package the model with the metadata the Flutter side needs to reproduce training-time preprocessing exactly.

Chapter 7 produced a trained Keras classifier and a `labels.txt`. This chapter turns those two files into a self-contained bundle that ships inside the Flutter app's `assets/` folder and runs on a phone. Training itself is standard Keras; what makes this chapter mobile-specific is the export path, the compression trade-offs, and the packaging discipline that keeps preprocessing consistent from Colab to Dart.

8.1 The Mobile-Deployment Pipeline

Chapter 3 argued for running inference on the phone rather than in the cloud. That decision fixes the workflow: train on a Colab GPU with full TensorFlow, then export a compact TFLite model that fits inside the Flutter app.

The training half is architecture-agnostic and looks the same whether you eventually deploy on a laptop, a phone, or a microcontroller. What is mobile-specific is what happens *after* training:

1. **Choose a compact architecture.** The model has to fit inside a few tens of megabytes of app size and return a prediction in a few hundred milliseconds.

Not every well-performing architecture qualifies.

2. **Convert to TFLite.** A single `TFLiteConverter` call turns the `.keras` file into a `.tflite` file the mobile runtime can load. Mandatory.
3. **Quantise.** Float32 works but wastes size and battery. Float16 halves the model with essentially no accuracy loss; int8 halves it again with a small accuracy hit. Picking the right level is the second interesting decision.
4. **Verify.** A silent mismatch between training-time preprocessing and mobile-runtime preprocessing is the most common way to ship a broken model. A short verification script catches almost every such bug before it reaches a device.
5. **Package.** Produce a self-contained bundle for the Flutter side: the `.tflite` file, a `labels.txt`, and a JSON metadata blob that specifies every preprocessing parameter. Chapter 12 loads this bundle from `assets/` into the on-device interpreter.

The rest of this chapter walks the five steps in order.

8.2 Choosing an Architecture

Four architectures are worth considering for a mobile bird-ID model on Mel spectrograms, ordered from easiest to iterate on toward highest accuracy:

Custom compact CNN. Four to six convolutional blocks with global average pooling and a dense classifier. Under 500 KB before quantisation. Trains from scratch on a modest dataset (a few hundred clips per species) in an hour on a Colab T4. A good starting point and the model you already trained in Chapter 7.

MynaNet. An audio-focused CNN designed to beat MobileNetV3-Small on bird-sound tasks [1]. Reference implementation at https://github.com/mun3im/mynanet. Architecture: MBV2 inverted-residual blocks with hard-sigmoid Squeeze-Excitation (MBV3-style) and 5×5 depthwise convolutions in the deeper blocks for a wider spectral-temporal receptive field. Reports 94.9% INT8 accuracy at **267 KB** on the 12-species MyGardenBird dataset — half the size of MobileNetV3-Small at comparable-or-higher accuracy on audio. If your custom CNN plateaus and you do not want to jump straight to a much larger MobileNet backbone, MynaNet is the intermediate step.

MobileNetV3-Small. The mobile ML default for image-shaped inputs. Adapt it to accept a 128×128 Mel spectrogram by tiling the single channel to three. Around 2 MB after float16 quantisation [2]. Higher accuracy than a custom CNN, but harder to iterate on and designed for photographs rather than spectrograms.

EfficientNet-lite0. Higher accuracy per parameter than MobileNetV3, at the cost of somewhat higher inference latency. Around 5 MB float16. Worth trying once the smaller architectures saturate.

For ZamZam v1, the custom CNN suffices. If field accuracy is inadequate, MynaNet is the natural next step — best audio accuracy per kilobyte. Reach for MobileNetV3-Small or EfficientNet-lite0 only if MynaNet also underperforms. Architecture choice is independent of the export and quantisation steps that follow.

Listing 8.1: A compact CNN for Mel-spectrogram bird classification

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

NUM_SPECIES = 100  # ~100 Malaysian species in the ZamZam MVP
INPUT_SHAPE = (128, 128, 1)   # 128 Mel bins x 128 time frames
```

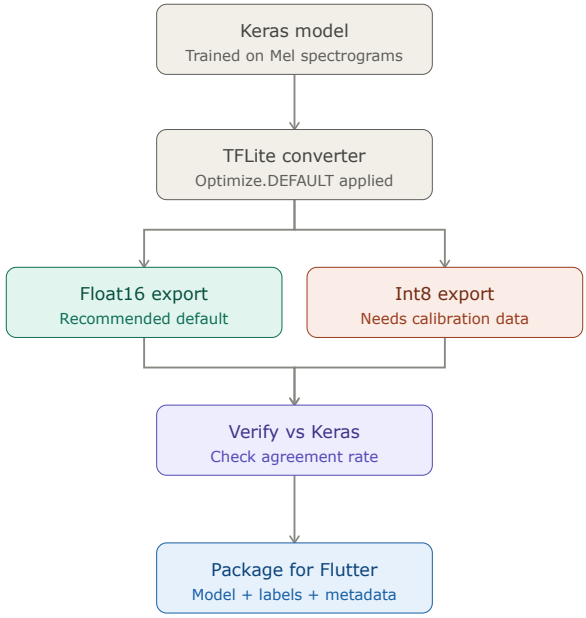

Figure 8.1: Mobile training and export pipeline: train a Keras model on desktop with full TensorFlow, then convert and quantise for TensorFlow Lite. The compact `.tflite` file ships inside the Flutter app alongside metadata describing the preprocessing parameters.

```
def build_model():
    inputs = keras.Input(shape=INPUT_SHAPE)
    x = layers.Conv2D(32, 3, activation="relu", padding="same")(inputs)
    x = layers.MaxPooling2D(2)(x)
    x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)
    x = layers.MaxPooling2D(2)(x)
    x = layers.Conv2D(128, 3, activation="relu", padding="same")(x)
    x = layers.MaxPooling2D(2)(x)
    x = layers.Conv2D(128, 3, activation="relu", padding="same")(x)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.3)(x)
    outputs = layers.Dense(NUM_SPECIES, activation="softmax")(x)
    return keras.Model(inputs, outputs, name="zamzam_compact_cnn")

model = build_model()
model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
model.summary()  # should print ~180K parameters
```

Train it with the 3-second Mel-spectrogram clips prepared in Chapter 7 and save the Keras model once accuracy is satisfactory.

Swapping the Model

The rest of this chapter — training loop, TFLite conversion, quantisation, verification, packaging — is architecture-agnostic. To swap the compact CNN in Listing 8.1 for one of the alternatives, replace only the `build_model()` function.

Switching to MobileNetV3-Small. Two small changes are needed. Keras applications expect three-channel input, so tile the single-channel Mel spectrogram along the channel axis:

Listing 8.2: Swap the compact CNN for MobileNetV3-Small

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

INPUT_SHAPE = (128, 128, 1)   # Mel spectrogram, single-channel
NUM_SPECIES = 100

def build_model():
    inputs = keras.Input(shape=INPUT_SHAPE)
    x = layers.Concatenate()([inputs, inputs, inputs])   # 1 -> 3 channels
    backbone = keras.applications.MobileNetV3Small(
        input_shape=(128, 128, 3),
        include_top=False,
        weights=None,           # train from scratch on Mel spectrograms
        pooling="avg",
    )
    x = backbone(x)
    x = layers.Dropout(0.3)(x)
    outputs = layers.Dense(NUM_SPECIES, activation="softmax")(x)
    return keras.Model(inputs, outputs, name="zamzam_mbv3s")
```

Switching to MynaNet. MynaNet is designed around a 64×300 Mel spectrogram (64 Mel bins by 300 time frames at 10 ms per frame, from a 3-second clip at 16 kHz) — a different input shape than the square 128×128 used by the compact CNN. Three coordinated changes are required: match the shape in your data pipeline, clone the MynaNet training script, and update the model metadata accordingly.

Listing 8.3: Adopt MynaNet by cloning the reference implementation

```
# Clone MynaNet (MIT-licensed)
!git clone https://github.com/mun3im/mynanet.git

# Train MynaNet on your dataset (path to per-species WAV folders)
!python mynanet/deploy/train.py \
  --flat_dir /path/to/mygardenbird16khz \
  --splits_csv /path/to/splits_mip_80_10_10.csv

# The script outputs bird_model_int8.tflite ready to ship.
# Skip the float16 export in the next section and use train.py's INT8
# output together with the verification and packaging steps below.
```

If you adopt MynaNet, update the JSON metadata your Flutter code will read in Chapter 12:

```
{
  "model_file":   "bird_model.tflite",
  "input_shape":  [1, 64, 300, 1],
  "sample_rate_hz": 16000,
  "n_mels":       64,
  "hop_length":   160,      // 10 ms per frame at 16 kHz
  "input_dtype":  "int8"
}
```

Keep the rest of the pipeline (verification, labels file, cross-check on known clips) unchanged.

8.3 Exporting to TFLite

TensorFlow's `TFLiteConverter` turns a Keras model into a `.tflite` file in a few lines. Three choices matter:

- **Optimisations.** `Optimize.DEFAULT` enables weight quantisation and other compression passes.
- **Target types.** `tf.float16` halves the model size versus float32 with negligible accuracy loss. `tf.int8` halves it again but requires a representative dataset for calibration.
- **Representative dataset.** A generator that yields a few hundred training-distribution inputs, used by TFLite to calibrate int8 quantisation ranges.

The following script exports both float16 and int8 versions so you can compare their sizes and accuracies before choosing one to ship.

Listing 8.4: Convert a Keras model to TFLite (float16 and int8)

```
import tensorflow as tf
import numpy as np

model = tf.keras.models.load_model("bird_model.keras")

# --- Float16 export (recommended default for mobile) ---
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.target_spec.supported_types = [tf.float16]
tflite_f16 = converter.convert()
with open("bird_model_f16.tflite", "wb") as f:
    f.write(tflite_f16)
print(f"float16 model: {len(tflite_f16) / 1024:.1f} KB")

# --- Int8 export (smallest, requires calibration data) ---
def representative_dataset():
    # 100 random samples from the training set
    for x in x_train[:100]:
        yield [x[np.newaxis, ...].astype(np.float32)]

converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_dataset
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8
tflite_int8 = converter.convert()
with open("bird_model_int8.tflite", "wb") as f:
    f.write(tflite_int8)
print(f"int8 model: {len(tflite_int8) / 1024:.1f} KB")
```

8.4 Post-Training Quantisation

Quantisation shrinks the model by storing weights in fewer bits. TFLite supports three modes, each with a different accuracy/size/tooling trade-off:

Dynamic-range quantisation. Weights stored as int8, activations kept in float. Easy (no representative dataset), $\sim 4\times$ size reduction, small latency win. Legacy default.

Float16 quantisation. Weights in float16. About $2\times$ smaller than float32, essentially no accuracy loss, no representative dataset required. **Recommended default for mobile.**

Full-integer (int8) quantisation. Both weights and activations in int8. $\sim 4\times$ smaller than float32, faster on integer-only hardware (mobile NPUs, Coral Edge TPU). Requires a representative dataset. Small accuracy hit — typically 1–2% for well-designed models.

Table 8.1 shows representative numbers for the compact CNN in Listing 8.1 on a 100-species ZamZam model. The exact figures depend on your architecture and data, but the ordering is stable.

Table 8.1: Quantisation trade-off for the ZamZam compact CNN (representative values).

Format	Size	Top-1 acc.	Latency (Pixel 6)	Notes
Keras (float32)	720 KB	82.1%	—	Reference
TFLite float32	715 KB	82.1%	42 ms	No compression
TFLite float16	360 KB	82.0%	38 ms	Recommended
TFLite dyn-range	190 KB	81.8%	35 ms	No calibration data needed
TFLite int8	185 KB	80.6%	22 ms	Needs representative dataset

For the ~ 180 K-parameter compact CNN, ship float16 — the size is already comfortable and the accuracy is indistinguishable from the reference. For a larger MobileNetV3-based model where APK size matters, ship int8 if you can spare the calibration step.

8.5 Verifying the TFLite Model

Never ship a `.tflite` file without confirming it matches the Keras original. TFLite conversion can silently drop ops, silently change output shapes, or — most commonly — silently break because of a preprocessing mismatch downstream. The verification script below runs both models on a held-out test set and prints the accuracy delta and the agreement rate.

Listing 8.5: Verify a TFLite model against the Keras source

```
import numpy as np
import tensorflow as tf

# Load both models
keras_model = tf.keras.models.load_model("bird_model.keras")
interpreter = tf.lite.Interpreter(model_path="bird_model_f16.tflite")
interpreter.allocate_tensors()
in_idx = interpreter.get_input_details()[0]["index"]
out_idx = interpreter.get_output_details()[0]["index"]

def tflite_predict(x):
    interpreter.set_tensor(in_idx, x.astype(np.float32))
    interpreter.invoke()
    return interpreter.get_tensor(out_idx)

# Run both on the test set
keras_preds = keras_model.predict(x_test).argmax(axis=1)
tflite_preds = np.array([tflite_predict(x[np.newaxis])[0].argmax()
                         for x in x_test])

keras_acc = (keras_preds == y_test).mean()
```

```
tflite_acc = (tflite_preds == y_test).mean()
agreement = (keras_preds == tflite_preds).mean()
print(f"Keras top-1:     {keras_acc:.3%}")
print(f"TFLite top-1:    {tflite_acc:.3%}")
print(f"Agreement rate:  {agreement:.3%}  (should be > 99%)")
```

If the agreement rate drops below 99%, something is wrong — usually a mismatch between the Mel-spectrogram parameters in training and the ones the mobile app will compute at runtime. Fix it here, not later in the Flutter debugger, where the loop takes ten times longer.

8.6 Packaging for Flutter

Hand the mobile developer a self-contained bundle so nothing needs to be reconstructed from memory. The minimum contents:

- `bird_model.tflite` — the exported model (float16 recommended).
- `labels.txt` — one species per line, in the exact order the model outputs. Index i in the softmax vector is the species on line i.
- `model_metadata.json` — input shape, sample rate, Mel bins, hop length, and any normalisation constants the audio pipeline must reproduce.

Listing 8.6: Model metadata bundled with the `.tflite` file for the Flutter side

```
{
  "model_file":      "bird_model.tflite",
  "input_shape":     [1, 128, 128, 1],
  "output_shape":    [1, 100],
  "sample_rate_hz":  16000,
  "clip_length_s":   3.0,
  "n_mels":          128,
  "n_fft":           1024,
  "hop_length":      512,
  "fmin_hz":         300,
  "fmax_hz":         8000,
  "input_dtype":     "float32",
  "normalisation":   "per_clip_db_scale",
  "labels_file":     "labels.txt"
}
```

If the Flutter app cannot reproduce the same Mel spectrogram the model was trained on, accuracy will drop — often catastrophically. Make every preprocessing parameter explicit in this file, and cross-check against a few known test clips end-to-end before considering the model shipped.

Transferring the Bundle From Colab to Your Flutter Project

The three files currently live inside your Colab notebook's ephemeral filesystem, which disappears when the runtime is recycled. Move them into the Flutter project before you close the tab. Three options, ordered from most to least manual:

Option 1: Direct download from the Colab file browser. Fastest for a one-off. In the Colab left sidebar, open the *Files* tab, right-click each file, and choose *Download*. Then drop them into your Flutter repo:

```
# On your laptop, after downloading the three files:
mkdir -p zamzam/assets
mv ~/Downloads/bird_model.tflite \
   ~/Downloads/labels.txt \
   ~/Downloads/model_metadata.json \
   zamzam/assets/
```

Option 2: Programmatic download from the notebook. Add a cell at the end of the training notebook that streams each file to the browser. Useful when you retrain often and want a repeatable last step.

Listing 8.7: Download the bundle as the final Colab cell

```
from google.colab import files

files.download("bird_model.tflite")
files.download("labels.txt")
files.download("model_metadata.json")
```

Option 3: Push to Google Drive or a Git repo. For team workflows where the Flutter developer is not the person training the model. Mount Drive from the notebook and copy the three files into a shared folder, or use a small script that pushes them to a versioned Git branch of the Flutter repo. Reproducible, but more setup.

Wiring the Bundle Into Flutter

Once the three files are inside `zamzam/assets/`, register them in `pubspec.yaml` so Flutter includes them in the build:

Listing 8.8: `pubspec.yaml` assets block for the model bundle

```
flutter:
  assets:
    - assets/bird_model.tflite
    - assets/labels.txt
```

```
    - assets/model_metadata.json
```

Run `flutter pub get` to refresh the asset manifest. Chapter 12 shows the Dart code that loads these assets at app start; from that point on, retraining is a matter of overwriting the three files in `assets/` and rebuilding.

8.7 Key Takeaways

- Training for mobile is standard Keras. What is mobile-specific is the export path (TFLite), the quantisation strategy, and the packaging discipline that keeps preprocessing consistent from Colab to Dart.
- Pick a compact architecture first: a custom CNN is enough for the MVP; MynaNet and MobileNetV3-Small are the natural next steps if accuracy plateaus.
- The five-step pipeline: choose architecture $\rightarrow$ convert to TFLite $\rightarrow$ quantise $\rightarrow$ verify $\rightarrow$ package. Skipping verification is the fastest way to ship a silently broken model.
- Float16 is the safe default: half the size, no measurable accuracy loss, no calibration data required. Int8 halves size again at a small accuracy cost, but needs a representative dataset.
- The most common shipping bug is a mismatch between training-time preprocessing and mobile-runtime preprocessing. Pin every parameter (sample rate, FFT size, hop length, Mel bins, dB reference, normalisation) in a JSON metadata file that ships with the `.tflite`.
- The deliverable to `assets/` in the Flutter project is three files: the `.tflite`, a `labels.txt`, and a `model_metadata.json`.
- Chapter 10 opens the Flutter project that hosts this bundle. Copy the three files into `assets/` before you run `flutter pub get`, and you will not have to touch them again until you retrain.

8.8 Exercises

8.1 Train the compact CNN in Listing 8.1 on Freefield1010 as a bird-vs-no-bird detector. Export both float16 and int8 versions. Report the sizes.

8.2 Run the verification script (Listing 8.5) on both exports. Report the accuracy delta versus the Keras source.

8.3 Measure inference latency on your laptop CPU using `time.perf_counter()` around `interpreter.invoke()`. Compare float16 and int8.

8.4 Replace the compact CNN with MobileNetV3-Small (via `tf.keras.applications.MobileNe` adapting the input layer to $128 \times 128 \times 1$. Compare model size, accuracy, and latency.

8.5 Write the `labels.txt` for your model in Malay + English + scientific-name format, one species per line. Verify the order matches the model's output indices.

8.9 Reflection

- Under what circumstances would you ship int8 instead of float16?
- How would you catch a Mel-spectrogram preprocessing mismatch between training and the Flutter runtime, before it reaches a user's phone?
- If your first-release model is 90% accurate on a 100-species set, would you ship it? What accuracy threshold is high enough?
- Post-training quantisation is one compression technique. What are the others (pruning, distillation, sparsification), and when would each be worth the extra complexity?

Bibliography

[1] Muhammad Mun'im Ahmad Zabidi. *MynaNet: A Specialised Audio-Focused Lightweight CNN for Bird-Sound Classification*. MIT-licensed reference implementation; 267 KB INT8 model achieves 94.9% accuracy on the MyGardenBird 12-species dataset and outperforms MobileNetV3-Small on audio tasks. 2025. URL: https://github.com/mun3im/mynanet.

[2] Andrew Howard et al. "Searching for MobileNetV3." In: *Proceedings of the IEEE/CVF international conference on computer vision*. 2019, pp. 1314–1324.

[3] Pete Warden and Daniel Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Chapter 3 introduces feature extraction for audio. O'Reilly Media, 2019. ISBN: 9781492052043.

Chapter 9

Expanding the Dataset: From 12 to 50+ Species

Learning Objectives

- Understand the provenance gradient that governs each expansion tier: Indo-Malayan core, broader-Asian Plus, global at 20 species, and imbalance-dominated at 50.
- Source recordings from Xeno-Canto and Macaulay Library using scripted queries.
- Apply quality-filtering heuristics before spending time on manual review.
- Segment long recordings into fixed 3-second clips using Audacity and the Stage4 annotator.
- Grow the species list from 14 to 20 species, accepting global sourcing where regional data is thin.
- Push toward 50 species and diagnose the class-imbalance failure modes that emerge.
- Apply class weights and per-species capping to recover minority-species accuracy.

Chapter 7 trained a working 12-species classifier on MyGardenBird. That model is enough to demonstrate the pipeline, but it identifies only a fraction of what a Malaysian birder hears in the field. This chapter walks the expansion: where to source new recordings, how to curate them, and what changes at each tier. The theme running through the chapter is *provenance*: how geographically strict you can afford to be shrinks as the species list grows, and by the time you reach fifty species, imbalanced data becomes the dominant challenge rather than recording origin.

The dataset taxonomy (repositories vs. curated corpora) and the reasons regional data matter were covered in Chapter 7. This chapter focuses on the practical side: sourcing, curation, and training adjustments at each expansion step.

9.1 Working with Datasets

Bioacoustic recordings are usually distributed as MP3, but machine-learning pipelines prefer uncompressed WAV. Preprocessing typically resamples to 16 kHz, converts stereo to mono, and normalises amplitude — preserving the frequency range that matters for bird calls while cutting file size and downstream computation.

Preprocessing Pipelines

Raw recordings arrive with silence, overlapping calls, and noise. A useful conversion script handles five jobs:

- Load MP3 or WAV and resample to 16 kHz mono.
- Normalise by RMS to avoid clipping.
- Segment into 3-second windows.
- Skip clips that are mostly silence (RMS < -40 dBFS).
- Write each output as a WAV file named `<species>_<source>_<id>.wav`.

Balancing and Augmentation

Class imbalance is common: rare species are outnumbered by common ones. The standard responses are undersampling the majority, oversampling the minority, and synthetic augmentation. Data augmentation — temporal shifts, additive noise, pitch modifications, and SpecAugment masking — improves generalisation by simulating varied acoustic conditions. Applied more heavily on minority species it can lift macro F1 by several points.

Training/Validation/Test Splits

Split the data into training (roughly 70%), validation (15%), and test (15%). The one hard rule is to avoid data leakage: recordings from the same location, session, or day must not appear in more than one split. MyGardenBird enforces this at the source-recording level using mixed-integer programming (Section 7.2); apply the same principle when curating new species.

9.2 Downloading from Xeno-Canto

Xeno-Canto (`https://xeno-canto.org/`) is the fastest way to pull large numbers of recordings. Each species has its own page — for example, Asian Koel lives at `https://xeno-canto.org/species/Eudynamys-scolopaceus` — listing every

submission chronologically with a unique `XC` identifier. You can download WAV files one at a time from the site, or use the API to fetch many MP3s in one pass:

Listing 9.1: Bulk download from Xeno-Canto by ID

```
import requests, re

xc_ids = ["1041026", "991328", "993640", "XC995636"]
base_url = "https://xeno-canto.org/{}/download"

for raw_id in xc_ids:
    match = re.search(r"\d+", raw_id)
    if not match:
        print(f"Invalid ID: {raw_id}"); continue
    xc_id = match.group(0)
    filename = f"xc{xc_id}.mp3"
    try:
        r = requests.get(base_url.format(xc_id))
        r.raise_for_status()
        with open(filename, "wb") as f:
            f.write(r.content)
        print(f"Saved: {filename}")
    except requests.RequestException as e:
        print(f"Failed xc{xc_id}: {e}")
```

For automated species queries rather than manual ID lists, the Xeno-Canto API supports search parameters. When you want to restrict to a region, pass a country or bounding box; when regional data is thin, widen the scope:

Listing 9.2: Query Xeno-Canto by species and region

```
def query_xc_species(species_name, country="Malaysia", max_clips=600):
    """Fetch recording IDs for a species, optionally by region."""
    url = "https://www.xeno-canto.org/api/2/recordings"
    params = {
        "query": f'species:"{species_name}" cnt:"{country}"',
        "page": 1,
    }
    ids = []
    while len(ids) < max_clips:
        response = requests.get(url, params=params).json()
        recordings = response.get("recordings", [])
        if not recordings:
            break
        ids.extend(rec["id"] for rec in recordings)
        params["page"] += 1
    return ids[:max_clips]

# Regional query: try this first
ids_my = query_xc_species("Orthotomus sericeus", "Malaysia")

# If regional count is too low, widen to a continent
if len(ids_my) < 300:
    ids_asia = query_xc_species("Orthotomus sericeus", country="")
    print(f"Regional: {len(ids_my)}, global: {len(ids_asia)}")
```

9.3 Downloading from Macaulay Library

Macaulay Library (https://macaulaylibrary.org/) is Cornell Lab of Ornithology's archive of over 71 million photos, 2.6 million audio files, and 300,000 videos. Its search portal at https://search.macaulaylibrary.org/catalog supports species queries across birds, mammals, reptiles, and more. Unlike Xeno-Canto, bulk download requires fetching assets by ID:

Listing 9.3: Download Macaulay Library assets by ID

```
import requests

asset_ids = ["1000904", "224431001", "641747208"]
base_url = "https://cdn.download.ams.birds.cornell.edu/api/v1/asset/{}/audio"

for asset_id in asset_ids:
    filename = f"ml{asset_id}.mp3"
    try:
        r = requests.get(base_url.format(asset_id))
        r.raise_for_status()
        with open(filename, "wb") as f:
            f.write(r.content)
        print(f"Saved: {filename}")
    except requests.RequestException as e:
        print(f"Failed {asset_id}: {e}")
```

Macaulay's coverage is often excellent for rare species, and many of its recordings carry precise geolocation metadata that helps filter by region when needed.

9.4 Preparing the ML-Ready Dataset

Downloaded files need shaping into fixed-length labelled clips before they can enter a training loop. Organise by species folder, convert to 16 kHz mono WAV, then slice into 3-second clips.

MP3 to 16 kHz Mono WAV

Listing 9.4: Convert MP3 to 16 kHz mono WAV

```
import librosa, soundfile as sf
from pathlib import Path

def convert_to_16khz_mono(input_path, output_path):
    try:
        y, sr = librosa.load(input_path, sr=16000, mono=True)
        sf.write(output_path, y, 16000, subtype='PCM_16')
        return True
    except Exception as e:
        print(f"Failed {input_path}: {e}"); return False

input_dir  = Path("./xc_downloads")
output_dir = Path("./xc_16khz_wav")
```

```
output_dir.mkdir(exist_ok=True)
for mp3_file in input_dir.glob("*.mp3"):
    wav_file = output_dir / f"{mp3_file.stem}.wav"
    if convert_to_16khz_mono(str(mp3_file), str(wav_file)):
        print(f"OK: {mp3_file.stem}")
```

Slicing Long Recordings into 3-Second Clips

Raw files are often 30 seconds to several minutes long with gaps between vocalisations. Audacity is the fastest way to visualise a file, spot the calls, and cut them to exactly 3 seconds. Figure 9.1 shows a typical view.

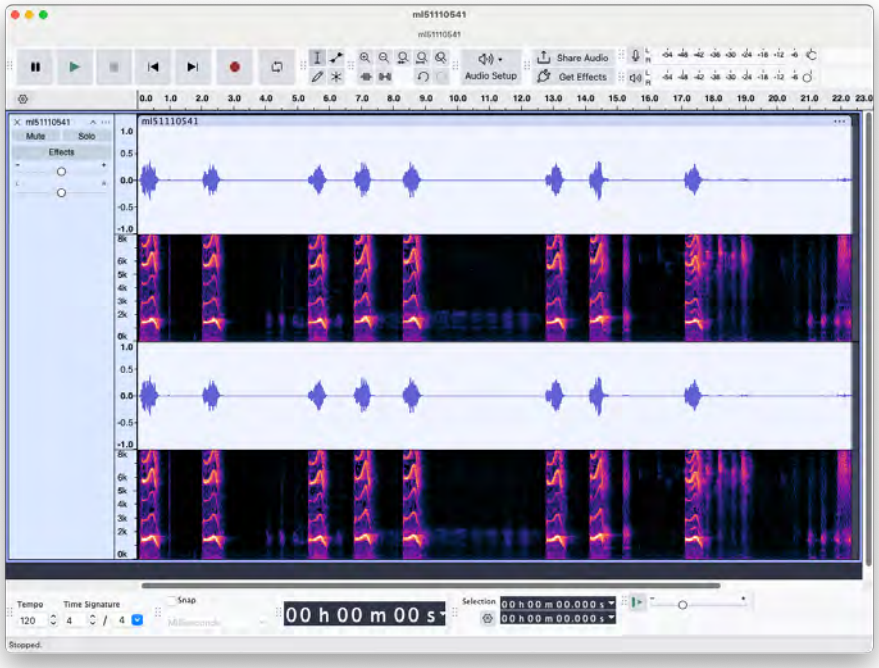

Figure 9.1: Audacity showing an Asian koel recording (Macaulay Library ML51110541.mp3) as a waveform and spectrogram. The paired view lets you visually spot bird calls before running an automatic label detector.

The three-step manual workflow:

Step 1: Detect Sounds. Import the audio into Audacity, then *Analyze* → *Sound Finder*. Audacity places a label on every non-silent region.

Step 2: Standardise to 3 seconds. Export labels via *File* → *Export* → *Export*

Labels. Each line has the form `start end name`. Rewrite the boundaries to centre a 3-second window on each label. Re-import and drag to adjust if needed.

Step 3: Split into clips. *File* → *Export* → *Export Multiple*, split by Labels. Audacity cuts one WAV per label at exactly 3 seconds.

Tip: assign keyboard shortcuts to Import Labels and Save Labels — you will use them constantly.

9.5 Accelerated Labeling with the Stage4 Annotator

Audacity's Sound Finder is a naive amplitude-threshold detector. On tropical recordings dominated by cicadas, wind, and rain, it either misses quiet calls or floods you with insect-chorus false positives. Manually reviewing each label takes 5–10 minutes per file; for a dataset of a few hundred files, that is the entire bottleneck.

The **Stage4 annotator** from the MyGardenBird pipeline [1] replaces this with an interactive Python tool that (a) uses a smarter blob-based detector borrowed from a winning BirdCLEF entry [2], (b) auto-generates fixed 3-second segments centred on detected vocal energy, (c) writes Audacity-compatible `.txt` label files, and (d) exposes a matplotlib+tkinter GUI for one-click accept/reject/nudge. Full source: https://github.com/mun3im/mygardenbird/blob/main/pipeline/Stage4_annotate_segments.py.

Sprengel Blob Detection

The algorithm treats the spectrogram as an image and finds blobs of vocal energy that stand out from the background [2]:

Listing 9.5: Sprengel blob detection from Stage4

```
import numpy as np, librosa
from scipy.ndimage import binary_dilation, binary_erosion

SPRENGEL_N_FFT, SPRENGEL_HOP = 512, 128
THRESHOLD_MULT = 3.0

def detect_sound_blobs(y, sr):
    S = np.abs(librosa.stft(y, n_fft=SPRENGEL_N_FFT,
                            hop_length=SPRENGEL_HOP))
    S = S / S.max()
    row_med = np.median(S, axis=1, keepdims=True)
    col_med = np.median(S, axis=0, keepdims=True)
    mask = (S > THRESHOLD_MULT * row_med) & \
           (S > THRESHOLD_MULT * col_med)
    struct = np.ones((4, 4), dtype=bool)
    mask = binary_erosion(mask, structure=struct)
    mask = binary_dilation(mask, structure=struct)
    col_active = mask.any(axis=0)
    for _ in range(2):
```

```
        col_active = binary_dilation(
            col_active, structure=np.ones(4, dtype=bool))
    events, run_start, in_run = [], 0, False
    time_axis = np.arange(len(col_active)) * SPRENGEL_HOP / sr
    for t, active in enumerate(col_active):
        if active and not in_run:
            run_start, in_run = t, True
        elif not active and in_run:
            events.append((time_axis[run_start], time_axis[t-1]))
            in_run = False
    if in_run:
        events.append((time_axis[run_start], time_axis[-1]))
    return events
```

The row-median-and-column-median test is the key trick: insect chorus raises the noise floor at every time frame (absorbed into the column median), and a mains hum sits at one frequency (absorbed into the row median). Only sounds locally bright in *both* time and frequency survive — exactly the profile of a bird call.

From Events to Fixed 3-Second Segments

Listing 9.6: Fixed-length segmentation

```
SEGMENT_DURATION = 3.0
MAX_SEGMENTS     = 10

def create_fixed_segments(events, audio_duration):
    half = SEGMENT_DURATION / 2.0
    segs = []
    for start, end in events:
        centre = (start + end) / 2.0
        bs = max(0, centre - half)
        be = min(bs + SEGMENT_DURATION, audio_duration)
        bs = max(0, be - SEGMENT_DURATION)
        segs.append((bs, be))
    segs.sort()
    final, last_end = [], -SEGMENT_DURATION
    for s, e in segs:
        if s >= last_end:
            final.append((s, e)); last_end = e
        if len(final) >= MAX_SEGMENTS:
            break
    return final
```

Segments are written as Audacity-compatible `.txt` label files so you can reopen the audio and fine-tune boundaries before exporting the final clips.

Interactive Review

For high-value files, Stage4 opens a matplotlib+tkinter viewer (Fig. 9.2) showing the display spectrogram with detected segments as yellow boxes.

Keyboard shortcuts: `a` accepts all segments and moves to the next file; `r` rejects

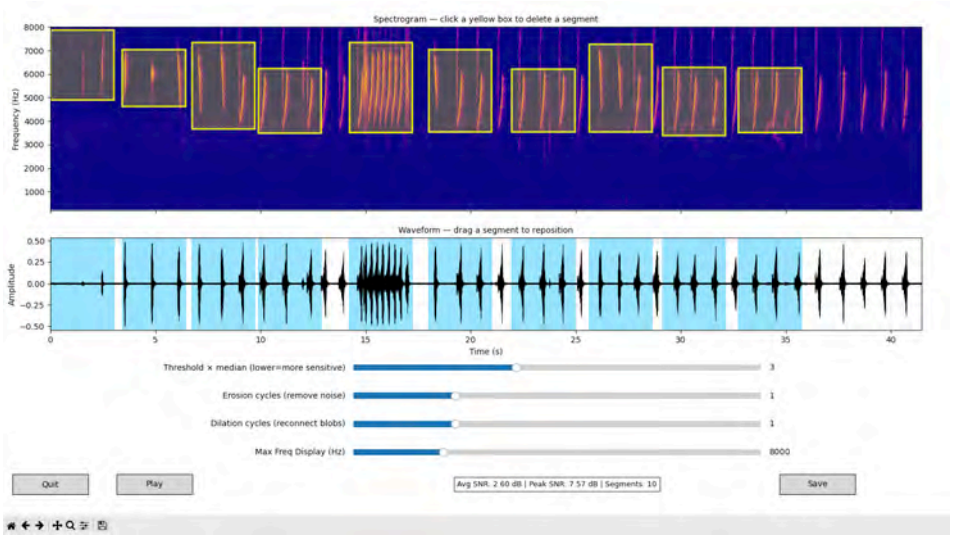

Figure 9.2: The Stage4 interactive annotator on a Malaysian bird recording. Top: display spectrogram with 10 detected 3-second segments as yellow boxes — click a box to delete it. Bottom: waveform with the same segments shaded blue — drag horizontally to reposition. The three sliders adjust the Sprengel detector on the fly. SNR and segment count are shown at the bottom.

one segment (click first); `space` plays the focused segment. A single reviewer can annotate 100+ files per hour — roughly $10\times$ faster than the manual Audacity workflow. The 7,200 clips in MyGardenBird were labelled this way [1].

9.6 Tier 1: MyGardenBirdPlus (14 Species, Broader-Asian Sourcing)

Before curating anything from scratch, use the shortcut: **MyGardenBirdPlus** is a downloadable addendum shipped inside the same Zenodo record as the core dataset [3, 1]. It adds two species that are ubiquitous in Malaysian gardens but could not be included in the core release under its strict regional sourcing rule: the **Common Myna** (*Acridotheres tristis*) and the **Zebra Dove** (*Geopelia striata*).

Why These Two Species Were Excluded From the Core

MyGardenBird restricts all source recordings to ASEAN/Indo-Malayan longitudes (60–140°E) so that the acoustic environment in every training clip matches what a user will encounter in the field. Under this constraint the paper reports only **75 usable clips** for Common Myna and **577 clips** for Zebra Dove — both short of the

600-clip minimum. The shortfall is a data-availability problem, not an acoustic one. Common Myna has been introduced worldwide; most Xeno-Canto submissions come from India, the Middle East, or Australia rather than Southeast Asia. Zebra Dove is similarly over-represented by introduced populations in Hawaii, the Mascarenes, and Central America.

The Plus addendum relaxes the filter to draw from the broader Asian pool — still near enough that the call character is representative of what is heard in Malaysia, but no longer restricted to the Indo-Malayan region. Each species reaches exactly 600 clips at the cost of mixed provenance.

What Is in the Addendum

Each addendum species ships with 600 clips at both 16 kHz and 44.1 kHz, using the same MIP source-level 80/10/10 split as the core release. Inside the extracted Zenodo archive the addendum lives at:

Listing 9.7: Addendum layout inside the Zenodo archive

```
mygardenbirdplus/
 16khz/
    Common Myna/         # 600 clips
    Zebra Dove/          # 600 clips
 44khz/                   # same, native sample rate
 metadata/
     16khz/
        clips.csv
        recordings.csv
        splits_mip_80_10_10.csv
     44khz/
```

Use the addendum when strict regional provenance is not a requirement. Skip it if you are deploying in a context where introduced-range vocalisations (different acoustic background, different subspecific calls) would confuse users.

Merging the Addendum With the Core Dataset

If you ran the Ch7 download, the addendum is already extracted inside the same bundle. Stitch its metadata into the pipeline:

Listing 9.8: Extend the pipeline to include MyGardenBirdPlus

```
import pandas as pd
from pathlib import Path

DATA_ROOT   = Path("/content/mygardenbird")
CORE_16K    = DATA_ROOT / "mygardenbird16khz"
PLUS_16K    = DATA_ROOT / "mygardenbirdplus" / "16khz"
CORE_META   = DATA_ROOT / "metadata16khz"
PLUS_META   = DATA_ROOT / "mygardenbirdplus" / "metadata" / "16khz"

species_dirs = sorted(
```

```
    [p for p in CORE_16K.iterdir() if p.is_dir()] +
    [p for p in PLUS_16K.iterdir() if p.is_dir()])
SPECIES      = [p.name for p in species_dirs]
NUM_CLASSES  = len(SPECIES)   # 14
sp_to_idx    = {s: i for i, s in enumerate(SPECIES)}

core_splits = pd.read_csv(CORE_META / "splits_mip_80_10_10.csv")
plus_splits = pd.read_csv(PLUS_META / "splits_mip_80_10_10.csv")
core_recs   = pd.read_csv(DATA_ROOT / "recordings.csv")
plus_recs   = pd.read_csv(DATA_ROOT / "mygardenbirdplus" / "recordings.csv")
recs_14     = pd.concat([core_recs, plus_recs], ignore_index=True)
core_clips  = pd.read_csv(CORE_META / "clips.csv")
plus_clips  = pd.read_csv(PLUS_META / "clips.csv")
clips_14    = pd.concat([core_clips, plus_clips], ignore_index=True)\
                .merge(recs_14[["source_id", "species_common"]],
                       on="source_id", how="left")
all_df = pd.concat([core_splits, plus_splits], ignore_index=True)\
           .merge(clips_14[["file_id", "species_common"]],
                  on="file_id", how="left")

CORE_SPECIES = {p.name for p in CORE_16K.iterdir() if p.is_dir()}
def clip_path(row):
    root = CORE_16K if row.species_common in CORE_SPECIES else PLUS_16K
    return str(root / row.species_common / f"{row.file_id}.wav")
```

Retraining for 14 Species

Three coordinated changes are required: new species folders join the tree, the labels file gains two entries at indices 12 and 13, and the output layer grows from 12 to 14 units. Retrain from scratch — the 12-unit softmax weights do not carry over.

Listing 9.9: Retrain the compact CNN for 14 species

```
# make_dataset() now uses all_df covering both core and Plus splits
train_14 = make_dataset("train")
val_14   = make_dataset("val")
test_14  = make_dataset("test")

model_14 = build_compact_cnn(NUM_CLASSES)   # 14
model_14.compile(optimizer=keras.optimizers.Adam(1e-3),
                 loss="sparse_categorical_crossentropy",
                 metrics=["accuracy"])
history_14 = model_14.fit(train_14, validation_data=val_14,
                          epochs=50, callbacks=callbacks)
model_14.save("zamzam_14sp.keras")
with open("labels_14sp.txt", "w") as f:
    f.write("\n".join(SPECIES))
```

Adding two well-chosen species with 600 balanced clips each typically costs 1–3 points of top-1 accuracy, recovered within a few extra epochs. If the drop is larger, check clip provenance and look for confusion between the new species and their closest acoustic neighbours in the existing twelve.

9.7 Tier 2: Moving to 20 Species — When Global Sourcing Becomes Necessary

Moving from 14 to 20 species is where the regional-sourcing assumption starts to crack. Some of the most visible Malaysian garden birds simply do not have enough Xeno-Canto submissions from the Indo-Malayan region to build a balanced 600-clip dataset. Two clear examples:

- **Eurasian Tree Sparrow** (*Passer montanus*) — arguably the most abundant bird in Malaysian cities and towns, but the vast majority of Xeno-Canto recordings come from Europe and Central Asia. Regional submissions are sparse.
- **House Crow** (*Corvus splendens*) — omnipresent in coastal Malaysian towns, but again, repository coverage skews heavily toward South Asia.

For these species, restricting to regional recordings means accepting a severely imbalanced contribution (perhaps 80–150 clips) or skipping them entirely. Neither is acceptable for an app that aims to identify the birds users actually see outside their windows. The pragmatic decision at the 20-species tier is to **source globally**, accepting that some training clips will carry vocalisation patterns from populations outside Malaysia.

What Global Sourcing Means in Practice

Global sourcing does not mean ignoring provenance entirely. The same quality-control pipeline still applies: check recording SNR, flag clipping, manually verify species ID, and prefer recordings that list a country in the Indo-Pacific region when they exist. The difference is that you no longer *discard* clips solely because they come from India, China, or Europe. For a widespread species whose calls are consistent across its range — House Crow's cawing sounds the same in Penang as in Kolkata — this is acoustically defensible. For species with strong regional dialects, it demands extra caution.

The Xeno-Canto query changes accordingly: drop the country filter and query globally, then optionally prioritise downloads from nearby countries (Thailand, Indonesia, Philippines, India) before pulling from further afield.

Listing 9.10: Global query strategy for data-thin species

```
def query_xc_tiered(species_name, target=600):
    """Try regional first; fall back to Asia, then global."""
    tiers = [
        ("Malaysia OR Indonesia OR Thailand OR Philippines", "regional"),
        ("Asia", "Asia-wide"),
        ("",     "global"),
    ]
    ids = []
```

```
    for region_query, label in tiers:
        q = f'species:"{species_name}"'
        if region_query:
            q += f' cnt:"{region_query}"'
        params = {"query": q, "page": 1}
        while len(ids) < target:
            r = requests.get(
                "https://www.xeno-canto.org/api/2/recordings",
                params=params).json()
            recs = r.get("recordings", [])
            if not recs:
                break
            ids.extend(rec["id"] for rec in recs)
            params["page"] += 1
        print(f"{label}: {len(ids)} IDs so far")
        if len(ids) >= target:
            break
    return ids[:target]
```

Candidate Species for the 20-Species List

Starting from the 14-species Plus dataset, good candidates for the remaining six slots include:

- **Eurasian Tree Sparrow** (*Passer montanus*) — globally sourced; omnipresent in Malaysian cities.
- **House Crow** (*Corvus splendens*) — globally sourced; unmistakable nasal caw.
- **Asian Glossy Starling** (*Aplonis panayensis*) — good regional coverage; metallic chattering calls.
- **Rufous-tailed Tailorbird** (*Orthotomus sericeus*) — good regional coverage; distinctive song.
- **Oriental Magpie-Robin** (*Copsychus saularis*) — good regional coverage; varied, melodic song.
- **White-crested Laughingthrush** (*Garrulax leucolophus*) — good regional coverage; loud chorus call.

The first two require global sourcing; the remaining four can be sourced regionally. Apply the tiered query strategy above to each, then run the same quality-filter and manual-review pipeline used for the core 14.

Quality Filtering

Before manual listening, flag clearly bad clips automatically:

Listing 9.11: Quality filtering: silence, clipping, SNR

```
import librosa, numpy as np
```

```
from pathlib import Path

def quality_score(wav_path):
    """Return a quality score 0--1 (1=best)."""
    y, sr = librosa.load(wav_path, sr=16000)
    rms     = librosa.feature.rms(y=y)[0]
    rms_db  = librosa.power_to_db(rms)
    silence = np.mean(rms_db < -40)
    clipped = np.mean(np.abs(y) > 0.99)
    power   = np.abs(y) ** 2
    snr_db  = 10 * np.log10(np.percentile(power, 90) /
                            (np.percentile(power, 10) + 1e-10))
    score = (1 - silence) * (1 - clipped * 10) * min(1.0, snr_db / 20)
    return max(0, score)

for wav in Path("./xc_16khz_wav").glob("*.wav"):
    s = quality_score(str(wav))
    print(f"{'POOR' if s < 0.5 else 'OK  '}: {wav.name} ({s:.2f})")
```

For the clips that pass, use the Jupyter widget from the earlier download workflow for manual accept/reject. Budget 1–2 hours per new species for the full sourcing-to-clips cycle.

9.8 Tier 3: Scaling to 50+ Species — The Imbalance Problem

At fifty species, provenance is no longer the central challenge — you have already accepted global sourcing for many species. The new dominant problem is **class imbalance**. When you draw from open repositories for fifty Malaysian species, the natural distribution of available recordings is highly skewed: common, frequently-recorded birds (Yellow-vented Bulbul, Spotted Dove) may have thousands of eligible clips; scarcer or less-recorded species might yield thirty. This imbalance is structural, not accidental, and left untreated it breaks the classifier.

The Shape of the Problem

A realistic 50-species Xeno-Canto-sourced corpus for Malaysia looks roughly like this:

- **Top 10 species:** 400–800 clips each (well-recorded common birds).
- **Middle 20 species:** 150–399 clips each.
- **Bottom 20 species:** 30–149 clips each (scarce, cryptic, or simply under-recorded in repositories).

The resulting Gini coefficient of about 0.55–0.65 is typical of raw Xeno-Canto queries at this scale [4]. Unlike the 12-species and 14-species datasets, you

cannot simply cap at 600 and call it balanced — the bottom 20 species don't have 600 clips to cap at.

Failure Mode: Majority Dominance

Train the compact CNN on this raw unbalanced corpus without any correction and you will typically see:

- Overall top-1 accuracy looks acceptable (75–82%) because majority species dominate the clip count and the loss.
- The per-species F1 report shows species with fewer than 100 clips scoring < 0.40.
- The confusion matrix shows minority species being predicted as the acoustically nearest majority species in over 60% of cases.
- Macro-average F1 trails weighted-average F1 by 15–20 points.

Always report **macro F1** alongside weighted F1 and top-1 accuracy. It is the only metric that penalises minority-species failures equally.

Fix 1: Class Weights

Keras's `class_weight` argument scales the loss contribution of each class inversely to its frequency, forcing the model to pay equal attention to rare and common species during training:

Listing 9.12: Compute and apply class weights for 50-species training

```
from sklearn.utils import compute_class_weight
import numpy as np

all_labels = []
for _, y_batch in train_50:
    all_labels.extend(y_batch.numpy())
all_labels = np.array(all_labels)

weights = compute_class_weight(
    class_weight="balanced",
    classes=np.arange(50),
    y=all_labels)
class_weight_dict = dict(enumerate(weights))

model_50 = build_compact_cnn(50)
model_50.compile(optimizer=keras.optimizers.Adam(1e-3),
                 loss="sparse_categorical_crossentropy",
                 metrics=["accuracy"])
model_50.fit(train_50, validation_data=val_50,
             epochs=50, class_weight=class_weight_dict,
             callbacks=callbacks)
```

Class weighting alone typically recovers 5–10 macro F1 points for minority species, at a cost of 2–4 weighted F1 points on the majority — a worthwhile trade for a general-purpose classifier.

Fix 2: Per-Species Capping

Class weights correct the loss but not the data distribution. A minority species with 50 clips still sees limited acoustic variety and will overfit to the specific recordings it has. Capping the majority species narrows the raw count gap:

Listing 9.13: Per-species capping to limit majority over-representation

```
import random

CAP = 400   # max clips per species in training split

def build_capped_dataset(split, sp_to_idx, data_path, cap=CAP):
    paths, labels = [], []
    for sp, idx in sp_to_idx.items():
        sp_paths = list((data_path / split / sp).glob("*.wav"))
        if split == "train" and len(sp_paths) > cap:
            sp_paths = random.sample(sp_paths, cap)
        for p in sp_paths:
            paths.append(str(p)); labels.append(idx)
    ds = tf.data.Dataset.from_tensor_slices((paths, labels))
    ds = ds.map(lambda p, l: (wav_to_logmel(load_wav(p)), l),
                num_parallel_calls=AUTOTUNE)
    if split == "train":
        ds = ds.shuffle(buffer_size=len(paths))
    return ds.batch(BATCH).prefetch(AUTOTUNE)

train_50_capped = build_capped_dataset("train", sp_to_idx_50, data_path_50)
```

Combine capping with class weights for the best result. Augmentation amplifies the effect further: for minority species with only 50–100 clips, synthetic augmentation (pitch shift, time stretch, SpecAugment masking) increases acoustic variety in a way that neither capping nor loss reweighting can.

Performance Summary

Table 9.1 shows typical results across the correction strategies on a 50-species corpus. Numbers are illustrative; your exact results depend on clip quality and species selection.

The top-1 numbers are deceptively close across all strategies. Macro F1 is the honest metric.

9.9 End-to-End Workflow

Regardless of tier, the expansion workflow follows the same repeatable pattern:

Table 9.1: 50-species training strategies compared.

Strategy	Top-1 acc.	Macro F1	Minority F1
No balancing	78%	0.58	0.32
Class weights only	76%	0.67	0.51
Capping + class weights	75%	0.71	0.58
Capping + weights + augmentation	74%	0.76	0.65

1. **Plan.** Choose the target species list. For each species, note whether regional or global sourcing will be needed.
2. **Source.** Use the tiered Xeno-Canto query (regional → Asia-wide → global) and Macaulay Library for coverage gaps.
3. **Prepare.** Convert to 16 kHz WAV, quality-filter, segment into 3-second clips using Stage4.
4. **Split.** Assign clips to train/val/test at the source-recording level to prevent leakage.
5. **Train.** Apply class weights and capping for imbalanced corpora; augmentation for minority species.
6. **Evaluate.** Confusion matrix, per-species F1, and macro F1.
7. **Export.** Save `.keras`, `labels.txt`, and `metadata.json` for Chapter 8.
8. **Version.** Tag every artifact — e.g. `zamzam_14sp_v1.0`, `zamzam_20sp_v1.1`.

9.10 Choosing Your Target Species Count

Table 9.2 summarises the provenance, data, and accuracy trade-offs at each tier.

Table 9.2: Expansion tiers: provenance, data, and accuracy trade-offs.

Tier	Sourcing	Data challenge	Typical accuracy
12sp (core)	Indo-Malayan region	None; 600 balanced	92–96% top-1
14sp (Plus)	Broader Asia	Provenance mixed	90–94% top-1
20sp	Global for some species	Provenance fully relaxed	85–90% top-1
50sp	Global	Imbalance dominates	macro F1 $\approx$ 0.70–0.76

Choose based on your users' expectations (casual birders are served well by 20 species; field researchers want 50+), your deployment target (phones handle

a 5 MB model; microcontrollers need <1 MB), and the curation effort you can commit. Ship the 12-species baseline first, validate it in the field, and expand only when user feedback shows a clear gap.

9.11 Key Takeaways

- The core MyGardenBird 12-species dataset is sourced strictly from the Indo-Malayan region; every training clip matches the acoustic environment a Malaysian user will encounter.
- MyGardenBirdPlus adds Common Myna and Zebra Dove by drawing from the broader Asian recording pool — still representative, but with relaxed regional provenance.
- Moving to 20 species requires accepting global sourcing for species with thin regional coverage (Eurasian Tree Sparrow, House Crow). Use a tiered query strategy to prioritise nearby countries before pulling from further away.
- At 50 species, provenance gives way to imbalance as the dominant problem. Some species simply cannot reach 600 clips from any source; the classifier needs help.
- Class weights (scale the loss) and per-species capping (limit majority data) are the two primary balancing tools. Use both; add augmentation for the strongest minority-species lift.
- Macro F1 is the honest metric for imbalanced multi-class problems. Report it alongside top-1 accuracy.
- Always quality-filter by RMS, clipping, and SNR before manual review. This cuts listening time roughly in half.

9.12 Exercises

9.1 Download 600 clips of Eurasian Tree Sparrow from Xeno-Canto using the tiered query strategy. Report how many come from the Indo-Malayan region versus the rest of Asia versus globally.

9.2 Merge the MyGardenBirdPlus addendum with the core dataset and retrain the compact CNN. Report test accuracy, macro F1, and any new confusion pairs compared with the 12-species baseline.

9.3 Build a 50-species corpus with realistic imbalance (top 10: 600 clips each, bottom 20: 50 clips each). Train without and with class weights. Compare macro F1.

9.4 Implement per-species capping at `CAP = 200`. Compare per-species F1 for the three lowest-clip-count species against the uncapped result.

9.5 Plot training curves for the 50-species model with and without class weights. Which metric diverges most?

9.13 Reflection

- At what species count does the compact CNN begin to saturate? How would you detect it in the training curves?
- You sourced Eurasian Tree Sparrow clips from Germany. The German population's calls are acoustically similar to Malaysian ones, but the background noise is different. Is this a problem? How would you test for it?
- If a new species consistently gets confused with an existing one, is that a data problem, a model problem, or an acoustic-similarity problem? How would you distinguish between them?
- Class weights fix the loss but not the data distribution. Which augmentation strategies would you prioritise for the bottom-20 minority species, and why?

Bibliography

[1] Munim Zabidi et al. *MyGardenBird: A Machine-Learning-Ready Bird Sound Dataset for Twelve Common Malaysian Birds*. 7,200 manually reviewed 3-second WAV clips at 16 kHz, CC BY-NC-SA 4.0. Zenodo, 2025. DOI: 10.5281/zenodo.20306877. URL: https://zenodo.org/records/20306877.

[2] Elias Sprengel et al. "Audio Based Bird Species Identification using Deep Learning Techniques." In: *CLEF Working Notes*. LifeCLEF Bird Task 2016 winning entry. 2016, pp. 547–559.

[3] Muhammad Mun'im Ahmad Zabidi, Mohd Yamani Idna Idris, and Norisma Idris. "MyGardenBird: A Machine-Learning-Ready Bird Sound Dataset for Twelve Common Malaysian Birds." In: (2026). arXiv: 2606.06975 [cs.SD]. URL: https://arxiv.org/abs/2606.06975.

[4] Muhammad Mun'im Ahmad Zabidi. "Edge AI for Avian Acoustic Monitoring: A Systematic Review and Deployment Framework." In: *IEEE Access* 4 (2026). In preparation.

[5] Macaulay. *Macaulay Library Cornell Library of Ornithology*. http://macaulaylibrary.org. Accessed: 2024-11-25. 2024.

[6] Willem-Pier Vellinga and Robert Planqué. "The Xeno-Canto collection and its relation to sound recognition and classification." In: *CLEF (Working Notes)*. 2015.

Chapter 10

Getting Started with Flutter

Learning Objectives

- Explain why Flutter is a good fit for a cross-platform bird-ID app.
- Install the Flutter SDK, Android Studio or Xcode, and confirm the toolchain with `flutter doctor`.
- Create, run, and hot-reload a Flutter project on an emulator or a real device.
- Read a widget tree and know when to use `StatelessWidget` versus `StatefulWidget`.
- Navigate between screens and pass data using `go_router`.
- Set up a localisation pipeline so the English MVP can accept Malay in v2 with a translation, not a refactor.

Chapters 1–8 built the model; Chapter 5 outlined the app architecture (five modules in `lib/`). By now, you have the trained bundle from Chapter 8—`bird_model.tflite`, `labels.txt`, and `metadata.json`—ready for the Flutter project's `assets/` folder. From here, the book shifts to writing the app. Flutter provides the framework for recording UI, on-device inference, results display, and history logs—all in a single Dart codebase for Android and iOS. This chapter installs the toolchain and introduces the widget model. Subsequent chapters wire the microphone (Ch 10) and TFLite model (Ch 11) into the widget tree.

10.1 Why Flutter

Four frameworks dominate cross-platform mobile development. Figure 10.1 shows Flutter's engine-plus-widget architecture: the app is written in Dart, the widget layer builds a scene graph, and Flutter's Skia rendering engine paints pixels directly on the platform's canvas rather than driving each OS's native widget set. This is why the same app looks pixel-identical on iOS and Android. Table 10.1 then lines up all four frameworks on the axes that matter for ZamZam.

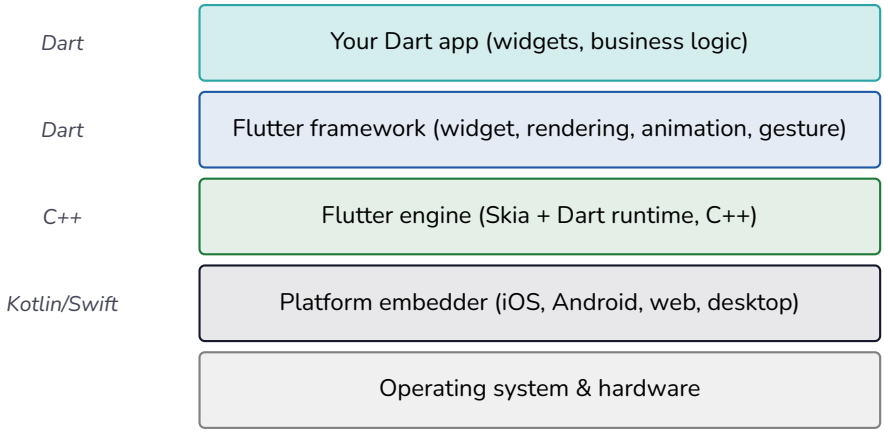

Figure 10.1: Flutter's layered architecture. Your Dart app sits on top; the Flutter framework and Skia-based engine render every pixel; the thin platform embedder handles OS integration (touch events, lifecycle, file I/O). Because Skia paints the canvas directly, the UI looks pixel-identical across iOS, Android, web, and desktop.

Table 10.1: Cross-platform mobile framework alternatives for ZamZam

Engineering Concern	Flutter	React Native	Native	Kivy
Codebases to maintain	1 (Dart)	1 (JS/TS)	2 (Swift/Kotlin)	1 (Python)
UI rendering	Custom (Impeller/Skia)	Native Host Widgets	Native OS Widgets	Custom (OpenGL ES)
Stateful Hot reload	Yes	Yes	Slow	Yes
TFLite plugin quality	Excellent	Fair	Native	Weak
Audio API capabilities	Robust/Low-latency	Mature	Complete Native Control	Basic / High-latency
App-store distribution	Standard	Standard	Standard	Awkward
Learning curve	Moderate	Low (If JS familiar)	Steep (Two separate stacks)	Low to medium
Compiled binary size	15–20 MB	20–30 MB	5–15 MB	30–50 MB

ZamZam uses **Flutter** for two reasons: (1) the mature `tflite_flutter` plugin (Chapter 11) and (2) one codebase with pixel-identical rendering on iOS and Android. Native development requires two teams; React Native's TFLite support is weaker; Kivy struggles on app stores. React Native is viable if you have existing TypeScript code; otherwise Flutter is recommended.

10.2 Installing Flutter

Installation takes 15–30 minutes and requires 20 GB of disk space. The steps below are for Flutter 3.x; check https://docs.flutter.dev/get-started/install for newer versions.

macOS

```
# 1. Install Flutter SDK
brew install --cask flutter

# 2. Install Xcode from the Mac App Store (large download).
#    Then install the command-line tools:
sudo xcode-select --install
sudo xcodebuild -runFirstLaunch

# 3. Install Android Studio (also via brew or from developer.android.com)
brew install --cask android-studio

# 4. Verify the toolchain
flutter doctor
```

Windows

```
# Download the Flutter SDK zip from flutter.dev, extract it,
# and add the flutter/bin folder to your PATH.

# Install Android Studio: https://developer.android.com/studio
# Open it once so it downloads the required SDK components.

# Verify the toolchain
flutter doctor
```

Note: iOS builds require macOS, so on Windows you can build and test the Android side only.

Linux (Ubuntu/Debian)

```
sudo snap install flutter --classic
sudo snap install android-studio --classic
flutter doctor
```

Fixing `flutter doctor` Warnings

`flutter doctor` prints a checklist with green and red marks. Address every red mark before continuing. Three common issues:

- **Android licenses not accepted.** Run `flutter doctor --android-licenses` and accept each with `y`.
- **CocoaPods not installed** (macOS only). Run `sudo gem install cocoapods` or `brew install cocoapods`.
- **No devices found.** Start an emulator from Android Studio (Device Manager — Pixel 6/7 with the latest stable API level is a safe default) or plug in a phone with USB debugging enabled. On macOS you can instead run `open -a Simulator` for an iOS device.

IDE Setup

Flutter works from any editor, but the two mainstream options are Android Studio (which you already installed for the Android SDK) and VS Code. Either way, install the **Flutter** and **Dart** plugins:

- **Android Studio**: `Settings` → `Plugins`, search for "Flutter" and install; the Dart plugin is pulled in automatically. Restart the IDE.
- **VS Code**: install the **Flutter** extension from the Extensions panel; it bundles the Dart extension.

The plugins give you hot-reload buttons, widget inspector, debugger, and code completion. Without them Flutter still runs, but the developer experience drops sharply.

Getting the Project Code

Two paths, depending on whether you are joining an existing project or starting fresh:

- **Joining the ZamZam project.** `git clone <repo-url>`, then `cd zamzam && flutter pub get` to fetch dependencies listed in `pubspec.yaml`. Open the folder in Android Studio or VS Code and let it index (30–60 s the first time).
- **Starting fresh.** Skip to §9.3 below, which walks through `flutter create` and the default counter app. This is the path we take for the rest of this chapter.

Chapter 5 laid out the app's architecture: five modules (`audio`, `dsp`, `model`, `data`, `ui`) inside `lib/`. Whether you clone or create, that structure is what you will fill in over the following chapters.

Once `flutter doctor` is all-green, the toolchain is ready.

10.3 Your First Flutter App

Create a project called `zamzam` and run the default counter app:

```
flutter create zamzam
cd zamzam
flutter run
```

If a device or emulator is connected, the app launches in debug mode. You should see a Material page with an ”increment” floating button. Each tap increments a counter, and saving `lib/main.dart` reloads the app within a second without losing state. This is **hot reload**, Flutter’s most beloved feature.

The important file is `lib/main.dart`. Open it and you will find, in about 100 lines, a complete Material app: an `App` widget, a `StatefulWidget` home page, a `Scaffold` with an app bar, and a `FloatingActionButton`. The generated code is the fastest way to learn the framework’s idioms.

Listing 10.1: The essential structure of `lib/main.dart`

```dart
import 'package:flutter/material.dart';

void main() => runApp(const ZamZamApp());

class ZamZamApp extends StatelessWidget {
  const ZamZamApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ZamZam',
      theme: ThemeData(colorSchemeSeed: Colors.teal, useMaterial3: true),
      home: const HomePage(),
    );
  }
}

class HomePage extends StatefulWidget {
  const HomePage({super.key});
  @override
  State<HomePage> createState() => _HomePageState();
}

class _HomePageState extends State<HomePage> {
  int _detections = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('ZamZam')),
      body: Center(child: Text('Detections: $_detections')),
      floatingActionButton: FloatingActionButton(
        onPressed: () => setState(() => _detections++),
        child: const Icon(Icons.mic),
      ),
    );
  }
}
```

Everything is a widget: text, buttons, layouts, and animations. Widgets nest into a tree; Flutter's rendering engine walks the tree and draws it. Change any widget, save, and hot reload repaints in milliseconds.

10.4 Widgets, State, Layout

Three ideas cover most of Flutter's day-to-day surface. But first, a picture: Figure 10.2 shows the widget tree for the counter app in Listing 10.1. Every visible element is a widget; layout widgets like `Scaffold`, `Center`, and `Column` do not paint themselves but arrange their children.

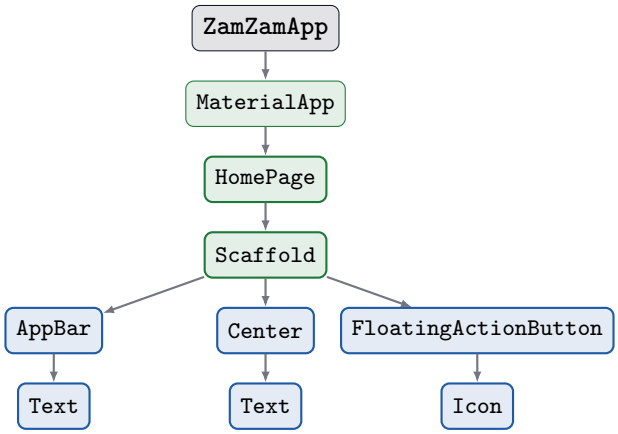

Figure 10.2: The widget tree of Listing 10.1. Root at the top (`ZamZamApp`) is a `StatelessWidget`; `HomePage` is stateful and owns the counter. Layout widgets (green) nest paint widgets (blue). Flutter walks this tree top-down on every rebuild.

Stateless vs. Stateful

`StatelessWidget` is a pure function: same inputs produce the same output, with no memory between builds. Use it when appearance depends only on input data.

`StatefulWidget` pairs a widget with a `State` object that persists across rebuilds. Use it for mutable data—a counter, form value, or recording state. Calling `setState` tells the framework state changed and triggers a rebuild.

Layout Primitives

Six widgets cover 90% of layouts. Everything else composes from these:

- `Row` and `Column` — horizontal and vertical stacks.
- `Container` — a box with padding, margin, colour, and border. The kitchen-sink layout widget.
- `Padding` — add space around a child (thinner than `Container`).
- `Center` and `Align` — position a child within the parent.
- `Expanded` and `Flexible` — share available space between siblings in a Row/Column.
- `SizedBox` — fixed-size gap, or a widget of an explicit size.

A typical ZamZam "result" card composes as follows:

Listing 10.2: A Material card showing a three-name species result

```
Card(
  child: Padding(
    padding: const EdgeInsets.all(16),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(malay, style: Theme.of(context).textTheme.titleLarge),
        Text(english, style: Theme.of(context).textTheme.titleMedium),
        Text(scientific, style: const TextStyle(fontStyle: FontStyle.italic)),
        const SizedBox(height: 8),
        LinearProgressIndicator(value: confidence),
      ],
    ),
  ),
)
```

Managing State Beyond a Single Widget

`setState` works for single-screen state. When state must persist across navigation or be shared between screens—like the detection history—use an app-wide solution. Two popular options:

- **Provider.** Simple, Google-endorsed, a good default.
- **Riverpod.** Provider's spiritual successor with better testability and compile-time safety. Slight learning curve.

ZamZam's data model is small (recording state, current inference result, past detections). Either is fine; we default to Provider throughout the code samples that follow.

10.5 Navigation and Routing

ZamZam has four screens: Record (home), Result, History, and Settings. Flutter's `Navigator.push` works for simple prototypes, but multi-screen apps benefit from `go_router`, which provides named routes, web back-button support, and deep links (useful for opening shared detections directly).

Listing 10.3: Route table with `go_router`

```
import 'package:go_router/go_router.dart';

final _router = GoRouter(
  routes: [
    GoRoute(
      path: '/',
      builder: (_, __) => const RecordScreen(),
    ),
    GoRoute(
      path: '/result/:detectionId',
      builder: (context, state) => ResultScreen(
        detectionId: state.pathParameters['detectionId']!,
      ),
    ),
    GoRoute(
      path: '/history',
      builder: (_, __) => const HistoryScreen(),
    ),
    GoRoute(
      path: '/settings',
      builder: (_, __) => const SettingsScreen(),
    ),
  ],
);

// Wire it into MaterialApp.router:
MaterialApp.router(routerConfig: _router, ...);
```

Navigate with `context.go('/history')` or `context.push('/result/42')`. Data flows via path parameters (as above) or extra objects passed with `state.extra`.

10.6 Extending Beyond Widgets: Platform Channels and Localization

Platform Channels

Most native functionality is wrapped by plugins—microphone recording, TFLite inference, file storage, sensors, and GPS. When no plugin provides what you need (e.g., low-latency audio buffers), Flutter's **platform channel** lets Dart call Kotlin or Swift code.

A platform channel is a named message pipe. Dart calls `channel.invokeMethod('startRecordin` the native side listens and responds. Message payloads serialize automatically

(JSON-like) both ways.

Listing 10.4: Calling native code from Dart via a platform channel

```dart
import 'package:flutter/services.dart';

const _audio = MethodChannel('com.zamzam/audio');

Future<void> startNativeRecording() async {
  final int sampleRate = await _audio.invokeMethod('startRecording', {
    'sampleRate': 16000,
    'channels': 1,
  });
  print('Native recorder returned sr = $sampleRate');
}
```

On the Android side, `MainActivity.kt` registers a `MethodChannel` handler in `configureFlutterEngine` and dispatches on the method name. iOS uses the same idea with `FlutterMethodChannel` in `AppDelegate.swift`. The details are in the Flutter docs; the important thing to remember now is that platform channels exist and you never have to give up on a feature just because a plugin does not.

We will use platform channels sparingly. For ZamZam, existing plugins cover microphone (`record` or `mic_stream`) and inference (`tflite_flutter`), so most of the code stays in Dart.

Localization-Ready From Day One

The MVP ships in English only, but design the codebase for translation. This takes 2 extra hours in Week 5 and saves days if you add Malay, Mandarin, or Tamil later.

Set it up now:

1. Add `flutter_localizations` and `intl` to `pubspec.yaml`.
2. Create `lib/l10n/app_en.arb` with every user-facing string as a key.
3. Run `flutter gen-l10n` to generate the delegate.
4. Wire `AppLocalizations.of(context)!.recordButton` throughout your widgets instead of hard-coded strings.

The rule is simple: **no string literals in widgets**. If you write `Text('Record')`, stop and add a key to the ARB file first. Adding Malay in v2 becomes translating one file and adding a switcher—not a full refactor.

Note that this discipline applies only to *UI chrome*. Species names are data, not strings, and live in the `assets/species.json` catalogue described in the publishing chapter.

10.7 Key Takeaways

- Flutter delivers one Dart codebase to both Android and iOS with near-native performance, hot reload, and a rich widget catalogue.
- The mental model is small: widgets nest into a tree, `setState` triggers rebuilds, Provider or Riverpod carries state across screens, `go_router` handles navigation.
- Flutter's plugin ecosystem covers the two capabilities ZamZam most needs — audio capture (`record`, `permission_handler`) and TFLite inference (`tflite_flutter`).
- Platform channels are the escape hatch when a plugin does not exist. Reach for them last, not first.
- Discipline UI strings into ARB files from day one (`flutter_localizations`) even if you ship English-only in the MVP — adding a second language later becomes a translation task, not a codebase-wide refactor.
- Chapter 10 opens the microphone; Chapter 11 wires the `.tflite` model into the widget tree.

10.8 Exercises

10.1 Create a new Flutter project and run it on both an Android emulator and an iOS simulator (or a real device).

10.2 Modify the default counter app so the button says "Record" and increments the counter only while pressed. Use `GestureDetector` or `Listener`.

10.3 Add a second screen accessed via `go_router`. Pass a string argument via the path and display it on the second screen.

10.4 Restructure the counter into a `StatelessWidget` that receives the count as a parameter and a `StatefulWidget` parent that owns the state. Which is easier to test?

10.5 Move all displayed strings in your project into an ARB file and read them via `AppLocalizations`. Confirm the app still runs.

10.9 Reflection

- Which felt easier: Flutter's widget-tree model or a UI framework you have used before?

- If you were shipping ZamZam to only one platform, would you still choose Flutter over the native SDK?
- Where would a platform channel be worth writing native code in your app? Where would you refuse and use a plugin instead?
- What is the accessibility cost of localising only UI chrome and leaving species names in a data file — and is that acceptable?

Chapter 11

Audio Capture and Feature Extraction on Device

Learning Objectives

- Request microphone permission on iOS and Android and handle the denied case gracefully.
- Capture live audio from the phone microphone at 16 kHz, 16-bit, mono.
- Compute a 128-bin log-Mel spectrogram in Dart that matches the training pipeline exactly.
- Prepare the input tensor for the TFLite model from Chapter 8.
- Stream inference continuously with a sliding-window buffer.
- Handle silence, clipping, and other input edge cases without crashing the app.

Chapter 9 exported the ZamZam model and its metadata. Chapter 10 set up Flutter. This chapter builds the audio pipeline between them: capture, framing, windowing, FFT, Mel filterbank, and log conversion to produce a $1 \times 128 \times 128 \times 1$ tensor for the TFLite interpreter in Chapter 12. Every step must match training-time preprocessing exactly, or accuracy will collapse.

11.1 Microphone Permissions

Both iOS and Android require users to grant microphone access before recording. Ask in context and handle denials gracefully.

iOS: Info.plist Declaration

iOS requires a human-readable reason to appear in the permission prompt. Add a `NSMicrophoneUsageDescription` key to `ios/Runner/Info.plist`:

Listing 11.1: `Info.plist` entry for microphone usage

```
<key>NSMicrophoneUsageDescription</key>
<string>ZamZam listens to a short recording to identify the
```

```
bird you are hearing. Audio never leaves your phone.</string>
```

App-store review will reject a missing or vague usage string, so be specific.

Android: Manifest Declaration and Runtime Prompt

Android splits the permission into a manifest declaration plus a runtime prompt (required on Android 6.0 and above). Add to `android/app/src/main/AndroidManifest.xml`:

Listing 11.2: `AndroidManifest.xml` microphone permission

```
<uses-permission android:name="android.permission.RECORD_AUDIO" />
```

The runtime prompt is easiest via the `permission_handler` package. Add it to `pubspec.yaml` (`permission_handler: ^11.0.0`), then wrap the record call:

Listing 11.3: Requesting microphone permission at runtime

```
import 'package:permission_handler/permission_handler.dart';

Future<bool> ensureMicPermission() async {
  final status = await Permission.microphone.status;
  if (status.isGranted) return true;
  if (status.isPermanentlyDenied) {
    // User said "Never ask again" -- send them to Settings.
    await openAppSettings();
    return false;
  }
  final result = await Permission.microphone.request();
  return result.isGranted;
}
```

Call `ensureMicPermission()` when the user taps Record, not at app launch. Unsolicited permission prompts have very low grant rates.

11.2 Recording Audio

Three Dart packages dominate the mic-capture space. Table 11.1 compares them.

For ZamZam's *tap-and-hold* MVP from Chapter 5, `record` suffices: tap → record 3 seconds to a file → read samples for processing. A streaming version would switch to `mic_stream`.

Listing 11.4: Recording 3s at 16 kHz mono with `record`

```
import 'dart:io';
import 'package:record/record.dart';
import 'package:path_provider/path_provider.dart';
```

Table 11.1: Flutter microphone-capture packages

Package	Best for	Output format	Notes
`record`	Fixed-length clips	WAV / M4A file on disk	Simple API, active maintenance
`flutter_sound`	Recording + playback	Encoded audio in files or streams	Larger, more features
`mic_stream`	Continuous PCM stream	`Stream<Uint8List>` of raw samples	Minimal, ideal for real-time DSP

```
Future<String> recordThreeSeconds() async {
  final recorder = AudioRecorder();
  if (!await recorder.hasPermission()) {
    throw StateError('Microphone permission denied');
  }
  final dir = await getTemporaryDirectory();
  final path = '${dir.path}/clip_${DateTime.now().millisecondsSinceEpoch}.wav';

  await recorder.start(
    const RecordConfig(
      encoder: AudioEncoder.wav,
      sampleRate: 16000,      // matches model_metadata.json from Ch 8
      numChannels: 1,         // mono
      bitRate: 16 * 16000,    // 16-bit PCM
    ),
    path: path,
  );
  await Future<void>.delayed(const Duration(seconds: 3));
  await recorder.stop();
  return path;
}
```

Recording to a WAV file (rather than memory) has two benefits: (1) WAV headers enable easy debugging (play the file back to hear the model input), and (2) the file is small (16 kHz $\times$ 3 s $\times$ 2 bytes $\approx$ 96 KB), making temp-directory clutter negligible.

11.3 From PCM to Mel Spectrogram

This is the highest-stakes step. If the Dart Mel spectrogram does not match the training-time librosa version, model accuracy will drop—often silently and catastrophically.

Three Implementation Options

Table 11.2 lays out the choices.

For ZamZam we use option 1 (Dart via `fftea`). It is easy to debug against librosa,

Table 11.2: Where to compute the Mel spectrogram

Option	Where it runs	Complexity	Best for
Dart via `fftea`	On-device, pure Dart	Low	MVP, easy debugging
Native via platform channel	Kotlin (CMSIS-DSP), Swift (Accelerate)	High	Low-latency streaming
In the TFLite model	Model input is raw waveform	Medium (build-time)	Zero-mismatch guarantee

plenty fast for 3-second clips, and does not require dropping into Kotlin/Swift. If in a later release the streaming latency budget becomes tight, options 2 or 3 remain open.

The Mel Pipeline in Dart

Match the training-time parameters exactly. From `model_metadata.json` (Chapter 8): `sample_rate_hz`=16000, `n_fft`=1024, `hop_length`=512, `n_mels`=128, `fmin_hz`=300, `fmax_hz`=8000, then `power_to_db(ref=max)`.

Listing 11.5: Log-Mel spectrogram in Dart matching librosa

```dart
import 'dart:math' as math;
import 'package:fftea/fftea.dart';

class MelExtractor {
  static const int sampleRate = 16000;
  static const int nFft       = 1024;
  static const int hopLength  = 512;
  static const int nMels      = 128;
  static const double fMin    = 300.0;
  static const double fMax    = 8000.0;

  final FFT _fft = FFT(nFft);
  final Float64List _window = _hannWindow(nFft);
  late final List<Float64List> _melBank =
      _buildMelFilterBank(nFft, nMels, sampleRate, fMin, fMax);

  /// Return a [nMels][nFrames] log-power Mel spectrogram in dB, ref = max.
  List<Float64List> logMel(Float32List pcm) {
    // Zero-pad so the last frame is complete (matches librosa center=False).
    final nFrames = 1 + (pcm.length - nFft) ~/ hopLength;
    final mel = List<Float64List>.generate(nMels, (_) => Float64List(nFrames));

    final frame = Float64List(nFft);
    for (int f = 0; f < nFrames; f++) {
      final start = f * hopLength;
      for (int i = 0; i < nFft; i++) {
        frame[i] = pcm[start + i] * _window[i];
      }
      final spectrum = _fft.realFft(frame);
      // Power spectrum, only the first nFft/2 + 1 bins.
      for (int m = 0; m < nMels; m++) {
        double energy = 0;
        final row = _melBank[m];
```

```
        for (int k = 0; k < row.length; k++) {
          final re = spectrum[k].x;
          final im = spectrum[k].y;
          energy += (re * re + im * im) * row[k];
        }
        mel[m][f] = energy;
      }
    }

    // Convert power -> dB with ref = max, top_db = 80 (librosa default).
    double refPower = 1e-10;
    for (final row in mel) {
      for (final v in row) {
        if (v > refPower) refPower = v;
      }
    }
    const double topDb = 80.0;
    final double floor = refPower * math.pow(10, -topDb / 10);
    for (final row in mel) {
      for (int f = 0; f < row.length; f++) {
        final v = row[f] < floor ? floor : row[f];
        row[f] = 10 * math.log(v / refPower) / math.ln10;
      }
    }
    return mel;
  }
}
```

The helpers `_hannWindow` and `_buildMelFilterBank` are standard implementations (a raised-cosine window and the same triangular filter recipe that produces the librosa filter bank). Copy them from the reference implementation in the book's GitHub repo, or reproduce from the formulas in Chapter 4.

Verifying the Dart Mel Against librosa

The Dart pipeline must agree with librosa to within a small tolerance. Implement this one-shot verification check:

1. Record a known clip on the phone, save it as `sample.wav`.
2. Run the Dart Mel pipeline on it in the app, save the resulting spectrogram to a JSON blob.
3. In Python, load the same `sample.wav` and run `librosa.feature.melspectrogram` with identical parameters.
4. Diff element-wise. Root-mean-square difference should be <1 dB.

If the diff is larger, check: (a) sample rate (confirm the recorder is 16 kHz), (b) window function (Hann vs Hamming), (c) filter-bank normalization (librosa's `norm='slaney'` vs none), or (d) dB conversion reference. Fix before Chapter 11 exposes the mismatch as poor accuracy.

11.4 Wiring It All Together

The end-to-end happy path assembles the pieces:

Listing 11.6: End-to-end record→Mel→tensor

```
Future<List<List<List<double>>>> captureMelTensor() async {
  if (!await ensureMicPermission()) return [];   // graceful bail
  final wavPath = await recordThreeSeconds();
  final pcm = await _readWavToFloat32(wavPath);   // -1..+1 range
  final mel = MelExtractor().logMel(pcm);         // [128][~94]

  // Model expects [1, 128, 128, 1]. Pad or centre-crop to 128 frames.
  final tensor = List.generate(1, (_) =>
    List.generate(128, (m) => List.generate(128, (f) {
      if (f >= mel[m].length) return -80.0;      // pad with silence dB
      return mel[m][f];
    }).map((v) => [v]).toList()));
  return tensor;
}
```

This function is what Chapter 11 will call from the Record button's `onPressed` handler. It returns the exact input tensor shape declared in `model_metadata.json`: `[1, 128, 128, 1]`.

11.5 Streaming Inference

The MVP records a single 3-second clip. A v2 could stream continuously, eliminating the need to tap for each bird call. The shift to streaming uses:

Listing 11.7: Streaming inference with a sliding window

```
import 'package:mic_stream/mic_stream.dart';

const int windowSamples = 3 * 16000;      // 3-s window
const int strideSamples = 1 * 16000;      // hop 1 s between inferences

Stream<List<double>> topSpecies(TfLiteRunner model) async* {
  final rolling = <int>[];                // signed 16-bit PCM
  final micStream = await MicStream.microphone(
    sampleRate: 16000, channelConfig: ChannelConfig.CHANNEL_IN_MONO);
  int sinceLastInfer = 0;

  await for (final chunk in micStream!) {
    rolling.addAll(chunk.buffer.asInt16List());
    while (rolling.length > windowSamples) {
      rolling.removeAt(0);                // drop oldest
    }
    sinceLastInfer += chunk.length ~/ 2;
    if (rolling.length == windowSamples && sinceLastInfer >= strideSamples) {
      final pcm = Float32List.fromList(
        rolling.map((s) => s / 32768.0).toList());
      final mel = MelExtractor().logMel(pcm);
      yield model.predict(mel);           // top-k probabilities
      sinceLastInfer = 0;
    }
```

```
  }
}
```

CPU cost is roughly one inference per second. On a mid-range Android device with the compact CNN from Chapter 8, this equals 5

To keep the on-screen result from flickering as consecutive windows disagree, debounce with a short median filter: only accept a new top species when it wins three of the last five inferences.

11.6 Handling Edge Cases

Field recordings often include conditions the training data lacks: silence, wind, speech, or loud noise. Detect and handle these rather than silently misclassifying.

- **Silence.** Compute RMS on the captured PCM; if $<$-40 dBFS, show "No sound detected — try recording closer to the bird." Skip inference entirely.
- **Clipping.** If more than 1% of samples are at ± 1.0 full-scale, show "Too loud — move back from the source." Still run inference (the clip may salvage), but flag confidence as unreliable.
- **Off-domain audio.** A softmax that gives $<$40% to its top class often means the sound is not one of the trained species. Show the top three anyway, but with a "low confidence — may not be a bird" banner.
- **Very short clip.** If the WAV file is $<$2 s (user let go too soon), pad with silence *on the right* to reach 3 s. Padding on the left shifts the call in time and hurts accuracy for calls near the start.

Every one of these checks is a one-line audio-statistic function; ship them behind a toggle in Settings so power users can override.

11.7 Key Takeaways

- The audio pipeline sits between the microphone and the TFLite interpreter — get every parameter right or the model silently misclassifies.
- Request microphone permission in context on the first record tap, never at cold launch. Handle the "permanently denied" branch by opening app settings.
- Record at 16 kHz mono, 16-bit — match `model_metadata.json` from Chapter 8 exactly.

- Compute the log-Mel spectrogram in Dart with `fftea` for the MVP. Verify against librosa to within <1 dB RMS before shipping — the mismatch is silent but catastrophic.
- The hand-off tensor is `[1, 128, 128, 1]`. Pad short clips with silence on the right, not the left.
- Streaming (sliding-window inference) is an option for v2 — one inference per second is cheap on a compact CNN, but re-measure CPU/battery if you swap in a larger model.
- Edge-case guards (silence RMS check, clipping check, low-confidence banner) protect users from confident but wrong answers.

11.8 Exercises

11.1 Record 3 seconds of audio on a real device and save it to a WAV file. Verify with any audio tool that the sample rate is 16 kHz and the bit depth is 16.

11.2 Compute a Mel spectrogram from that recording in Dart using Listing 11.5. Compute the same spectrogram in Python with librosa. Report the element-wise RMS difference.

11.3 Add a live level meter that reads the microphone RMS thirty times per second while recording.

11.4 Modify Listing 11.4 to record five seconds instead of three. Which downstream steps need to change to keep the model tensor shape correct?

11.5 Implement the silence guard as a Dart function that takes a `Float32List` and returns `true` if the RMS is below -40 dBFS.

11.9 Reflection

- Would you compute Mel spectrograms in Dart, in native code, or inside the TFLite model? What would tip the decision?
- How would you extend the app to record and identify multiple birds in a single session — one clip per bird, or one continuous stream?
- The pipeline in this chapter is preprocessing; the model in Chapter 8 is a classifier. If they disagree on which one owns silence-detection, who wins and why?
- If a user reports "the app never identifies anything correctly outdoors," what is the first diagnostic you would run?

Chapter 12

Running TFLite in Flutter

Learning Objectives

- Add the `tflite_flutter` plugin to a Flutter project on both Android and iOS.
- Bundle a `.tflite` model and `labels.txt` as Flutter assets that survive release builds.
- Load the interpreter once at app start and reuse it for every inference.
- Feed the Mel tensor from Chapter 10 through the interpreter and turn the softmax output into a ranked list of species.
- Choose the right hardware delegate (XNNPACK, GPU, NNAPI, Core ML) for the target device.
- Measure inference latency and know which knob to turn if it is too slow.
- Diagnose the two most common inference bugs: tensor-shape mismatch and preprocessing drift.

Chapters 8 and 10 provided everything needed: a compact `.tflite` file, `labels.txt`, and a Dart function producing a `[1, 128, 128, 1]` Mel tensor. This chapter loads the model at startup, runs inference in tens of milliseconds, and converts softmax output to three-name results. The Record button is then fully wired: tap → tensor → inference → species display.

12.1 The `tflite_flutter` Plugin

`tflite_flutter` is Google's official TensorFlow Lite binding for Flutter. It provides pre-built libraries for iOS, Android, macOS, Windows, and Linux. It is the standard option for on-device inference.

Add it (and the helper package for image and audio preprocessing shims we may need later) to `pubspec.yaml`:

Listing 12.1: `pubspec.yaml` entries for TFLite

```
dependencies:
  flutter:
    sdk: flutter
```

```
  tflite_flutter: ^0.10.4          # pin exactly, breaking changes are common
  tflite_flutter_helper_plus: ^0.0.2
```

Then `flutter pub get`. On Android the plugin drops `libtensorflowlite_jni.so` into the APK per architecture (arm64-v8a is what matters for real phones). On iOS the plugin includes a static TensorFlowLiteC framework. No manual Xcode or Gradle wiring is needed.

Verifying the Install

Before touching the model, confirm the plugin loads:

Listing 12.2: Sanity check that the TFLite runtime is available

```dart
import 'package:tflite_flutter/tflite_flutter.dart';

void main() {
  final version = tfl.version;
  print('TFLite runtime version: $version');
}
```

If this prints a version string, you are done with setup. If the build fails with a shared-library error, force-clean and re-fetch: `flutter clean && flutter pub get`.

12.2 Bundling Model Assets

Flutter ships the model file with the app by declaring it in the `assets:` block. The exact same block is needed for `labels.txt` and the `species.json` catalogue we will build in the publishing chapter.

Listing 12.3: `pubspec.yaml` assets block

```
flutter:
  assets:
    - assets/bird_model.tflite
    - assets/labels.txt
    - assets/species.json          # three-name catalogue
```

Impact on APK / IPA Size

Assets are included in the shipped binary verbatim. Table 12.1 shows typical sizes for the ZamZam MVP.

Play Store and App Store both accept apps up to 100 MB without extra steps, so we have room to spare. If a future release grows the model to 20 MB (e.g. a MobileNetV3 backbone), that is still fine.

Table 12.1: Asset sizes for the ZamZam MVP

Asset	Size	Notes
Flutter runtime baseline	15–20 MB	Ships with any Flutter app
`bird_model.tflite` (float16)	360 KB	Compact CNN from Ch 8
`bird_model.tflite` (int8)	185 KB	If you shipped the int8 variant
`labels.txt`	3 KB	100 species, one per line
`species.json`	8 KB	Three-name catalogue
Total for ZamZam MVP	~16 MB	Well under the 100 MB soft cap

12.3 Loading the Model

Load the interpreter once at startup and keep it alive for the process. Loading takes 50–200 ms—too slow for every Record tap.

Listing 12.4: Load the interpreter and labels once at app start

```dart
import 'package:tflite_flutter/tflite_flutter.dart';
import 'package:flutter/services.dart' show rootBundle;

class ZamZamModel {
  static const _modelPath  = 'assets/bird_model.tflite';
  static const _labelsPath = 'assets/labels.txt';

  late final Interpreter _interpreter;
  late final List<String> labels;

  Future<void> load() async {
    // Load the interpreter with default options (XNNPACK on CPU).
    _interpreter = await Interpreter.fromAsset(_modelPath);
    _interpreter.allocateTensors();
    labels = (await rootBundle.loadString(_labelsPath))
        .split('\n')
        .where((s) => s.isNotEmpty)
        .toList();
    // Warm up: one dummy inference triggers XNNPACK's kernel selection.
    _warmup();
  }

  void _warmup() {
    final input  = List.filled(1 * 128 * 128 * 1, 0.0).reshape([1, 128, 128, 1]);
    final output = List.filled(1 * labels.length, 0.0).reshape([1, labels.length]);
    _interpreter.run(input, output);
  }

  Interpreter get interpreter => _interpreter;
}
```

Wire this into your app's `main.dart`, before `runApp`:

Listing 12.5: Load the model before the first frame paints

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final model = ZamZamModel();
  await model.load();
  runApp(ZamZamApp(model: model));
}
```

Pass `model` down through Provider (Chapter 8) so any widget can call `context.read<ZamZamModel>()`.

12.4 Running Inference

Now the useful part. Given the `[1, 128, 128, 1]` tensor produced by `captureMelTensor()` in Chapter 10, feed it to the interpreter and turn the softmax vector into a ranked list of species.

Listing 12.6: End-to-end inference from Mel tensor to species

```
class Prediction {
  final String label;
  final double confidence;
  const Prediction(this.label, this.confidence);
  @override
  String toString() =>
      '${label} (${(confidence * 100).toStringAsFixed(1)}%)';
}

extension Inference on ZamZamModel {
  List<Prediction> predict(List<List<List<List<double>>>> melTensor) {
    // Model produces [1, numClasses] softmax probabilities.
    final output = List.filled(1 * labels.length, 0.0)
        .reshape([1, labels.length]) as List<List<double>>;
    _interpreter.run(melTensor, output);
    final probs = output[0];

    // Rank the top-3 without sorting the whole vector.
    final ranked = List<int>.generate(probs.length, (i) => i)
      ..sort((a, b) => probs[b].compareTo(probs[a]));
    return ranked.take(3).map((i) => Prediction(labels[i], probs[i])).toList();
  }
}
```

The Chapter 8 model outputs softmax probabilities directly, so no manual softmax is needed. If you ever swap in a model that outputs raw logits, apply softmax first:

Listing 12.7: Softmax for logit outputs (skip if the model already softmaxes)

```
List<double> softmax(List<double> logits) {
  final maxL = logits.reduce(math.max);
  final exps = logits.map((v) => math.exp(v - maxL)).toList();
  final sum  = exps.reduce((a, b) => a + b);
  return exps.map((e) => e / sum).toList();
}
```

Wiring Predict to the Record Button

Combine with the audio pipeline from Chapter 10:

Listing 12.8: The Record button's handler, end-to-end

```
Future<List<Prediction>> onRecordPressed(ZamZamModel model) async {
  final mel = await captureMelTensor();  // Ch 10
  if (mel.isEmpty) return const [];      // permission denied
  return model.predict(mel);
}
```

That is the full ZamZam happy path in three function calls.

12.5 Hardware Acceleration

TFLite ships several *delegates* that route inference to specialised hardware. Each has its own trade-offs; Table 12.2 summarises.

Table 12.2: TFLite delegates on mobile

Delegate	Platform	Speed vs CPU	When to use
XNNPACK	Both (default)	Baseline	Always on. Free 30–50% CPU speedup vs older kernels.
GPU	Android + iOS	2–5×	Larger models (>5 MB) or streaming. Adds a few MB to APK.
NNAPI	Android only	2–10×	Recent Android devices with an NPU (Pixel 6+, mid-range 2023+).
Core ML	iOS only	3–8×	Any A12+ iPhone. Uses the Neural Engine.

For ZamZam's compact model, the default XNNPACK CPU delegate delivers 30–50 ms per inference on any phone from the last five years — well under the 500 ms UX target. GPU / NNAPI are worth turning on if you swap in MobileNetV3, MynaNet, or a larger backbone.

Listing 12.9: Enabling delegates on demand

```
InterpreterOptions _optionsForPlatform() {
  final opts = InterpreterOptions()..threads = 2;
  if (Platform.isAndroid) {
    try {
      opts.addDelegate(GpuDelegateV2());   // fall back to CPU if unsupported
```

```
    } on Exception {
      // silent -- CPU path still works
    }
  } else if (Platform.isIOS) {
    try {
      opts.addDelegate(CoreMlDelegate());
    } on Exception {
      // silent
    }
  }
  return opts;
}
// Load with: Interpreter.fromAsset(_modelPath, options: _optionsForPlatform());
```

Wrap every delegate in a `try/catch`. Delegates fail silently on devices that lack the required hardware; the fallback is the CPU path, which is always correct.

12.6 Latency and Troubleshooting

Measuring Latency

Instrument the interpreter once, use it forever. A tight loop of a few hundred inferences on your dev device gives you the numbers you need.

Listing 12.10: Micro-benchmark for inference latency

```
Future<double> measureLatencyMs(ZamZamModel m, int trials) async {
  final mel = List.generate(1, (_) =>
      List.generate(128, (_) =>
      List.generate(128, (_) => List.filled(1, 0.0))));
  // Warm-up (first inference triggers delegate compile).
  m.predict(mel);
  final sw = Stopwatch()..start();
  for (var i = 0; i < trials; i++) {
    m.predict(mel);
  }
  sw.stop();
  return sw.elapsedMicroseconds / trials / 1000.0;
}
```

Table 12.3 gives representative numbers for the Chapter 8 compact CNN across a few devices. Your mileage varies; measure your own targets.

Anything under 200 ms feels instant to the user. If you land above 500 ms the UI needs a spinner; anything above 1 s is a bug.

Troubleshooting Common Bugs

Almost every inference-time bug falls into one of four buckets. Table 12.4 maps each to a fix.

The last row is the killer: TFLite conversion itself rarely breaks accuracy (Chapter

Table 12.3: Typical inference latency for the ZamZam compact CNN (float16)

Device	Best delegate	Latency
Redmi Note 12 (Snapdragon 685, 2023)	XNNPACK CPU	45 ms
Pixel 7 (Tensor G2, 2022)	NNAPI	12 ms
Samsung Galaxy A54 (Exynos 1380)	XNNPACK CPU	38 ms
iPhone SE 3rd gen (A15, 2022)	Core ML	8 ms
iPhone 15 (A17 Pro, 2023)	Core ML	4 ms

Table 12.4: Common TFLite bugs and their fixes

Symptom	Root cause	Fix
`Invalid argument: expected [1,128,128,1] got [128,128,1]`	Missing batch dimension	Wrap the tensor in an outer list
`Op is not supported by the TFLite runtime`	Newer TF ops in the model	Re-export from Ch 8 with older ops, or add `SelectTfOps`
First inference is 500 ms, next ones are 20 ms	Delegate warm-up (JIT + kernel compile)	Run a dummy inference at app start (Listing 12.4)
Accuracy dropped from 82% (Keras) to 3% (device)	Preprocessing mismatch, not TFLite	Compare Dart Mel spectrogram against librosa (Ch 10)

8's verification script would catch that), but a Dart Mel spectrogram that disagrees with librosa by a few dB collapses the model silently. If your device accuracy is way off, the debugger to run is the librosa-diff check from Chapter 10, not the TFLite one.

12.7 Key Takeaways

- With `tflite_flutter`, adding on-device inference is a few dozen lines of Dart — most of the code loads the model, feeds the tensor, and reads the output.
- Bundle the `.tflite` file as a Flutter asset. Load it once at app startup, then reuse the interpreter for every inference.
- Reuse the input and output tensor buffers across inferences to avoid GC pressure on hot paths.
- Enable the GPU/NNAPI delegate on Android and the Core ML delegate on iOS for a 2–$5\times$ speedup, but keep the CPU path as a fallback.
- The two most common bugs are (a) tensor shape mismatch (usually a $[H, W, C]$ vs $[N, H, W, C]$ confusion) and (b) accuracy that diverges from Keras — almost always a preprocessing mismatch, not an interpreter bug.
- The previous chapter gave us the audio pipeline; the next chapter takes the ranked predictions from this chapter and shows how to test them in the field.

12.8 Exercises

12.1 Add the Chapter 8 `bird_model.tflite` to a fresh Flutter project and run inference on an all-zeros input tensor. Report the top-3 softmax values (they will look random — that is fine).

12.2 Extend the app to load a bird-call WAV file bundled as an asset, decode it, run the Chapter 10 Mel pipeline, and print the top species with confidence.

12.3 Measure end-to-end latency (Mel computation + inference) on your test device using Listing 12.10. Report both XNNPACK and (if available) GPU / NNAPI numbers.

12.4 Enable the GPU delegate on Android and measure the speedup versus XNNPACK on your device. Where does the crossover point sit — for what model size is GPU faster than CPU?

12.5 Deliberately break the `captureMelTensor` shape (return `[128, 128, 1]` instead of `[1, 128, 128, 1]`) and observe the error. Which line of the stack trace pointed you at the fix fastest?

12.9 Reflection

- Was the accuracy on your device the same as on your laptop? If not, where did the difference come from — the model or the preprocessing?
- Which hardware delegate gave the best trade-off between speed, size, and battery for your app?
- If your model's inference latency is 200 ms and the librosa-diff check passes, but users complain the app "feels slow," where else in the pipeline might the time be going?
- You get a new phone with an NPU 10× faster than any existing one. Which decisions from Chapters 8 and 10 would you revisit? Which would you keep?

Chapter 13

Field Testing and Usability

Learning Objectives

- Design a field-test protocol that covers the environments ZamZam will actually be used in.
- Measure end-to-end accuracy, latency, and battery consumption on a real phone in a real forest.
- Run a usability test with five real birders and interpret the results.
- Prioritise the bugs and UX issues discovered into a fixable list before Week 10.
- Distinguish issues that only surface in the field from those a unit test could have caught.

This chapter maps to **Week 9** of the intern work plan.

13.1 Why Field Testing is Non-Negotiable

Apps that work in the lab often fail in the field. Wet gear, poor light, one-handed use, wind, and noise expose assumptions that development never challenged. Field testing transforms ZamZam from a demo into a product.

Consumer technology history shows a pattern: products that pass internal testing often fail with real users. Google Glass seemed impressive in staged demos but struggled in daylight and public settings. Early Merlin Sound ID releases underperformed on Southeast Asian species—not from poor algorithms, but because the training data was North American. Early voice assistants struggled with accents uncommon in the development office. The lesson is clear: assumptions that hold in controlled testing collapse when deployment environments differ.

ZamZam faces exactly this risk. It is being built by a small team, in a university lab, with a limited number of test recordings. The Malaysian rainforest at 06:00 is a very different acoustic environment from the lab at 14:00. Field testing is the only way to know whether the model, the app, and the user flow all survive the transition.

What Field Testing Catches That Unit Tests Do Not

Unit tests catch **regressions in known behavior**: they verify functions return expected values. They cannot test what you did not anticipate. Field testing reveals unknown unknowns:

- **Domain shift.** The model was trained on Xeno-Canto recordings made with directional microphones at short range. Real users hold their phone up in a tree canopy at 20 m. The signal-to-noise ratio is different, and accuracy will drop by an amount you cannot predict from Colab metrics alone.
- **Environmental noise the training set never saw.** Cicada choruses in the 6–8 kHz band, motorcycles at 200 m, mosque prayer calls, thunder rumbling under the noise floor. Each is a distribution the classifier has to survive.
- **Ergonomics.** Nobody in the lab records with wet fingers, direct sunlight glare on the screen, or one hand holding binoculars. All three break assumptions about tap size, contrast, and interaction rate.
- **Battery reality.** Simulated inference on a plugged-in phone hides the fact that continuous mic + display + GPS drains a real battery in two hours.
- **The user's mental model.** A birder expects the app to behave like Merlin or eBird. Every unstated deviation costs seconds of confusion — which compound across a session.

Field testing is essential, not optional. These five categories of bugs only appear in the field.

13.2 The Four Environments

Design the test so that between them, the four environments span the acoustic reality of Malaysian birding. Table 13.1 summarises the coverage.

Table 13.1: The four field-test environments and what each stresses.

Environment	Example locations	What it stresses
Quiet forest	Taman Negara, FRIM, Endau-Rompin	Baseline best-case accuracy; low noise floor.
Roadside	Bukit Gasing, Ampang forest reserve edge	Broadband vehicle noise; band-pass and model tolerance.
Urban park	KLCC Park, Taman Tugu, Taman Botani Perdana	Human activity, mixed species, background music.
Dawn chorus	Any of the above at ∼06:00	Simultaneous multi-species vocalisations; the hardest test.

Quiet forest. Taman Negara, FRIM, or a similar low-traffic reserve. Low noise floor, clean calls, small number of species per minute. Baseline for best-case accuracy. If the app fails here it will fail everywhere.

Roadside. Urban forest edge with traffic noise. Broadband noise from vehicles; useful to see how the band-pass and the model tolerate it.

Urban park. KLCC Park, Taman Tugu, Taman Botani Perdana. Mixed birds and human activity (children, music, joggers). Reveals how the model handles non-target sounds it was not trained on.

Dawn chorus. Any of the above at ∼06:00. Multiple species calling simultaneously; the hardest test case for the model.

Record at least 15 minutes in each environment, with at least one team member noting what they hear (species, direction, distance) as ground truth.

Two Recording Modes: Wild and Controlled

Field tests should combine two recording modes:

Wild. The team walks the environment and records whatever birds are actually calling. Ground truth is provided by a birder who identifies each call in real time. Advantages: realistic user experience, real acoustic conditions. Disadvantages: species mix is uncontrolled, some target species may not appear, and the ground truth is only as good as the birder.

Controlled. Play known Xeno-Canto recordings through a Bluetooth speaker in the test environment. Ground truth is exact (you chose the file). Advantages: same 20 clips across all four environments, apples-to-apples comparison. Disadvantages: not what real users will do.

Use both modes. Wild recording shows whether users can succeed; controlled playback isolates environment effects from species mix.

13.3 Metrics to Collect

For each environment, log the metrics in Table 13.2. Consistent per-session logging is the difference between “we tested it” and “we can prove which change fixed the problem”.

The MVP targets are starting points. Adjust once you have baseline numbers from the first session.

Per-Session CSV Template

Every team member fills the same CSV per test session. A minimal schema:

Table 13.2: Field-test metrics, how to measure them, and useful targets for the MVP.

Metric	How to measure	MVP target
Top-1 accuracy	Match against birder ground truth.	≥70% quiet forest.
Top-3 recall	True species in the top-3 shown.	≥85% quiet forest.
Latency (record→result)	Stopwatch or on-screen timing overlay.	<1500 ms.
False-positive rate	30 s of silence, count non-empty results.	<5%.
Battery drain per hour	Phone battery stats, active-use hour.	<15%/h.
Crashes / ANRs per hour	Firebase Crashlytics or manual log.	0 crashes.
Cold-start time	Tap icon to Record button ready.	<2 s.

Listing 13.1: `field_test_log.csv` template

```
session_id,date,time,environment,tester,phone_model,os_version,
app_version,clip_id,ground_truth,top1_pred,top1_conf,top2_pred,
top2_conf,top3_pred,top3_conf,latency_ms,notes
S03,2026-08-10,06:12,dawn_chorus,aiman,Redmi Note 12,A13,
1.0.3,c01,zebra_dove,zebra_dove,0.91,spotted_dove,0.05,
peaceful_dove,0.02,780,clean_recording
S03,2026-08-10,06:14,dawn_chorus,aiman,Redmi Note 12,A13,
1.0.3,c02,coppersmith_barbet,coppersmith_barbet,0.88,
lineated_barbet,0.07,brown_barbet,0.03,820,close_range
S03,2026-08-10,06:16,dawn_chorus,aiman,Redmi Note 12,A13,
1.0.3,c03,pied_fantail,pied_fantail,0.42,common_iora,
0.35,oriental_magpie_robin,0.15,910,two_birds_calling
```

Aggregate at the end of the week: per-species top-1, per-environment top-1, latency distribution, and any species that is consistently confused for another (candidate for targeted retraining in Week 10).

Measuring Latency Reliably

Manual stopwatch timing has a 200–400 ms error floor from human reaction time. Acceptable for rough MVP estimates, but for reliable tracking, instrument the code:

Listing 13.2: Instrumented latency measurement in Dart

```
final t0 = DateTime.now().microsecondsSinceEpoch;
final tensor = await captureMelTensor();
final tCap = DateTime.now().microsecondsSinceEpoch;
final preds = model.predict(tensor);
final tInf = DateTime.now().microsecondsSinceEpoch;

debugPrint('capture: ${(tCap - t0) / 1000} ms');
```

```
debugPrint('inference: ${(tInf - tCap) / 1000} ms');
debugPrint('total: ${(tInf - t0) / 1000} ms');
```

Log these three numbers per record tap. Capture time and inference time drift for different reasons (capture on OS/mic changes, inference on model or delegate changes), so separate columns catch regressions the aggregate would hide.

Measuring Battery Realistically

Android's built-in stats are noisy over short periods. Two approaches give reliable measurements:

- **One-hour active session.** Fully charge, disable auto-brightness, keep the screen at 50% brightness, and use the app continuously for exactly one hour. Read the delta from the battery stats page. Repeat on three days, average.
- **Battery Historian.** For deeper analysis (which subsystems drain what), `adb bugreport` → upload to Battery Historian. Overkill for the MVP but essential if you find drain rates you cannot explain.

Include phone model and Android version in every battery measurement — a 15%/h drain on a Redmi Note 9 is not comparable to a 15%/h drain on a Pixel 8.

13.4 Usability Testing with Five Birders

Test the app in the field, then hand it to non-developers and observe. Nielsen found that five users uncover 80

Recruit five people who match ZamZam's target audience: casual birders, biology students, park rangers. For each session:

1. Explain that you are testing the app, not the user.
2. Ask them to **think aloud** while attempting three tasks:
    - Task 1: Record a bird sound and identify it. (Use a phone playing a bird call if no real bird is available.)
    - Task 2: Look up a previously recorded detection.
    - Task 3: Delete a saved detection, then look up a different one by scrolling the history.
3. Take notes on every hesitation, wrong tap, or misunderstanding.
4. After the session, ask three questions:
    - What did you like?

- What frustrated you?
- What would you change first?

Consent and Recording

Every session needs the participant's explicit consent. A minimal consent script:

> *Hi, thank you for helping test ZamZam. This session will take about 20 minutes. I will ask you to try three tasks in the app and to think aloud as you go. I will take written notes. With your permission, I would like to audio-record the session so I do not miss anything you say — the recording will only be shared with the team, will not be published, and will be deleted after the app ships. You can stop at any time and skip any task without giving a reason. Is that OK?*

Record "Yes" verbally at the start of the audio, or use a one-page written form the participant signs. Store recordings in an encrypted folder and delete after Week 10.

Note-Taking Sheet

Use a consistent template. Table 13.3 shows a minimal per-participant sheet.

Verbatim quotes are the most powerful evidence in the write-up — "I thought the microphone icon meant call, not record" cuts through more arguments than any summary paragraph.

Turning Observations Into Findings

Translate notes into findings within 24 hours while memory is fresh. Each finding is one sentence: what happened, who it affected, and severity. Examples:

- **U03** could not find the history screen; assumed the "Recent" tab was for network updates. (Severity: Blocker; U01 and U04 also hesitated for > 5 s.)
- **U02** tapped the confidence percentage expecting a details page. (Severity: Friction; low-effort fix, high user value.)
- **U05** wanted to share a detection to WhatsApp; feature does not exist. (Severity: Wish; deferred to v2.)

Findings are the raw material for the bug list in the next section.

13.5 Turning Findings Into a Bug List

Group the field-test and usability-test findings into three buckets:

Table 13.3: Usability-test note-taking template.

Field	Notes
Participant ID	U01 (anonymised)
Date / time	
Phone used	Redmi 12 / test-provided Pixel 6a / own device
Task 1 outcome	Completed / partial / abandoned; time to complete
Task 1 obstacles	Where did they hesitate, wrong-tap, misinterpret?
Task 2 outcome	
Task 2 obstacles	
Task 3 outcome	
Task 3 obstacles	
Verbatim quote 1	Their exact words when frustrated / delighted
Verbatim quote 2	
Verbatim quote 3	
Post-session: liked	
Post-session: frustrated	
Post-session: would change	

Blockers Things that stop a user completing a task, cause a crash, or produce dangerously wrong results. Fix in Week 10.

Frictions Things that slow a user down but do not stop them. Fix in Week 10 if trivial; otherwise triage for v2.

Wishes Feature requests. Do not fix in Week 10; add to a public roadmap.

Aim to close every Blocker before submission to the app stores. Track them on a shared board (GitHub Projects, Trello, or a simple spreadsheet) with columns for owner, estimate in hours, and status. A useful rule: any Blocker that costs more than a day to fix probably deserves a v1.1 patch release rather than blocking the v1 submission.

Cross-Referencing Findings

Issues found by multiple testers are stronger signals than unique findings. Group similar findings before triaging:

- If ≥ 2 testers had the same issue $\Rightarrow$ almost certainly a Blocker or high-severity Friction.

- If 1 tester had the issue ⇒ Friction unless the fix is trivial.
- If it happened only in one specific environment / phone model ⇒ note the reproduction conditions.

Unique findings have value, but shared findings get fixed first.

13.6 Deliverables for Week 9

- Field-test report with per-environment metrics table (4 environments × 7 metrics).
- Raw CSV logs from all sessions committed to the repo (`docs/field_tests/`).
- Usability-test findings with the top three issues from each of the five users, plus verbatim quotes.
- Prioritized bug list (Blockers / Frictions / Wishes) with owners and time estimates.
- One-page executive summary for stakeholders that leads with the numbers, not the process.

13.7 Key Takeaways

- Field testing takes the model out of the lab and into the environment it will actually serve — and unlike unit tests, it surfaces bugs you did not know to look for.
- Four environments (quiet forest, roadside, urban park, dawn chorus) span the acoustic reality of Malaysian birding. Test all four.
- Combine wild recordings (realistic UX) and controlled playback (apples-to-apples comparison across environments).
- Log seven metrics per session in a shared CSV: top-1, top-3, latency, false positives, battery, crashes, cold-start.
- Measure latency by instrumenting the code (not a stopwatch); measure battery with fixed screen brightness over a full active hour.
- Recruit five real users per usability round — more users yield diminishing returns; more rounds are better than a single bigger round.
- Consent, verbatim quotes, and same-day write-ups are the three habits that separate a real usability test from a demo.

- Triage findings into Blockers, Frictions, and Wishes with owners and estimates. Only Blockers must clear before release. Shared findings (multiple testers) beat lonely ones for prioritisation.

13.8 Exercises

13.1 Design a fifth test environment relevant to your local area. What acoustic conditions does it test that the four in the chapter do not?

13.2 Draft a one-sentence recruitment message for the five usability testers. What incentive would you offer, and what would you avoid promising?

13.3 In your field test, how would you handle a species that the model consistently misidentifies as a different local species? Design an experiment to distinguish a training-data problem from a preprocessing problem.

13.4 Take one weekend to run the full test protocol on your current build in one environment (start with the quietest). Fill in Table 13.2. Report the three biggest surprises.

13.5 Write a Python script that ingests the CSV in Listing 13.1 and outputs a per-species confusion matrix.

13.9 Reflection

- Was there any Blocker that could have been caught by unit tests instead of a field test? What would that unit test have looked like?
- If a user's phone is running Android 8 (old), would you support them in v1, defer to v2, or drop them? What data would you want before deciding?
- You have time to test either *four environments with two testers each* or *one environment with eight testers*. Which do you pick, and what does that choice imply about your priors?

Bibliography

[1] Jakob Nielsen. *Usability Engineering*. San Diego, CA: Academic Press, 1993. ISBN: 978-0125184069.

Chapter 14

Publishing and Localization

Learning Objectives

- Structure the species catalogue so every bird surfaces its Malay, English, and scientific name in the UI.
- Prepare release builds for the Play Store and the App Store, including signing, icons, and splash screens.
- Understand privacy policy, permissions declarations, and the review process.
- Plan and execute a multi-language UI rollout: English (v1), Malay (v1.1), French (v2).
- Ship updates: model swaps, species-list expansions, bug fixes.

The MVP ships in English only, keeping submission simple. Localization was not deferred by accident—Chapter 10 enforced ARB files so adding Malay and French becomes translation, not refactoring. This chapter covers shipping v1 and planning Malay and French follow-ups.

14.1 Species Names in the Catalogue

Every bird in ZamZam surfaces three names in the UI:

- **Malay common name** in bold as the primary heading (e.g. *Murai Kampung*).
- **English common name** directly beneath (e.g. *Oriental Magpie-Robin*).
- **Scientific name** in italics as a third line (e.g. *Copsychus saularis*).

The catalogue is a JSON asset bundled with the app. Species names are **data, not UI strings**—they live in `assets/species.json`, not ARB files. This distinction matters for French: ARB files translate button labels, while the catalogue adds French common-name fields. Schema:

Listing 14.1: `assets/species.json` schema with room for later languages

```
[
  {
    "id": 42,
```

```
    "scientific": "Copsychus saularis",
    "english":    "Oriental Magpie-Robin",
    "malay":      "Murai Kampung",
    "french":     "Shama dayal"
  },
  {
    "id": 43,
    "scientific": "Pycnonotus jocosus",
    "english":    "Red-whiskered Bulbul",
    "malay":      "Merbah Jambul",
    "french":     "Bulbul orphée"
  }
]
```

The `id` matches the model output index, so the classifier returns an integer and the UI looks up names. Load the file once at startup and keep it in memory—100 species is just kilobytes with all fields.

Which Name is Primary?

The *primary* name shown depends on the current UI locale:

- English UI → English common name is primary; Malay and scientific are secondary.
- Malay UI → Malay common name is primary; English and scientific are secondary.
- French UI → French common name is primary; English and scientific are secondary. (Malay may be shown as a secondary line if the user has explicitly enabled it.)

This is a UI-layer decision; the catalogue always ships all fields. If a French common name is missing for a species, fall back to English rather than showing a blank line — see the fallback strategy in Section 14.2.

Sourcing the Names

Pick each name from a canonical reference so users see what they expect:

- **Malay:** The *Malaysian Nature Society* bird checklist and the *Buku Panduan Burung Malaysia* are authoritative.
- **English:** eBird / Clements taxonomy (updated annually).
- **French:** The Commission internationale des noms français des oiseaux (CINFO) list, or the Avibase entry for each species.
- **Scientific:** Also from eBird/Clements to stay consistent with the English source.

Ship the JSON as `assets/species.json`. Version the catalogue independently of the app (`catalogue_version` field at the top) so a species addition is a data update, not an app update.

14.2 Localizing the UI: Plan for Malay and French

The MVP ships in English. The next two releases add Malay (v1.1, three weeks after launch) and French (v2, three months after launch). Table 14.1 lays out the timeline; the rest of this section explains how each row happens in practice.

Table 14.1: Localization roadmap for ZamZam.

Release	Locale	Scope	Key task
v1.0	`en`	English UI, three-name species results	Ship to stores; validate ARB pipeline.
v1.1	`en, ms`	Add Malay UI + Malay-primary species headings	Translate `app_en.arb` → `app_ms.arb`; add in-app language switcher.
v2.0	`en, ms, fr`	Add French UI + French common names in catalogue	Translate `app_en.arb` → `app_fr.arb`; add `french` field to every species.

The ARB Pipeline (Recap From Chapter 10)

Every user string is a key in `lib/l10n/app_en.arb`. Widgets read them via `AppLocalizations.of(context).<key>`. Adding a language:

1. Copy `app_en.arb` to `app_ms.arb` (or `app_fr.arb`).
2. Translate each string value; leave the `@key` metadata blocks intact so the translator sees the description and placeholder types.
3. Add the locale to `supportedLocales` in `MaterialApp`.
4. Run `flutter gen-l10n`; the widget code needs no changes.

The whole workflow is designed so the developer's job is plumbing (steps 1, 3, 4) and the translator's job is the ARB values. Neither has to touch the other's files.

ARB File Example

Here is what a small slice of the three ARB files looks like once Malay and French are in place:

Listing 14.2: `lib/l10n/app_en.arb`

```
{
  "@@locale": "en",
  "appTitle": "ZamZam",
  "@appTitle": {"description": "App name; do not translate."},

  "recordButton": "Record",
  "@recordButton": {"description": "Main CTA on the home screen."},

  "topKTitle": "Top {k} matches",
  "@topKTitle": {
    "description": "Header above the ranked species list.",
    "placeholders": {"k": {"type": "int"}}
  },

  "confidence": "{pct}% confident",
  "@confidence": {
    "description": "Confidence badge on each species card.",
    "placeholders": {"pct": {"type": "int"}}
  },

  "noSoundDetected": "No sound detected --- try closer.",
  "@noSoundDetected": {"description": "Silence guard message."}
}
```

Listing 14.3: `lib/l10n/app_ms.arb`

```
{
  "@@locale": "ms",
  "appTitle": "ZamZam",
  "recordButton": "Rakam",
  "topKTitle": "{k} padanan teratas",
  "confidence": "{pct}% yakin",
  "noSoundDetected": "Tiada bunyi dikesan --- cuba dekat sedikit."
}
```

Listing 14.4: `lib/l10n/app_fr.arb`

```
{
  "@@locale": "fr",
  "appTitle": "ZamZam",
  "recordButton": "Enregistrer",
  "topKTitle": "{k} meilleures correspondances",
  "confidence": "confiance de {pct}%",
  "noSoundDetected": "Aucun son détecté --- rapprochez-vous."
}
```

Note the placeholder syntax (`{k}`, `{pct}`) is preserved across locales — Flutter substitutes the value at runtime.

Locale Selection and the Language Switcher

Two behaviours matter:

- **Automatic on first launch.** Read `Localizations.localeOf(context)` and

match against `supportedLocales`. A Malaysian phone set to `ms_MY` picks Malay; a French phone picks French; anything else falls back to English.

- **Manual override in Settings.** Some users want the app in English even on a Malay phone. Store the choice in `SharedPreferences` and expose it in Settings → Language, with options *System default*, *English*, *Bahasa Melayu*, *Français*.

Listing 14.5: Locale-aware app root with in-app override

```
class ZamZamApp extends StatelessWidget {
  const ZamZamApp({super.key, required this.overrideLocale});
  final Locale? overrideLocale;

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      onGenerateTitle: (ctx) => AppLocalizations.of(ctx)!.appTitle,
      locale: overrideLocale,                       // null = follow system
      supportedLocales: const [
        Locale('en'), Locale('ms'), Locale('fr'),
      ],
      localizationsDelegates: const [
        AppLocalizations.delegate,
        GlobalMaterialLocalizations.delegate,
        GlobalWidgetsLocalizations.delegate,
        GlobalCupertinoLocalizations.delegate,
      ],
      home: const HomeScreen(),
    );
  }
}
```

Handling Missing Translations

Two kinds of miss:

- **Missing UI string.** If `app_ms.arb` is missing a key, Flutter falls back to `app_en.arb` automatically. That is usually acceptable for a new v1.1 string that has not been translated yet, but track them in an issue so v1.2 completes the coverage.
- **Missing species name.** If `species.json` has a Malay name but no French, the UI helper picks the first non-null field in the order `french` → `english` → `scientific`. Never render a blank line.

Listing 14.6: Species-name fallback helper

```
String primaryName(Species s, Locale l) {
  switch (l.languageCode) {
    case 'ms': return s.malay ?? s.english ?? s.scientific;
    case 'fr': return s.french ?? s.english ?? s.scientific;
    default:   return s.english ?? s.scientific;
  }
}
```

Working with Translators

Both Malay and French are best handled by a human translator, not a machine translation pass. Two practical patterns:

- **ARB round-trip.** Send the translator the `app_en.arb` file and ask them to fill in the values for their target locale. The `@key` metadata blocks (description, placeholder types) give the translator the context they need. Machine-translating first and asking the translator to *correct* is worse than starting from scratch — the translator ends up unpicking bad choices.
- **Screenshots as context.** A word out of context translates badly. "Record" can be a verb (button label) or a noun (database entry). Ship every string with a screenshot of where it appears; the translator's accuracy jumps.

Budget one afternoon of translator time per locale for the v1 string set (roughly 60–80 strings). Species names are a separate task: a birder-translator, not a general translator, should pick the French names.

Testing the Localized Build

Beyond "does it compile":

1. Switch the phone's system language to Bahasa Melayu; the app should follow.
2. Switch to Français; same.
3. Open Settings → Language, override to English; the whole UI should switch immediately without a restart. This exposes any string that was accidentally cached (see the widget-rebuild note in Ch 10).
4. Force a missing-string scenario by removing one key from `app_ms.arb`. Verify the English fallback kicks in without a crash.
5. Screenshot every screen in every locale — French words tend to be 15–25% longer than English, and Malay compound words can be longer still. Buttons that wrap awkwardly need layout tweaks, not translation rework.

Store Listings Per Locale

Play Store and App Store both allow per-locale store listings. When Malay ships in v1.1, add *Bahasa Melayu* listings with a Malay description, translated screenshot captions, and Malay keywords (*burung*, *pengecaman bunyi*, *Malaysia*). The English listing stays as the default. When French ships in v2, repeat for *Français*. This dramatically improves store search ranking in the target locale — browsing Malaysian users see a Malay listing rather than an English one, which is a stronger signal than the UI locale alone.

14.3 App Icons and Branding

Two Flutter plugins remove almost all the platform-specific icon busywork. Add them both as dev dependencies:

Listing 14.7: `pubspec.yaml` for icons and splash

```
dev_dependencies:
  flutter_launcher_icons: ^0.13.1
  flutter_native_splash:  ^2.3.10

flutter_launcher_icons:
  android: true
  ios: true
  image_path: "assets/icon/zamzam_1024.png"    # single 1024x1024 source
  adaptive_icon_background: "#16192C"           # matches HeaderNavy
  adaptive_icon_foreground: "assets/icon/zamzam_foreground.png"

flutter_native_splash:
  color:       "#16192C"
  image:       "assets/icon/zamzam_splash.png"
  android_12:                              # Android 12+ uses its own splash format
    color: "#16192C"
    image: "assets/icon/zamzam_splash_android12.png"
```

Generate all the platform variants in one command:

```
flutter pub run flutter_launcher_icons
flutter pub run flutter_native_splash:create
```

That writes the correct Android `mipmap-*` folders, the iOS `Assets.xcassets/AppIcon.appiconset` with all thirteen required sizes, and the launch storyboard. If Xcode complains about a missing 1024×1024 marketing icon, it is because Apple requires the source PNG to be exactly that size with no alpha channel — re-export and re-run.

14.4 Release Builds

The Flutter arc closes with two commands, one per platform.

Android: App Bundle for the Play Store

Play Store requires an *app bundle* (`.aab`), not a raw APK. It also requires a signed release build; debug builds are rejected. Set up signing once, then `flutter build appbundle` handles the rest.

Listing 14.8: One-time keystore + signing config for Android release

```
# 1. Create the upload keystore (once per app, keep the file safe)
```

```
keytool -genkey -v -keystore ~/zamzam-upload.jks -keyalg RSA \
        -keysize 2048 -validity 10000 -alias upload

# 2. Create android/key.properties (git-ignored)
cat > android/key.properties <<'EOF'
storePassword=<your-store-password>
keyPassword=<your-key-password>
keyAlias=upload
storeFile=/Users/you/zamzam-upload.jks
EOF
```

Then edit `android/app/build.gradle` to load `key.properties` in the `signingConfigs` block (Flutter's docs have the copy-paste snippet). After that, every release build uses the upload key automatically:

```
flutter build appbundle --release
# outputs build/app/outputs/bundle/release/app-release.aab
```

Upload the `.aab` in Play Console under *Internal testing* first. Play Store's *Play App Signing* will re-sign with the final release key; your upload key stays a secret.

iOS: IPA for the App Store

iOS requires an Apple Developer account ($99/year), provisioning profiles, and a build from a Mac. Once the account and Xcode signing team are configured, the build itself is one line:

```
flutter build ipa --release
# outputs build/ios/ipa/zamzam.ipa
```

Upload with `xcrun altool` or Transporter.app to App Store Connect. TestFlight (Apple's internal-testing track) accepts the same `.ipa`; ship there first before going to public review.

Expected Sizes

Both stores compress on top of what Flutter ships. Table 14.2 shows what to expect for the ZamZam MVP; deviations of more than 30% suggest the release build has debug symbols left over or the model file was duplicated across ABIs.

Table 14.2: Expected release-build sizes for the ZamZam MVP

Artefact	Uncompressed	Store download
Android `.aab` (arm64+armeabi-v7a)	22 MB	14 MB
iOS `.ipa`	28 MB	18 MB

Both are well under the 100 MB Play/App-Store cellular-download cap. Adding Malay and French ARB files adds a few kilobytes each — the localization plan does not change these numbers meaningfully. If you need to trim further, an Android *split APK* per ABI cuts another 3–4 MB.

14.5 Store Listings

A store listing is what a potential user sees before installing — it does more marketing work than the app icon itself. Table 14.3 lists the required fields.

Table 14.3: Store listing fields required by the Play Store and App Store.

Field	Format	Notes for ZamZam
App name	30 chars	*ZamZam — Bird ID*
Short description	80 chars (Play only)	Lead with "offline, Malaysian birds".
Full description	4000 chars	Feature list, permissions rationale, model source.
Screenshots	3–8 per device size	Show three-name results; localize captions per store locale.
Feature graphic	1024×500 (Play only)	Icon + tagline; localize for v1.1 (Malay) and v2 (French) listings.
Privacy policy URL	Public URL	Required by both stores; covers mic, GPS if used, no upload.
Age rating	IARC questionnaire	"Everyone / 3+"; no violence, ads, or user-generated content.
Keywords	100 chars (iOS)	Include Malay and (in v2) French bird-name keywords.

Privacy Policy

Both stores require a public privacy-policy URL. For an on-device app that never uploads audio, the policy is short but must be specific:

- What is collected: audio (temporarily), device model + OS version (for crash reports), coarse-grained analytics (if enabled).
- Where it goes: audio never leaves the device; crash reports go to Firebase Crashlytics; analytics is opt-in.
- How long it is kept: audio is deleted immediately after inference; recordings you save to History live only on the device until you delete them.
- Third parties: none for audio; Google Firebase for crash reports.

- User rights: access, deletion, opt-out of analytics.

Host the policy at a stable URL (a GitHub Pages site is fine) and version it. If you add analytics in v1.1, update the policy *before* the update ships, not after.

Screenshot Strategy

Screenshots do 70% of the store-listing conversion work. Follow a template:

1. **Screenshot 1 — The hook.** The Record screen with a clean three-name species result in a card. Overlay a bold caption: "Identify Malaysian birds by sound."
2. **Screenshot 2 — The offline promise.** Same result screen, with a small icon and caption: "Works offline in the forest."
3. **Screenshot 3 — The three names.** Zoomed-in card highlighting the Malay + English + scientific stack.
4. **Screenshot 4 — History.** The saved-detections list.
5. **Screenshot 5 — Settings.** Language switcher visible (matters for v1.1 and v2).

Localize the overlay captions per store locale. For v1.1 the Malay listing gets Malay captions; for v2 the French listing gets French captions. The underlying screenshots may or may not be re-shot in the target locale — shipping localized UI screenshots is more work but converts better.

14.6 After You Submit

Reviews and Rejections

Both stores will reject a first submission for one of a few common reasons. Run through this checklist before hitting *Submit*:

- **Missing permission descriptions.** iOS requires `NSMicrophoneUsageDescription` in `Info.plist` with a specific human-readable reason. Vague strings ("for app functionality") get rejected.
- **Unclear or missing privacy policy.** The URL must be reachable, and the policy must match what the app actually does. A discrepancy (policy says "no analytics" but Firebase Analytics is bundled) is an automatic rejection.
- **Screenshot placeholders.** Screenshots showing debug overlays, Lorem Ipsum text, or the "Recording . . ." state with no result are rejected as "not representative".

- **Crash on cold start.** Reviewers install and open the app cold. If the interpreter load fails silently and the app hangs on the splash screen, that reads as a crash. Test cold start with airplane mode enabled.
- **Metadata mismatch.** Description mentions features that are not in the shipped build (e.g. "streaming inference") → rejection.
- **iOS: minimum deployment target.** Setting iOS 12 as minimum still works, but stores now warn. Set 13.0 or newer unless there is a reason not to.
- **Android: target SDK.** Play Store enforces a rolling minimum target-SDK level (34 as of late 2025). Bump before submitting.

Expect at least one review round on the first submission. Reviewers respond within 24–48 hours (iOS) or 1–7 days (Android). Fix the issue, bump the build number, and resubmit — do *not* argue with the review, even when it looks wrong. Rewriting a permission-use string is faster than an appeal.

Shipping Updates

Once the app is live, updates fall into four categories.

Model Updates

Two options for shipping a new model:

- **Bundled.** Ship the new `.tflite` inside the next app update. Simple, tested, and stores the model in the reviewed binary. This is the default for the MVP.
- **Downloaded at first launch.** Host the model on a CDN; the app checks for a new version and downloads if needed. Enables shipping model updates without a store review. Adds complexity (download progress UI, integrity check, rollback), so defer to v2 or later.

Whichever you choose, version the model file (`bird_model_v2.tflite`) and record the version in analytics — when a v3 report of "worse accuracy" comes in, you want to know which model shipped.

Species-List Additions

Adding a new species is more than a data edit — the model output layer is fixed at training time. Two workflows:

- **Retrain and re-ship.** The clean option: retrain with the new species included, export a new `.tflite`, bundle in the next update. Requires enough training data for the new species.

- **Deferred rare-species handling.** Add the species to `species.json` only, with a UI hint: "This species is not yet in the classifier; report a manual sighting?" Bridges the gap until the retrain.

For every new species, ensure all three (or, in v2, four) name fields are filled — a Play Store user browsing Malay species should not encounter blank cards.

Analytics and Crash Reporting

Instrument from v1 so real-world data guides v1.1 and v2:

- **Firebase Crashlytics** for crashes and ANRs — free, low-friction, well-supported in Flutter.
- **Privacy-preserving analytics** (e.g. Aptabase or a self-hosted PostHog) for event counts: record-tap frequency, average clips per session, feature-flag usage.
- **No PII.** Do not log species names by user, GPS coordinates, or audio. Aggregate everything.
- **Opt-in.** Show a Settings toggle for analytics; default to enabled but honour the user's choice. Some stores require explicit consent flows in certain locales (e.g. EU); build the toggle in v1 even if the flow is not required for Malaysian users.

Responding to User Reviews

Reply to every review in the first weeks after launch. A public reply demonstrates engagement to the next browser and moves the star rating faster than any store-optimization trick. Templates:

- **Bug report.** "Thank you for the report — we have logged this and will investigate. If you can, please email logs to <address>."
- **Feature request.** "Great suggestion. It is on our roadmap for v2; watch for the update."
- **Wrong species report.** "The model gets some species wrong. Please share the recording if possible — it helps us improve the next release."

Never argue with a review, even when the reviewer misunderstands the app. Reply, learn, move on.

14.7 Key Takeaways

- Shipping is where the real work begins — the store submission is a milestone, not the finish line.
- Prepare store assets early: icon, screenshots, feature graphic, privacy policy, and a clear description. Rework late in the cycle is expensive.
- Android and iOS have different signing, review, and release flows. Budget time for both, and expect at least one round of review comments.
- The localization plan is a three-release arc: English (v1), Malay (v1.1), French (v2). The ARB pipeline set up in Ch 10 lets each language ship as a translation task, not a code change.
- Distinguish UI strings (ARB files) from species names (`species.json` catalogue) — they follow different translation workflows.
- Support both automatic locale detection and an explicit in-app language switcher; power users on shared devices need the override.
- Always fall back gracefully: missing ARB key → English; missing species name → English → scientific. Never render a blank line.
- Send translators the ARB files with `@key` metadata plus screenshots for context. Machine translation as a starting point costs more than it saves.
- Localize store listings per locale; a Malay Play Store listing dramatically improves discoverability for Malaysian users.
- Instrument privacy-preserving analytics and crash reporting from v1 so you can act on real-world data.
- Read every review and respond promptly in the first weeks; early feedback shapes the v2 roadmap and Play/App Store ranking.

14.8 Exercises

14.1 Fill in `assets/species.json` for at least 20 Malaysian birds with all three names (Malay, English, scientific). Cite the source for each Malay name.

14.2 Create `lib/l10n/app_ms.arb` by translating every string in `app_en.arb`. Run `flutter gen-l10n` and screenshot every screen with the system language set to Bahasa Melayu.

14.3 Repeat exercise 2 for French (`app_fr.arb`). Note any strings whose French translation runs 20%+ longer and any layout issues that surface.

14.4 Implement the language switcher from Listing 14.5. Verify that switching language in Settings takes effect immediately without an app restart.

14.5 Prepare a set of Play Store screenshots at all required sizes, ensuring the three-name species result is clearly visible.

14.6 Write a one-page privacy policy for ZamZam that covers microphone, storage, and any analytics. Host it at a stable URL.

14.7 Publish the app to the Play Store internal testing track. Report any review feedback and estimate what proportion could have been avoided by the pre-submit checklist in Section 5.

14.8 Draft a Malay store listing (short description, full description, keyword list). Compare it with the English listing — which words changed the most in translation?

14.9 Reflection

- Which was harder — building the app or getting it approved?
- How would you decide when the app is ready to move from internal testing to public release?
- If a user in Sarawak reports poor accuracy for a specific local species, what is your rollout plan for a fix?
- Why ship English v1 first and Malay only in v1.1, rather than Malay from the start? Is there a version of the argument that reverses that order?
- French speakers are a small fraction of ZamZam's likely audience. What makes the v2 French investment worthwhile — if it is?